

wPlot2D

version: 0.1.1

Generated by Doxygen 1.14.0

1 wPlot2D - ECS-Based 2D Plotting Engine	1
1.1 wPlot2D	1
1.1.1 Introduction	1
1.1.2 Features	1
1.1.3 example	1
1.1.4 Links	2
1.1.5 Author	2
2 Namespace Index	3
2.1 Namespace List	3
3 Hierarchical Index	5
3.1 Class Hierarchy	5
4 Class Index	7
4.1 Class List	7
5 File Index	9
5.1 File List	9
6 Namespace Documentation	11
6.1 wEngine Namespace Reference	11
6.1.1 Function Documentation	12
6.1.1.1 getNextComponentTypeID()	12
6.1.1.2 getComponentTypeID()	12
6.2 wPlot2D Namespace Reference	12
6.2.1 Enumeration Type Documentation	13
6.2.1.1 AxisType	13
6.2.1.2 NotchPosition	13
6.2.1.3 TitleAlignment	13
7 Class Documentation	15
7.1 wEngine::AssetManager Class Reference	15
7.1.1 Detailed Description	15
7.1.2 Constructor & Destructor Documentation	16
7.1.2.1 AssetManager() [1/2]	16
7.1.2.2 AssetManager() [2/2]	16
7.1.2.3 ~AssetManager()	16
7.1.3 Member Function Documentation	16
7.1.3.1 operator=()	16
7.1.3.2 LoadFont() [1/2]	16
7.1.3.3 LoadFont() [2/2]	17
7.1.3.4 getFont()	17
7.1.3.5 RemoveFont()	18

7.1.3.6 debugPrintFonts()	18
7.2 wPlot2D::AxisEntity Class Reference	19
7.2.1 Detailed Description	21
7.2.2 Constructor & Destructor Documentation	22
7.2.2.1 AxisEntity()	22
7.2.2.2 ~AxisEntity()	22
7.2.3 Member Function Documentation	22
7.2.3.1 setColor()	22
7.2.3.2 setThickness()	22
7.2.3.3 setArrowSize()	23
7.2.3.4 addTitle() [1/2]	23
7.2.3.5 addTitle() [2/2]	23
7.2.3.6 setTitleFont()	23
7.2.3.7 setTitleCharacterSize()	24
7.2.3.8 setTitleColor()	24
7.2.3.9 setTitleOffset()	24
7.2.3.10 getTitleOffset()	24
7.2.3.11 addNotches()	24
7.2.3.12 setNotchesColor()	25
7.2.3.13 setNotchesThickness()	25
7.2.3.14 setNotchesLength()	25
7.2.3.15 setLabelsFont()	25
7.2.3.16 setLabelsColor()	26
7.2.3.17 getLabelsOffset()	26
7.2.3.18 setLabelsOffset()	26
7.2.3.19 addLabelsOffset()	26
7.2.3.20 setLabelsCharacterSize()	26
7.2.3.21 setLabelsDecimalPlaces()	27
7.2.3.22 setCustomLabels()	27
7.2.3.23 render()	27
7.3 wEngine::ColorComponent Class Reference	28
7.3.1 Detailed Description	29
7.3.2 Constructor & Destructor Documentation	29
7.3.2.1 ColorComponent()	29
7.3.2.2 ~ColorComponent()	30
7.3.3 Member Function Documentation	30
7.3.3.1 getColor()	30
7.3.3.2 setColor()	30
7.3.3.3 debugPrint()	30
7.4 wEngine::Component Class Reference	31
7.4.1 Detailed Description	31
7.4.2 Constructor & Destructor Documentation	31

7.4.2.1 ~Component()	31
7.4.2.2 Component()	32
7.4.3 Member Function Documentation	32
7.4.3.1 enable()	32
7.4.3.2 disable()	32
7.4.3.3 isEnabled()	32
7.4.3.4 setParent()	32
7.4.3.5 getParent()	32
7.5 wPlot2D::DataPlotEntity Class Reference	33
7.5.1 Detailed Description	35
7.5.2 Constructor & Destructor Documentation	35
7.5.2.1 DataPlotEntity()	35
7.5.2.2 ~DataPlotEntity()	36
7.5.3 Member Function Documentation	36
7.5.3.1 getColor()	36
7.5.3.2 getThickness()	36
7.5.3.3 getLineStyle()	36
7.5.3.4 getDashLength()	36
7.5.3.5 getGapLength()	37
7.5.3.6 setColor()	37
7.5.3.7 setThickness()	37
7.5.3.8 setLineStyle()	37
7.5.3.9 setDashLength()	37
7.5.3.10 setGapLength()	38
7.5.3.11 drawDataPlot()	38
7.6 wEngine::DiscontinuityComponent Class Reference	38
7.6.1 Detailed Description	40
7.6.2 Constructor & Destructor Documentation	40
7.6.2.1 DiscontinuityComponent()	40
7.6.2.2 ~DiscontinuityComponent()	40
7.6.3 Member Function Documentation	41
7.6.3.1 getExcludedIntervals()	41
7.6.3.2 addExcludedInterval()	41
7.6.3.3 clearExcludedIntervals()	41
7.6.3.4 isInExcludedInterval()	41
7.6.3.5 debugPrint()	42
7.7 wEngine::Entity Class Reference	42
7.7.1 Detailed Description	43
7.7.2 Constructor & Destructor Documentation	43
7.7.2.1 Entity()	43
7.7.2.2 ~Entity()	43
7.7.3 Member Function Documentation	43

7.7.3.1 getEntityID()	43
7.7.3.2 clearComponents()	44
7.7.3.3 resetEntityIDCounter()	44
7.7.3.4 addComponent()	44
7.7.3.5 removeComponent()	44
7.7.3.6 hasComponent()	45
7.7.3.7 getComponent()	45
7.7.3.8 requireComponent()	45
7.7.3.9 getInterfaceComponent()	46
7.8 wEngine::FontComponent Class Reference	47
7.8.1 Detailed Description	48
7.8.2 Constructor & Destructor Documentation	48
7.8.2.1 FontComponent()	48
7.8.2.2 ~FontComponent()	49
7.8.3 Member Function Documentation	49
7.8.3.1 getFont()	49
7.8.3.2 setFont()	49
7.8.3.3 debugPrint()	49
7.9 wPlot2D::FrameEntity Class Reference	50
7.9.1 Detailed Description	52
7.9.2 Constructor & Destructor Documentation	52
7.9.2.1 FrameEntity()	52
7.9.2.2 ~FrameEntity()	53
7.9.3 Member Function Documentation	53
7.9.3.1 setEnabled()	53
7.9.3.2 isEnabled()	53
7.9.3.3 getFillColor()	53
7.9.3.4 getOutlineColor()	54
7.9.3.5 getThickness()	54
7.9.3.6 getPadding()	54
7.9.3.7 setFillColor()	54
7.9.3.8 setOutlineColor()	54
7.9.3.9 setThickness()	55
7.9.3.10 setPadding()	55
7.9.3.11 update()	55
7.9.3.12 render()	55
7.10 wEngine::FunctionComponent Class Reference	56
7.10.1 Detailed Description	57
7.10.2 Constructor & Destructor Documentation	57
7.10.2.1 FunctionComponent()	57
7.10.2.2 ~FunctionComponent()	58
7.10.3 Member Function Documentation	58

7.10.3.1 calculate()	58
7.10.3.2 debugPrint()	58
7.11 wPlot2D::FunctionEntity Class Reference	59
7.11.1 Detailed Description	61
7.11.2 Constructor & Destructor Documentation	62
7.11.2.1 FunctionEntity()	62
7.11.2.2 ~FunctionEntity()	62
7.11.3 Member Function Documentation	62
7.11.3.1 getPosition()	62
7.11.3.2 getColor()	62
7.11.3.3 getThickness()	63
7.11.3.4 getLineStyle()	63
7.11.3.5 getDashLength()	63
7.11.3.6 getGapLength()	63
7.11.3.7 getOffset()	64
7.11.3.8 getRotation()	64
7.11.3.9 setPosition()	64
7.11.3.10 setColor()	64
7.11.3.11 setThickness()	64
7.11.3.12 setLineStyle()	65
7.11.3.13 setDashLength()	65
7.11.3.14 setGapLength()	65
7.11.3.15 setOffset()	66
7.11.3.16 setRotation()	66
7.11.3.17 setScale()	66
7.11.3.18 addExcludedInterval()	67
7.11.3.19 clearExcludedIntervals()	67
7.11.3.20 alignToYAxis()	67
7.11.3.21 drawFunction()	67
7.12 wPlot2D::GraphicsEntity Class Reference	68
7.12.1 Detailed Description	71
7.12.2 Constructor & Destructor Documentation	72
7.12.2.1 GraphicsEntity()	72
7.12.2.2 ~GraphicsEntity()	72
7.12.3 Member Function Documentation	72
7.12.3.1 getWindow()	72
7.12.3.2 getWindowSize()	73
7.12.3.3 setWindowSize()	73
7.12.3.4 setWindowTitle()	73
7.12.3.5 setBackgroundColor()	73
7.12.3.6 addFont()	73
7.12.3.7 getFont()	74

7.12.3.8	getOrigin()	74
7.12.3.9	setOrigin()	74
7.12.3.10	getScale()	75
7.12.3.11	setScale()	75
7.12.3.12	getOffset()	75
7.12.3.13	setOffset()	76
7.12.3.14	addAxis()	76
7.12.3.15	addTitle() [1/2]	76
7.12.3.16	addTitle() [2/2]	76
7.12.3.17	addFunction()	77
7.12.3.18	addDataPlot()	77
7.12.3.19	addLegend()	77
7.12.3.20	addText() [1/2]	78
7.12.3.21	addText() [2/2]	78
7.12.3.22	addLine()	78
7.12.3.23	saveToFile()	79
7.13	wPlot2D::LabelEntity Class Reference	79
7.13.1	Detailed Description	82
7.13.2	Constructor & Destructor Documentation	82
7.13.2.1	LabelEntity()	82
7.13.2.2	~LabelEntity()	83
7.13.3	Member Function Documentation	83
7.13.3.1	getValue()	83
7.13.3.2	getCharacterSize()	83
7.13.3.3	getDecimalPlaces()	83
7.13.3.4	setFont()	83
7.13.3.5	setLabelText()	84
7.13.3.6	setCharacterSize()	84
7.13.3.7	setDecimalPlaces()	84
7.13.3.8	setCustomLabels()	84
7.13.3.9	usesCustomLabels()	85
7.13.3.10	formatLabel()	85
7.13.3.11	render()	85
7.14	wPlot2D::LegendEntity Class Reference	86
7.14.1	Detailed Description	88
7.14.2	Constructor & Destructor Documentation	89
7.14.2.1	LegendEntity()	89
7.14.2.2	~LegendEntity()	89
7.14.3	Member Function Documentation	89
7.14.3.1	addItem() [1/4]	89
7.14.3.2	addItem() [2/4]	89
7.14.3.3	addItem() [3/4]	90

7.14.3.4 addItem() [4 / 4]	90
7.14.3.5 setFrameEnabled()	90
7.14.3.6 setFrameFillColor()	90
7.14.3.7 setFrameOutlineColor()	91
7.14.3.8 setFrameThickness()	91
7.14.3.9 setPadding()	91
7.14.3.10 setFont()	91
7.14.3.11 setCharacterSize()	92
7.14.3.12 setTextColor()	92
7.14.3.13 render()	92
7.15 wEngine::LengthComponent Class Reference	93
7.15.1 Detailed Description	94
7.15.2 Constructor & Destructor Documentation	94
7.15.2.1 LengthComponent()	94
7.15.2.2 ~LengthComponent()	95
7.15.3 Member Function Documentation	95
7.15.3.1 getLength()	95
7.15.3.2 setLength()	95
7.15.3.3 debugPrint()	95
7.16 wEngine::LineDrawer Class Reference	96
7.16.1 Detailed Description	96
7.16.2 Member Function Documentation	97
7.16.2.1 drawLine()	97
7.16.2.2 drawPolylineRound()	98
7.17 wPlot2D::LineEntity Class Reference	98
7.17.1 Detailed Description	101
7.17.2 Constructor & Destructor Documentation	101
7.17.2.1 LineEntity()	101
7.17.2.2 ~LineEntity()	102
7.17.3 Member Function Documentation	102
7.17.3.1 setColor()	102
7.17.3.2 setThickness()	102
7.17.3.3 getThickness()	102
7.17.3.4 setLineStyle()	102
7.17.3.5 setDashLength()	103
7.17.3.6 setGapLength()	103
7.17.3.7 getStartPoint()	103
7.17.3.8 getEndPoint()	103
7.17.3.9 hasArrow()	104
7.17.3.10 getArrowSize()	104
7.17.3.11 setArrowSize()	104
7.17.3.12 render()	104

7.18 wEngine::LineStyleComponent Class Reference	105
7.18.1 Detailed Description	106
7.18.2 Member Enumeration Documentation	106
7.18.2.1 LineStyle	106
7.18.3 Constructor & Destructor Documentation	107
7.18.3.1 LineStyleComponent()	107
7.18.3.2 ~LineStyleComponent()	107
7.18.4 Member Function Documentation	107
7.18.4.1 getStyle()	107
7.18.4.2 setStyle()	107
7.18.4.3 getDashLength()	108
7.18.4.4 setDashLength()	108
7.18.4.5 getGapLength()	108
7.18.4.6 setGapLength()	108
7.18.4.7 debugPrint()	109
7.19 wEngine::MathUtils Class Reference	109
7.19.1 Detailed Description	109
7.19.2 Member Function Documentation	109
7.19.2.1 linspace()	109
7.20 wPlot2D::NotchEntity Class Reference	110
7.20.1 Detailed Description	112
7.20.2 Constructor & Destructor Documentation	113
7.20.2.1 NotchEntity()	113
7.20.2.2 ~NotchEntity()	113
7.20.3 Member Function Documentation	113
7.20.3.1 render()	113
7.21 wEngine::NotchIntervalComponent Class Reference	114
7.21.1 Detailed Description	115
7.21.2 Constructor & Destructor Documentation	115
7.21.2.1 NotchIntervalComponent()	115
7.21.2.2 ~NotchIntervalComponent()	116
7.21.3 Member Function Documentation	116
7.21.3.1 getInterval()	116
7.21.3.2 setInterval()	116
7.21.3.3 debugPrint()	116
7.22 wEngine::OffsetComponent Class Reference	117
7.22.1 Detailed Description	118
7.22.2 Constructor & Destructor Documentation	118
7.22.2.1 OffsetComponent()	118
7.22.2.2 ~OffsetComponent()	119
7.22.3 Member Function Documentation	119
7.22.3.1 getOffset()	119

7.22.3.2	setOffset()	119
7.22.3.3	addOffset()	119
7.22.3.4	debugPrint()	120
7.23	wEngine::PaddingComponent Class Reference	120
7.23.1	Detailed Description	121
7.23.2	Constructor & Destructor Documentation	122
7.23.2.1	PaddingComponent()	122
7.23.2.2	~PaddingComponent()	122
7.23.3	Member Function Documentation	122
7.23.3.1	setPadding()	122
7.23.3.2	getPadding()	122
7.23.3.3	debugPrint()	122
7.24	wEngine::PathUtils Class Reference	123
7.24.1	Detailed Description	123
7.24.2	Member Function Documentation	124
7.24.2.1	getExecutablePath()	124
7.24.2.2	getExecutableDir()	124
7.25	wEngine::PositionComponent Class Reference	125
7.25.1	Detailed Description	126
7.25.2	Constructor & Destructor Documentation	126
7.25.2.1	PositionComponent()	126
7.25.2.2	~PositionComponent()	127
7.25.3	Member Function Documentation	127
7.25.3.1	getPosition()	127
7.25.3.2	getLastPosition()	127
7.25.3.3	setPosition()	127
7.25.3.4	move()	127
7.25.3.5	debugPrint()	128
7.26	wEngine::RotationComponent Class Reference	128
7.26.1	Detailed Description	129
7.26.2	Constructor & Destructor Documentation	129
7.26.2.1	RotationComponent()	129
7.26.2.2	~RotationComponent()	130
7.26.3	Member Function Documentation	130
7.26.3.1	setAngle()	130
7.26.3.2	getAngle()	130
7.26.3.3	debugPrint()	130
7.27	wEngine::ScaleComponent Class Reference	131
7.27.1	Detailed Description	132
7.27.2	Constructor & Destructor Documentation	132
7.27.2.1	ScaleComponent()	132
7.27.2.2	~ScaleComponent()	133

7.27.3 Member Function Documentation	133
7.27.3.1 <code>getScale()</code>	133
7.27.3.2 <code>setScale()</code>	133
7.27.3.3 <code>debugPrint()</code>	133
7.28 <code>wEngine::ThicknessComponent</code> Class Reference	134
7.28.1 Detailed Description	135
7.28.2 Constructor & Destructor Documentation	135
7.28.2.1 <code>ThicknessComponent()</code>	135
7.28.2.2 <code>~ThicknessComponent()</code>	136
7.28.3 Member Function Documentation	136
7.28.3.1 <code>getThickness()</code>	136
7.28.3.2 <code>setThickness()</code>	136
7.28.3.3 <code>debugPrint()</code>	136
7.29 <code>wPlot2D::TitleEntity</code> Class Reference	137
7.29.1 Detailed Description	139
7.29.2 Constructor & Destructor Documentation	140
7.29.2.1 <code>TitleEntity()</code> [1/2]	140
7.29.2.2 <code>TitleEntity()</code> [2/2]	140
7.29.2.3 <code>~TitleEntity()</code>	140
7.29.3 Member Function Documentation	140
7.29.3.1 <code>getCharacterSize()</code>	140
7.29.3.2 <code>getTextSize()</code>	141
7.29.3.3 <code>setTextColor()</code>	141
7.29.3.4 <code>setOffset()</code>	141
7.29.3.5 <code>setCharacterSize()</code>	141
7.29.3.6 <code>setFont()</code>	141
7.29.3.7 <code>getFrameOutlineColor()</code>	142
7.29.3.8 <code>getFrameFillColor()</code>	142
7.29.3.9 <code>getFrameThickness()</code>	142
7.29.3.10 <code>getPadding()</code>	142
7.29.3.11 <code>isFrameEnabled()</code>	143
7.29.3.12 <code>setFrameEnabled()</code>	143
7.29.3.13 <code>setFrameOutlineColor()</code>	143
7.29.3.14 <code>setFrameFillColor()</code>	143
7.29.3.15 <code>setFrameThickness()</code>	143
7.29.3.16 <code>setPadding()</code>	144
7.29.3.17 <code>render()</code>	144
8 File Documentation	145
8.1 <code>main.cpp</code> File Reference	145
8.1.1 Function Documentation	145
8.1.1.1 <code>main()</code>	145

8.2 wColorComponent.cpp File Reference	146
8.2.1 Detailed Description	146
8.3 wColorComponent.hpp File Reference	146
8.4 wColorComponent.hpp	147
8.5 wDiscontinuityComponent.cpp File Reference	148
8.5.1 Detailed Description	148
8.6 wDiscontinuityComponent.hpp File Reference	149
8.7 wDiscontinuityComponent.hpp	150
8.8 wFontComponent.cpp File Reference	150
8.8.1 Detailed Description	151
8.9 wFontComponent.hpp File Reference	151
8.10 wFontComponent.hpp	152
8.11 wFunctionComponent.cpp File Reference	153
8.11.1 Detailed Description	154
8.12 wFunctionComponent.hpp File Reference	154
8.13 wFunctionComponent.hpp	155
8.14 wLengthComponent.cpp File Reference	155
8.14.1 Detailed Description	156
8.15 wLengthComponent.hpp File Reference	156
8.16 wLengthComponent.hpp	157
8.17 wLineStyleComponent.cpp File Reference	158
8.17.1 Detailed Description	158
8.18 wLineStyleComponent.hpp File Reference	159
8.19 wLineStyleComponent.hpp	160
8.20 wNotchIntervalComponent.cpp File Reference	160
8.20.1 Detailed Description	161
8.21 wNotchIntervalComponent.hpp File Reference	161
8.22 wNotchIntervalComponent.hpp	162
8.23 wOffsetComponent.cpp File Reference	163
8.23.1 Detailed Description	163
8.24 wOffsetComponent.hpp File Reference	164
8.25 wOffsetComponent.hpp	164
8.26 wPaddingComponent.cpp File Reference	165
8.26.1 Detailed Description	166
8.27 wPaddingComponent.hpp File Reference	166
8.28 wPaddingComponent.hpp	167
8.29 wPositionComponent.cpp File Reference	168
8.29.1 Detailed Description	169
8.30 wPositionComponent.hpp File Reference	169
8.31 wPositionComponent.hpp	170
8.32 wRotationComponent.cpp File Reference	170
8.32.1 Detailed Description	171

8.33 wRotationComponent.hpp File Reference	172
8.34 wRotationComponent.hpp	173
8.35 wScaleComponent.cpp File Reference	173
8.35.1 Detailed Description	174
8.36 wScaleComponent.hpp File Reference	174
8.37 wScaleComponent.hpp	175
8.38 wThicknessComponent.cpp File Reference	176
8.38.1 Detailed Description	177
8.39 wThicknessComponent.hpp File Reference	177
8.40 wThicknessComponent.hpp	178
8.41 wComponent.cpp File Reference	178
8.41.1 Detailed Description	179
8.42 wComponent.hpp File Reference	179
8.43 wComponent.hpp	180
8.44 wEntity.cpp File Reference	181
8.44.1 Detailed Description	181
8.45 wEntity.hpp File Reference	181
8.46 wEntity.hpp	183
8.47 wAxisEntity.cpp File Reference	185
8.47.1 Detailed Description	185
8.48 wAxisEntity.hpp File Reference	185
8.49 wAxisEntity.hpp	186
8.50 wDataPlotEntity.cpp File Reference	188
8.50.1 Detailed Description	188
8.51 wDataPlotEntity.hpp File Reference	188
8.52 wDataPlotEntity.hpp	190
8.53 wFrameEntity.cpp File Reference	190
8.53.1 Detailed Description	191
8.54 wFrameEntity.hpp File Reference	191
8.55 wFrameEntity.hpp	192
8.56 wFunctionEntity.cpp File Reference	193
8.56.1 Detailed Description	193
8.57 wFunctionEntity.hpp File Reference	194
8.58 wFunctionEntity.hpp	195
8.59 wGraphicsEntity.cpp File Reference	196
8.59.1 Detailed Description	196
8.60 wGraphicsEntity.hpp File Reference	196
8.61 wGraphicsEntity.hpp	198
8.62 wLabelEntity.cpp File Reference	199
8.62.1 Detailed Description	200
8.63 wLabelEntity.hpp File Reference	200
8.64 wLabelEntity.hpp	201

8.65 wLegendEntity.cpp File Reference	202
8.65.1 Detailed Description	202
8.66 wLegendEntity.hpp File Reference	203
8.67 wLegendEntity.hpp	204
8.68 wLineEntity.cpp File Reference	205
8.68.1 Detailed Description	205
8.69 wLineEntity.hpp File Reference	205
8.70 wLineEntity.hpp	206
8.71 wNotchEntity.cpp File Reference	207
8.71.1 Detailed Description	208
8.72 wNotchEntity.hpp File Reference	208
8.73 wNotchEntity.hpp	209
8.74 wTitleEntity.cpp File Reference	210
8.74.1 Detailed Description	210
8.75 wTitleEntity.hpp File Reference	210
8.76 wTitleEntity.hpp	211
8.77 wAssetManager.cpp File Reference	212
8.77.1 Detailed Description	213
8.78 wAssetManager.hpp File Reference	213
8.79 wAssetManager.hpp	214
8.80 wLineDrawer.cpp File Reference	215
8.80.1 Detailed Description	216
8.81 wLineDrawer.hpp File Reference	216
8.82 wLineDrawer.hpp	217
8.83 wMathUtils.cpp File Reference	217
8.83.1 Detailed Description	218
8.84 wMathUtils.hpp File Reference	218
8.85 wMathUtils.hpp	219
8.86 wPathUtils.cpp File Reference	220
8.86.1 Detailed Description	220
8.87 wPathUtils.hpp File Reference	220
8.88 wPathUtils.hpp	221
8.89 wPlot2D.hpp File Reference	222
8.89.1 Detailed Description	222
8.90 wPlot2D.hpp	222
Index	223

Chapter 1

wPlot2D - ECS-Based 2D Plotting Engine

1.1 wPlot2D

1.1.1 Introduction

[wPlot2D](#) is a lightweight C++ plotting library designed to create clean and customizable 2D visualizations. It provides essential features such as axes, labels, titles, legends, and annotations, while allowing users to export high-quality graphics for reports, teaching, or research. The library is built with a modular design, making it easy to integrate into existing C++ projects. Its focus is on clarity, precision, and reproducibility, providing an accessible tool for academic and scientific work.

1.1.2 Features

- Entity and Component system
- Dynamic component management with type-safe access
- Support for SFML-based rendering

1.1.3 example

```
#include <wPlot2D.hpp>
#include <cmath>

int main( )
{
    // --- Graph creation -----
    auto graph = wPlot2D::GraphicsEntity( );
    graph.setOrigin( { 0.5f, 0.5f } );
    graph.setScale( { 0.1f, 0.1f } );

    // --- Graph title -----
    auto title = graph.addTitle( "wPlot2D", wPlot2D::TitleAlignment::Top );
    title->setFont( graph.getFont( "Inconsolata" ) );
    title->setFrameThickness( 5.0f );
    title->setPadding( { 20.0f, 20.0f } );

    // --- X axis -----
    auto xAxis = graph.addAxis( wPlot2D::AxisType::X_AXIS, { -3.8f, 3.8f } );
    xAxis->addTitle( "x" );
    xAxis->addNotches( 0.5f, wPlot2D::NotchPosition::Center, true );

    // --- Y axis -----
    auto yAxis = graph.addAxis( wPlot2D::AxisType::Y_AXIS, { -3.8f, 3.8f } );
    yAxis->addTitle( "f(x)" );
```

```
    yAxis->addNotches( 0.5f, wPlot2D::NotchPosition::Center, true );

    // --- Function (sinus) -----
    auto fSinus = graph.addFunction( [ ](double x ) { return std::sin( x ); }, -3.3, 3.3 );
    fSinus->setColor( sf::Color::Red );

    // --- Legend -----
    auto legend = graph.addLegend( { 0.2f, 0.8f }, true );
    legend->addItem( "Sinus", fSinus );
    legend->setCharacterSize( 30.0f );
    legend->setFont( graph.getFont( "Courier" ) );

    // --- Save the graph -----
    graph.saveToFile( "../output/Example.png" );

    return 0;
}
```

1.1.4 Links

- [GitHub Repository](#)
- [itch.io Page](#)
- [Project Website](#)

1.1.5 Author

Created by Wilfried Koch.

Copyright © 2025 Wilfried Koch. All rights reserved.

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all namespaces with brief descriptions:

wEngine		11
wPlot2D		12

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

wEngine::AssetManager	15
wEngine::Component	31
wEngine::ColorComponent	28
wEngine::DiscontinuityComponent	38
wEngine::FontComponent	47
wEngine::FunctionComponent	56
wEngine::LengthComponent	93
wEngine::LineStyleComponent	105
wEngine::NotchIntervalComponent	114
wEngine::OffsetComponent	117
wEngine::PaddingComponent	120
wEngine::PositionComponent	125
wEngine::RotationComponent	128
wEngine::ScaleComponent	131
wEngine::ThicknessComponent	134
wEngine::Entity	42
wPlot2D::AxisEntity	19
wPlot2D::DataPlotEntity	33
wPlot2D::FrameEntity	50
wPlot2D::FunctionEntity	59
wPlot2D::GraphicsEntity	68
wPlot2D::LabelEntity	79
wPlot2D::LegendEntity	86
wPlot2D::LineEntity	98
wPlot2D::NotchEntity	110
wPlot2D::TitleEntity	137
wEngine::LineDrawer	96
wEngine::MathUtils	109
wEngine::PathUtils	123

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

wEngine::AssetManager	Manages graphical assets such as fonts for reuse across the application	15
wPlot2D::AxisEntity	Represents a visual axis (X or Y) in a 2D plot with optional notches and title	19
wEngine::ColorComponent	ECS component that holds a color value	28
wEngine::Component	Abstract base class for all ECS components	31
wPlot2D::DataPlotEntity	Entity for plotting raw data points as a connected polyline	33
wEngine::DiscontinuityComponent	ECS component that manages excluded intervals for function plotting	38
wEngine::Entity	Represents an entity in the ECS (Entity-Component System) architecture	42
wEngine::FontComponent	Holds a reference to an SFML font for rendering text	47
wPlot2D::FrameEntity	Entity representing a rectangular frame around content	50
wEngine::FunctionComponent	ECS component that stores a mathematical function $f(x)$	56
wPlot2D::FunctionEntity	Represents a mathematical function as a drawable entity in a 2D plot	59
wPlot2D::GraphicsEntity	Central entity responsible for graphical rendering in wPlot2D	68
wPlot2D::LabelEntity	Represents a textual label or a collection of axis labels	79
wPlot2D::LegendEntity	Represents a legend box that describes functions and data plots	86
wEngine::LengthComponent	ECS component that defines the length of a drawable object	93
wEngine::LineDrawer	Utility class for rendering thick lines and polylines with style support	96
wPlot2D::LineEntity	Entity representing a straight line segment with optional arrowhead	98
wEngine::LineStyleComponent	ESC component that defines the style of a line (solid, dotted, dashed)	105

wEngine::MathUtils	
Provides common mathematical helper functions for plotting and geometry	109
wPlot2D::NotchEntity	
Represents a single tick mark ("notch") on a 2D axis	110
wEngine::NotchIntervalComponent	
ECS component that defines the interval between notches on an axis	114
wEngine::OffsetComponent	
ECS component that defines a logical coordinate offset	117
wEngine::PaddingComponent	
ECS component representing internal padding for UI-like elements	120
wEngine::PathUtils	
Utility class providing static functions for managing executable and resource paths across plat- forms	123
wEngine::PositionComponent	
ECS component storing the position of an entity in 2D space and supports movement tracking	125
wEngine::RotationComponent	
ECS component that stores a rotation angle (in degrees)	128
wEngine::ScaleComponent	
ECS component that defines the scaling factor for an entity in 2D space	131
wEngine::ThicknessComponent	
ECS component that defines the thickness of a drawable object	134
wPlot2D::TitleEntity	
Represents a textual label (typically an axis title or main plot title) in a 2D plot	137

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

main.cpp	145
wColorComponent.cpp	
Implementation of the ColorComponent class	146
wColorComponent.hpp	146
wDiscontinuityComponent.cpp	
Implementation of the DiscontinuityComponent class	148
wDiscontinuityComponent.hpp	149
wFontComponent.cpp	
Implementation of the FontComponent class	150
wFontComponent.hpp	151
wFunctionComponent.cpp	
Implementation of the FunctionComponent class	153
wFunctionComponent.hpp	154
wLengthComponent.cpp	
Implementation of the LengthComponent class	155
wLengthComponent.hpp	156
wLineStyleComponent.cpp	
Implementation of the LineStyleComponent class	158
wLineStyleComponent.hpp	159
wNotchIntervalComponent.cpp	
Implementation of the wNotchIntervalComponent class	160
wNotchIntervalComponent.hpp	161
wOffsetComponent.cpp	
Implementation of the OffsetComponent class	163
wOffsetComponent.hpp	164
wPaddingComponent.cpp	
Implementation of the PaddingComponent class	165
wPaddingComponent.hpp	166
wPositionComponent.cpp	
Implementation of the PositionComponent class	168
wPositionComponent.hpp	169
wRotationComponent.cpp	
Implementation of the RotationComponent class	170
wRotationComponent.hpp	172
wScaleComponent.cpp	
Implementation of the ScaleComponent class	173

wScaleComponent.hpp	174
wThicknessComponent.cpp	
Implementation of the ThicknessComponent class	176
wThicknessComponent.hpp	177
wComponent.cpp	
Implementation of the Component class	178
wComponent.hpp	179
wEntity.cpp	
Implementation of the Entity class	181
wEntity.hpp	181
wAxisEntity.cpp	
Implementation of the AxisEntity class	185
wAxisEntity.hpp	185
wDataPlotEntity.cpp	
Implementation of the DataPlotEntity class	188
wDataPlotEntity.hpp	188
wFrameEntity.cpp	
Implementation of the FrameEntity class	190
wFrameEntity.hpp	191
wFunctionEntity.cpp	
Implementation of the FunctionEntity class	193
wFunctionEntity.hpp	194
wGraphicsEntity.cpp	
Implementation of the GraphicsEntity class	196
wGraphicsEntity.hpp	196
wLabelEntity.cpp	
Implementation of the LabelEntity class	199
wLabelEntity.hpp	200
wLegendEntity.cpp	
Implementation of the LegendEntity class	202
wLegendEntity.hpp	203
wLineEntity.cpp	
Implementation of the LineEntity class	205
wLineEntity.hpp	205
wNotchEntity.cpp	
Implementation of the NotchEntity class	207
wNotchEntity.hpp	208
wTitleEntity.cpp	
Implementation of the TitleEntity class	210
wTitleEntity.hpp	210
wAssetManager.cpp	
Implementation of the AssetManager class	212
wAssetManager.hpp	213
wLineDrawer.cpp	
Implementation of the LineDrawer class	215
wLineDrawer.hpp	216
wMathUtils.cpp	
Implementation of the MathUtils class	217
wMathUtils.hpp	218
wPathUtils.cpp	
Implementation of the PathUtils class	220
wPathUtils.hpp	220
wPlot2D.hpp	
Main public include file for the wPlot2D library	222

Chapter 6

Namespace Documentation

6.1 wEngine Namespace Reference

Classes

- class [AssetManager](#)
Manages graphical assets such as fonts for reuse across the application.
- class [ColorComponent](#)
ECS component that holds a color value.
- class [Component](#)
Abstract base class for all ECS components.
- class [DiscontinuityComponent](#)
ECS component that manages excluded intervals for function plotting.
- class [Entity](#)
Represents an entity in the ECS (Entity-Component System) architecture.
- class [FontComponent](#)
Holds a reference to an SFML font for rendering text.
- class [FunctionComponent](#)
ECS component that stores a mathematical function $f(x)$.
- class [LengthComponent](#)
ECS component that defines the length of a drawable object.
- class [LineDrawer](#)
Utility class for rendering thick lines and polylines with style support.
- class [LineStyleComponent](#)
ESC component that defines the style of a line (solid, dotted, dashed).
- class [MathUtils](#)
Provides common mathematical helper functions for plotting and geometry.
- class [NotchIntervalComponent](#)
ECS component that defines the interval between notches on an axis.
- class [OffsetComponent](#)
ECS component that defines a logical coordinate offset.
- class [PaddingComponent](#)
ECS component representing internal padding for UI-like elements.
- class [PathUtils](#)
Utility class providing static functions for managing executable and resource paths across platforms.
- class [PositionComponent](#)

- *ECS component storing the position of an entity in 2D space and supports movement tracking.*
- class [RotationComponent](#)
ECS component that stores a rotation angle (in degrees).
- class [ScaleComponent](#)
ECS component that defines the scaling factor for an entity in 2D space.
- class [ThicknessComponent](#)
ECS component that defines the thickness of a drawable object.

Functions

- `std::size_t getNextComponentTypeID ()`
- `template<typename ComponentType>
std::size_t getComponentTypeID () noexcept`

6.1.1 Function Documentation

6.1.1.1 getNextComponentTypeID()

```
std::size_t wEngine::getNextComponentTypeID () [inline]
```

6.1.1.2 getComponentTypeID()

```
template<typename ComponentType>  
std::size_t wEngine::getComponentTypeID () [noexcept]
```

6.2 wPlot2D Namespace Reference

Classes

- class [AxisEntity](#)
Represents a visual axis (X or Y) in a 2D plot with optional notches and title.
- class [DataPlotEntity](#)
Entity for plotting raw data points as a connected polyline.
- class [FrameEntity](#)
Entity representing a rectangular frame around content.
- class [FunctionEntity](#)
Represents a mathematical function as a drawable entity in a 2D plot.
- class [GraphicsEntity](#)
Central entity responsible for graphical rendering in wPlot2D.
- class [LabelEntity](#)
Represents a textual label or a collection of axis labels.
- class [LegendEntity](#)
Represents a legend box that describes functions and data plots.
- class [LineEntity](#)
Entity representing a straight line segment with optional arrowhead.
- class [NotchEntity](#)
Represents a single tick mark ("notch") on a 2D axis.
- class [TitleEntity](#)
Represents a textual label (typically an axis title or main plot title) in a 2D plot.

Enumerations

- enum class [AxisType](#) { [X_AXIS](#) , [Y_AXIS](#) }
Enum representing the type of axis to render.
- enum class [NotchPosition](#) { [Center](#) , [Above](#) , [Below](#) }
Enum controlling the visual placement of notches relative to the axis.
- enum class [TitleAlignment](#) { [Top](#) , [Bottom](#) }
Defines the vertical placement of the main plot title.

6.2.1 Enumeration Type Documentation

6.2.1.1 AxisType

```
enum class wPlot2D::AxisType [strong]
```

Enum representing the type of axis to render.

Enumerator

X_AXIS	
Y_AXIS	

6.2.1.2 NotchPosition

```
enum class wPlot2D::NotchPosition [strong]
```

Enum controlling the visual placement of notches relative to the axis.

Enumerator

Center	
Above	
Below	

6.2.1.3 TitleAlignment

```
enum class wPlot2D::TitleAlignment [strong]
```

Defines the vertical placement of the main plot title.

Enumerator

Top	
Bottom	

Chapter 7

Class Documentation

7.1 wEngine::AssetManager Class Reference

Manages graphical assets such as fonts for reuse across the application.

```
#include <wAssetManager.hpp>
```

Public Member Functions

- [AssetManager](#) ()=default
- [AssetManager](#) (const [AssetManager](#) &)=delete
- [AssetManager](#) & operator= (const [AssetManager](#) &)=delete
- [~AssetManager](#) ()=default
- void [LoadFont](#) (const std::string &name, const std::string &fileName)
Loads a font from file and stores it under a given name.
- void [LoadFont](#) (const std::string &name, const std::vector< std::string > &possiblePaths)
Loads a font by trying multiple possible file paths until success.
- sf::Font & [getFont](#) (const std::string &name)
Retrieves a reference to a previously loaded font.
- void [RemoveFont](#) (const std::string &name)
Removes a previously loaded font from memory.
- void [debugPrintFonts](#) () const
Prints the list of loaded fonts to standard output.

7.1.1 Detailed Description

Manages graphical assets such as fonts for reuse across the application.

This class provides a centralized way to load, access, and manage graphical assets, currently supporting fonts via SFML. Assets are identified by string keys and stored internally to avoid reloading them multiple times.

7.1.1.0.1 Key features:

- Load fonts from file and associate them with a name.
- Access loaded fonts via their name.
- Remove fonts from memory when no longer needed.
- Debug printing of loaded assets.

This manager is non-copyable to ensure centralized ownership and avoid accidental duplication of resources.

Note

If an asset fails to load or is accessed without being loaded first, a `std::runtime_error` is thrown.

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.1.2 Constructor & Destructor Documentation

7.1.2.1 AssetManager() [1/2]

```
wEngine::AssetManager::AssetManager () [default]
```

7.1.2.2 AssetManager() [2/2]

```
wEngine::AssetManager::AssetManager (
    const AssetManager & ) [delete]
```

7.1.2.3 ~AssetManager()

```
wEngine::AssetManager::~~AssetManager () [default]
```

7.1.3 Member Function Documentation

7.1.3.1 operator=()

```
AssetManager & wEngine::AssetManager::operator= (
    const AssetManager & ) [delete]
```

7.1.3.2 LoadFont() [1/2]

```
void wEngine::AssetManager::LoadFont (
    const std::string & name,
    const std::string & fileName)
```

Loads a font from file and stores it under a given name.

If successful, the font is stored under the given `name` and can later be retrieved with `getFont ( name )`. If loading fails, an exception is thrown.

Parameters

<i>name</i>	The unique name used to identify the font.
<i>fileName</i>	The path to the font file on disk.

Exceptions

<i>std::runtime_error</i>	if the font cannot be loaded from file.
---------------------------	---

7.1.3.3 LoadFont() [2/2]

```
void wEngine::AssetManager::LoadFont (
    const std::string & name,
    const std::vector< std::string > & possiblePaths)
```

Loads a font by trying multiple possible file paths until success.

This function attempts to load the specified font (*name*) by iterating through several possible paths provided in *possiblePaths*.

It silences SFML's internal error messages during the process to avoid clutter.

- The function stops at the first successful load.
- If none of the paths work, it throws a *std::runtime_error* with the list of paths tested.
- The successfully loaded font is stored in the internal font map.

Parameters

<i>name</i>	Logical name of the font (used as key in the font map).
<i>possiblePaths</i>	List of file paths to try (in order of priority).

Exceptions

<i>std::runtime_error</i>	if all paths fail to load the font.
---------------------------	-------------------------------------

Note

This approach makes font loading robust across different directory structures, such as development (../↔ Resources/Fonts/...) and distribution (../wPlot2D/Resources/Fonts/...).

7.1.3.4 getFont()

```
sf::Font & wEngine::AssetManager::getFont (
    const std::string & name)
```

Retrieves a reference to a previously loaded font.

Parameters

<i>name</i>	The name of the font previously loaded.
-------------	---

Returns

Reference to the corresponding sf::Font object.

Exceptions

<i>std::runtime_error</i>	if the font does not exist.
---------------------------	-----------------------------

7.1.3.5 RemoveFont()

```
void wEngine::AssetManager::RemoveFont (
    const std::string & name)
```

Removes a previously loaded font from memory.

Parameters

<i>name</i>	The name of the font to remove.
-------------	---------------------------------

Exceptions

<i>std::runtime_error</i>	if the font does not exist.
---------------------------	-----------------------------

7.1.3.6 debugPrintFonts()

```
void wEngine::AssetManager::debugPrintFonts () const
```

Prints the list of loaded fonts to standard output.

The documentation for this class was generated from the following files:

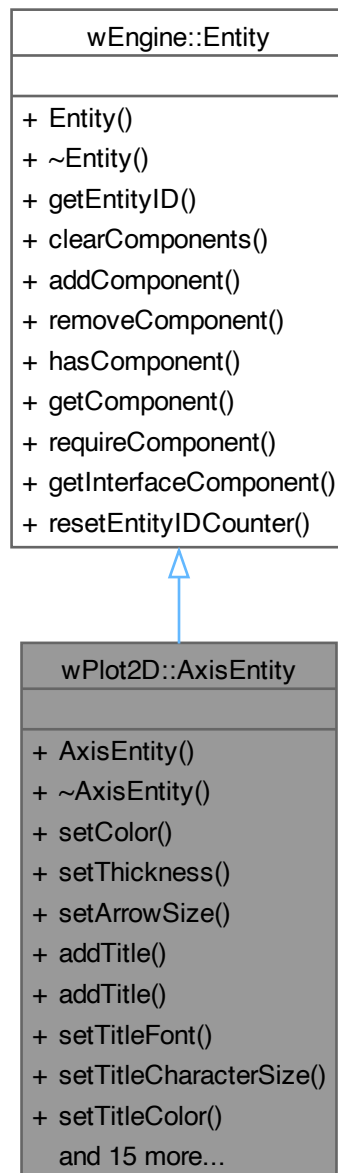
- [wAssetManager.hpp](#)
- [wAssetManager.cpp](#)

7.2 wPlot2D::AxisEntity Class Reference

Represents a visual axis (X or Y) in a 2D plot with optional notches and title.

```
#include <wAxisEntity.hpp>
```

Inheritance diagram for wPlot2D::AxisEntity:



Public Member Functions

- [AxisEntity](#) (sf::Font &font, sf::Vector2f origin, sf::Vector2f scale, sf::Vector2f offset, [AxisType](#) type, sf::Vector2f axisRange)

- Constructs an [AxisEntity](#) with a given orientation, origin, scale, and range.
- virtual [~AxisEntity](#) ()=default

Virtual destructor.
- void [setColor](#) (sf::Color color)

Sets the color of the axis line.
- void [setThickness](#) (float thickness)

Sets the thickness of the axis line (in pixels).
- void [setArrowSize](#) (float arrowSize)

Sets the size of the arrowhead at the end of the axis.
- void [addTitle](#) (const std::string &title)

Adds a title to the axis.
- void [addTitle](#) (const std::wstring &title)

Adds a title to the axis.
- void [setTitleFont](#) (const sf::Font &font)

Sets the font of the axis title.
- void [setTitleCharacterSize](#) (unsigned int size)

Sets the character size of the axis title.
- void [setTitleColor](#) (sf::Color newColor)

Sets the color of the axis title.
- void [setTitleOffset](#) (sf::Vector2f titleOffset)

Sets a manual offset for the title position.
- sf::Vector2f [getTitleOffset](#) () const

Gets the current title offset.
- void [addNotches](#) (float interval, [NotchPosition](#) position, bool hasLabels=false)

Adds notches along the axis.
- void [setNotchesColor](#) (const sf::Color &color)

Sets the color of all notches.
- void [setNotchesThickness](#) (float thickness)

Sets the thickness of all notches.
- void [setNotchesLength](#) (float newLength)

Sets the length of all notches.
- void [setLabelsFont](#) (const sf::Font &font)

Sets the font of all labels.
- void [setLabelsColor](#) (const sf::Color &color)

Sets the color of all labels.
- std::vector< sf::Vector2f > [getLabelsOffset](#) () const

Gets the current offset of all labels.
- void [setLabelsOffset](#) (sf::Vector2f offset)

Sets a new offset for all labels.
- void [addLabelsOffset](#) (sf::Vector2f delta)

Applies an additional offset to all labels.
- void [setLabelsCharacterSize](#) (unsigned int newSize)

Sets the character size of all labels.
- void [setLabelsDecimalPlaces](#) (int places)

Sets the number of decimal places for numeric labels.
- void [setCustomLabels](#) (const std::vector< std::string > &labels)

Replaces numeric labels with a custom set of strings.
- void [render](#) (sf::RenderWindow &window)

Renders the axis (line, arrow, title, notches, labels).

Public Member Functions inherited from [wEngine::Entity](#)

- [Entity](#) ()
- virtual [~Entity](#) ()
- unsigned int [getEntityID](#) () const
Returns the unique ID associated with this entity.
- void [clearComponents](#) ()
Removes all components currently attached to the entity.
- template<typename T, typename... Args>
std::shared_ptr< T > [addComponent](#) (Args &&... args)
Adds a new component of type T to the entity.
- template<typename T>
void [removeComponent](#) ()
Removes the component of type T from the entity.
- template<typename T>
bool [hasComponent](#) () const noexcept
Checks whether the entity has a component of type T.
- template<typename T>
std::shared_ptr< T > [getComponent](#) () const
Retrieves the component of type T attached to the entity.
- template<typename T>
std::shared_ptr< T > [requireComponent](#) (const std::string &context="") const
Retrieves the component of type T and throws if it's missing.
- template<typename Interface>
std::shared_ptr< Interface > [getInterfaceComponent](#) () const
Returns the first component that implements the specified interface.

Additional Inherited Members

Static Public Member Functions inherited from [wEngine::Entity](#)

- static void [resetEntityIDCounter](#) ()
Resets the global entity ID counter to zero.

7.2.1 Detailed Description

Represents a visual axis (X or Y) in a 2D plot with optional notches and title.

This class manages the rendering of a coordinate axis in a Cartesian 2D system. It supports:

- Rendering of an axis line with an arrowhead.
- Adding notches (tick marks) with optional labels.
- Attaching a customizable axis title.

See also

[TitleEntity](#), [LabelEntity](#), [NotchEntity](#), [LineEntity](#)

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.2.2 Constructor & Destructor Documentation

7.2.2.1 AxisEntity()

```
wPlot2D::AxisEntity::AxisEntity (
    sf::Font & font,
    sf::Vector2f origin,
    sf::Vector2f scale,
    sf::Vector2f offset,
    AxisType type,
    sf::Vector2f axisRange)
```

Constructs an [AxisEntity](#) with a given orientation, origin, scale, and range.

Parameters

<i>font</i>	Reference to a font used for the title and labels.
<i>origin</i>	Pixel position of the logical origin (typically from GraphicsEntity).
<i>scale</i>	Scaling factor (pixels per logical unit).
<i>offset</i>	Logical displacement of the axis system.
<i>type</i>	Axis type (X_AXIS or Y_AXIS).
<i>axisRange</i>	Logical range covered by the axis (e.g., [-5, 5]).

7.2.2.2 ~AxisEntity()

```
virtual wPlot2D::AxisEntity::~~AxisEntity () [virtual], [default]
```

Virtual destructor.

7.2.3 Member Function Documentation

7.2.3.1 setColor()

```
void wPlot2D::AxisEntity::setColor (
    sf::Color color)
```

Sets the color of the axis line.

Parameters

<i>color</i>	New axis color.
--------------	-----------------

Exceptions

<i>std::runtime_error</i>	if ColorComponent is missing.
---------------------------	-------------------------------

7.2.3.2 setThickness()

```
void wPlot2D::AxisEntity::setThickness (
    float thickness)
```

Sets the thickness of the axis line (in pixels).

Parameters

<i>thickness</i>	New thickness (must be > 0).
------------------	------------------------------

Exceptions

<i>std::invalid_argument</i>	if thickness <= 0.
<i>std::runtime_error</i>	if ThicknessComponent is missing.

7.2.3.3 setArrowSize()

```
void wPlot2D::AxisEntity::setArrowSize (
    float arrowSize)
```

Sets the size of the arrowhead at the end of the axis.

Parameters

<i>arrowSize</i>	Arrowhead size in pixels.
------------------	---------------------------

7.2.3.4 addTitle() [1/2]

```
void wPlot2D::AxisEntity::addTitle (
    const std::string & title)
```

Adds a title to the axis.

Parameters

<i>title</i>	Title string (narrow string).
--------------	-------------------------------

7.2.3.5 addTitle() [2/2]

```
void wPlot2D::AxisEntity::addTitle (
    const std::wstring & title)
```

Adds a title to the axis.

Parameters

<i>title</i>	Title string (wide string).
--------------	-----------------------------

7.2.3.6 setTitleFont()

```
void wPlot2D::AxisEntity::setTitleFont (
    const sf::Font & font)
```

Sets the font of the axis title.

Parameters

<i>font</i>	Reference to an SFML font.
-------------	----------------------------

7.2.3.7 setTitleCharacterSize()

```
void wPlot2D::AxisEntity::setTitleCharacterSize (
    unsigned int size)
```

Sets the character size of the axis title.

Parameters

<i>size</i>	Character size in pixels.
-------------	---------------------------

7.2.3.8 setTitleColor()

```
void wPlot2D::AxisEntity::setTitleColor (
    sf::Color newColor)
```

Sets the color of the axis title.

Parameters

<i>newColor</i>	New text color.
-----------------	-----------------

7.2.3.9 setTitleOffset()

```
void wPlot2D::AxisEntity::setTitleOffset (
    sf::Vector2f titleOffset)
```

Sets a manual offset for the title position.

Parameters

<i>titleOffset</i>	Pixel offset applied to the title position.
--------------------	---

7.2.3.10 getTitleOffset()

```
sf::Vector2f wPlot2D::AxisEntity::getTitleOffset () const [nodiscard]
```

Gets the current title offset.

Returns

Offset vector in pixels.

7.2.3.11 addNotches()

```
void wPlot2D::AxisEntity::addNotches (
    float interval,
    NotchPosition position,
    bool hasLabels = false)
```

Adds notches along the axis.

Parameters

<i>interval</i>	Logical spacing between notches (> 0).
<i>position</i>	Placement relative to axis (Center, Above, Below).
<i>hasLabels</i>	If true, labels are displayed for each notch.

7.2.3.12 setNotchesColor()

```
void wPlot2D::AxisEntity::setNotchesColor (
    const sf::Color & color)
```

Sets the color of all notches.

Parameters

<i>color</i>	New notch color.
--------------	------------------

7.2.3.13 setNotchesThickness()

```
void wPlot2D::AxisEntity::setNotchesThickness (
    float thickness)
```

Sets the thickness of all notches.

Parameters

<i>thickness</i>	New thickness in pixels.
------------------	--------------------------

7.2.3.14 setNotchesLength()

```
void wPlot2D::AxisEntity::setNotchesLength (
    float newLength)
```

Sets the length of all notches.

Parameters

<i>newLength</i>	New length in pixels.
------------------	-----------------------

7.2.3.15 setLabelsFont()

```
void wPlot2D::AxisEntity::setLabelsFont (
    const sf::Font & font)
```

Sets the font of all labels.

Parameters

<i>font</i>	Reference to an SFML font.
-------------	----------------------------

7.2.3.16 setLabelsColor()

```
void wPlot2D::AxisEntity::setLabelsColor (
    const sf::Color & color)
```

Sets the color of all labels.

Parameters

<i>color</i>	New text color.
--------------	-----------------

7.2.3.17 getLabelsOffset()

```
std::vector< sf::Vector2f > wPlot2D::AxisEntity::getLabelsOffset () const [nodiscard]
```

Gets the current offset of all labels.

Returns

Vector of offsets (one per label).

7.2.3.18 setLabelsOffset()

```
void wPlot2D::AxisEntity::setLabelsOffset (
    sf::Vector2f offset)
```

Sets a new offset for all labels.

Parameters

<i>offset</i>	Offset vector in pixels.
---------------	--------------------------

7.2.3.19 addLabelsOffset()

```
void wPlot2D::AxisEntity::addLabelsOffset (
    sf::Vector2f delta)
```

Applies an additional offset to all labels.

Parameters

<i>delta</i>	Delta offset in pixels.
--------------	-------------------------

7.2.3.20 setLabelsCharacterSize()

```
void wPlot2D::AxisEntity::setLabelsCharacterSize (
    unsigned int newSize)
```

Sets the character size of all labels.

Parameters

<i>newSize</i>	Character size in pixels.
----------------	---------------------------

7.2.3.21 setLabelsDecimalPlaces()

```
void wPlot2D::AxisEntity::setLabelsDecimalPlaces (  
    int places)
```

Sets the number of decimal places for numeric labels.

Parameters

<i>places</i>	Digits after decimal point (≥ 0).
---------------	--

7.2.3.22 setCustomLabels()

```
void wPlot2D::AxisEntity::setCustomLabels (  
    const std::vector< std::string > & labels)
```

Replaces numeric labels with a custom set of strings.

Parameters

<i>labels</i>	Vector of user-defined label strings.
---------------	---------------------------------------

7.2.3.23 render()

```
void wPlot2D::AxisEntity::render (  
    sf::RenderWindow & window)
```

Renders the axis (line, arrow, title, notches, labels).

Parameters

<i>window</i>	Target render window.
---------------	-----------------------

The documentation for this class was generated from the following files:

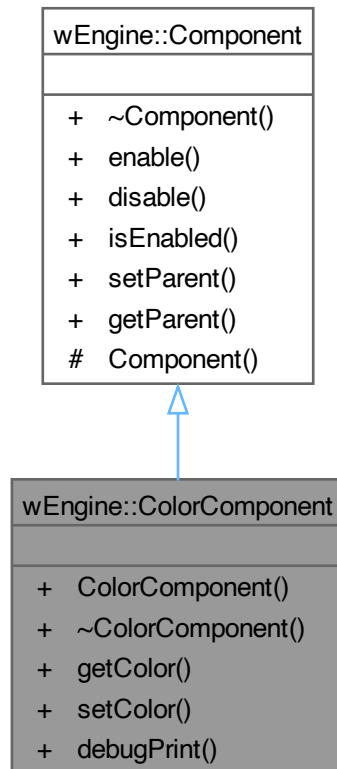
- [wAxisEntity.hpp](#)
- [wAxisEntity.cpp](#)

7.3 wEngine::ColorComponent Class Reference

ECS component that holds a color value.

```
#include <wColorComponent.hpp>
```

Inheritance diagram for wEngine::ColorComponent:



Public Member Functions

- `ColorComponent` (sf::Color color=sf::Color::Black)
Constructor with optional color.
- virtual `~ColorComponent` ()=default
- sf::Color `getColor` () const
Gets the current color.
- void `setColor` (sf::Color newColor)
Sets a new color.
- void `debugPrint` () const
Outputs the RGBA values of the color to standard output.

Public Member Functions inherited from wEngine::Component

- virtual `~Component()`=default
- virtual void `enable()`
- virtual void `disable()`
- bool `isEnabled()` const
Checks whether the component is currently active.
- void `setParent(Entity *parent)`
Sets the parent entity of this component.
- `Entity *` `getParent()` const
Returns the parent entity of this component.

Additional Inherited Members

Protected Member Functions inherited from wEngine::Component

- `Component()`
Protected constructor to restrict instantiation to derived classes.

7.3.1 Detailed Description

ECS component that holds a color value.

This component stores an RGBA color (from SFML) associated with an entity. It can be used to define the rendering color of graphical elements.

7.3.1.0.1 Usage Examples:

- With predefined SFML colors:

```
addComponent< wEngine::ColorComponent >( sf::Color::Red );
```
- With a custom RGBA color:

```
sf::Color customColor(128, 64, 200, 255); // R, G, B, A
addComponent< wEngine::ColorComponent >( customColor );
```

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.3.2 Constructor & Destructor Documentation

7.3.2.1 ColorComponent()

```
wEngine::ColorComponent::ColorComponent (
    sf::Color color = sf::Color::Black)
```

Constructor with optional color.

Parameters

<i>color</i>	The initial color (default is sf::Color::Black).
--------------	--

7.3.2.2 ~ColorComponent()

```
virtual wEngine::ColorComponent::~~ColorComponent () [virtual], [default]
```

7.3.3 Member Function Documentation**7.3.3.1 getColor()**

```
sf::Color wEngine::ColorComponent::getColor () const [nodiscard]
```

Gets the current color.

Returns

The color stored in the component.

7.3.3.2 setColor()

```
void wEngine::ColorComponent::setColor (  
    sf::Color newColor)
```

Sets a new color.

Parameters

<i>newColor</i>	The new color to assign.
-----------------	--------------------------

7.3.3.3 debugPrint()

```
void wEngine::ColorComponent::debugPrint () const
```

Outputs the RGBA values of the color to standard output.

The documentation for this class was generated from the following files:

- [wColorComponent.hpp](#)
- [wColorComponent.cpp](#)

7.4 wEngine::Component Class Reference

Abstract base class for all ECS components.

```
#include <wComponent.hpp>
```

Inheritance diagram for wEngine::Component:



Public Member Functions

- virtual `~Component()`=default
- virtual void `enable()`
- virtual void `disable()`
- bool `isEnabled()` const
Checks whether the component is currently active.
- void `setParent(Entity *parent)`
Sets the parent entity of this component.
- `Entity *getParent()` const
Returns the parent entity of this component.

Protected Member Functions

- `Component()`
Protected constructor to restrict instantiation to derived classes.

7.4.1 Detailed Description

Abstract base class for all ECS components.

Defines the minimal interface required by any component: activation control and access to its owning entity.

Intended to be subclassed by specific component implementations.

Created by Wilfried Koch.

Copyright © 2025 Wilfried Koch. All rights reserved.

7.4.2 Constructor & Destructor Documentation

7.4.2.1 ~Component()

```
virtual wEngine::Component::~~Component () [virtual], [default]
```

7.4.2.2 Component()

```
wEngine::Component::Component () [protected]
```

Protected constructor to restrict instantiation to derived classes.

7.4.3 Member Function Documentation

7.4.3.1 enable()

```
void wEngine::Component::enable () [virtual]
```

7.4.3.2 disable()

```
void wEngine::Component::disable () [virtual]
```

7.4.3.3 isEnabled()

```
bool wEngine::Component::isEnabled () const [nodiscard]
```

Checks whether the component is currently active.

Returns

True if enabled, false if disabled.

7.4.3.4 setParent()

```
void wEngine::Component::setParent (  
    Entity * parent)
```

Sets the parent entity of this component.

Parameters

<i>parent</i>	A pointer to the entity that owns this component.
---------------	---

7.4.3.5 getParent()

```
Entity * wEngine::Component::getParent () const [nodiscard]
```

Returns the parent entity of this component.

Returns

Pointer to the owning [Entity](#).

The documentation for this class was generated from the following files:

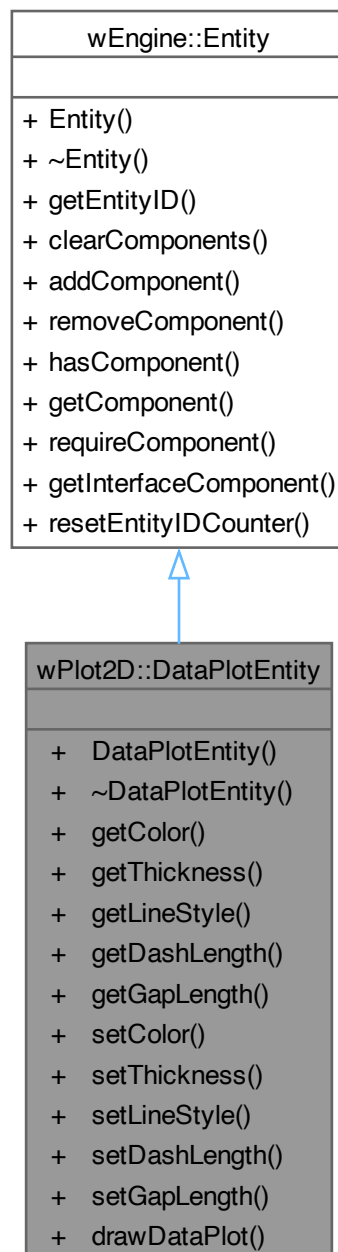
- [wComponent.hpp](#)
- [wComponent.cpp](#)

7.5 wPlot2D::DataPlotEntity Class Reference

Entity for plotting raw data points as a connected polyline.

```
#include <wDataPlotEntity.hpp>
```

Inheritance diagram for wPlot2D::DataPlotEntity:



Public Member Functions

- [DataPlotEntity](#) (const sf::Vector2f origin, const sf::Vector2f scale, const std::vector< sf::Vector2f > &data↵ Points)
Constructs a [DataPlotEntity](#) with given origin, scale, and raw data points.
- virtual [~DataPlotEntity](#) ()=default
Virtual destructor.
- sf::Color [getColor](#) () const
Get the current line color.
- float [getThickness](#) ()
Get the line thickness in pixels.
- [wEngine::LineStyleComponent::LineStyle](#) [getLineStyle](#) ()
Get the current line style.
- float [getDashLength](#) ()
Get the dash length for dashed lines.
- float [getGapLength](#) ()
Get the gap length for dashed/dotted lines.
- void [setColor](#) (sf::Color color)
Sets the color of the plotted line.
- void [setThickness](#) (float thickness)
Sets the line thickness in pixels.
- void [setLineStyle](#) ([wEngine::LineStyleComponent::LineStyle](#) style)
Sets the line style (Solid, Dashed, or Dotted).
- void [setDashLength](#) (float dashLength)
Sets the dash length for dashed lines.
- void [setGapLength](#) (float gapLength)
Sets the gap length between dashes or dots.
- void [drawDataPlot](#) (sf::RenderWindow &window)
Draws the connected data points to the window.

Public Member Functions inherited from [wEngine::Entity](#)

- [Entity](#) ()
- virtual [~Entity](#) ()
- unsigned int [getEntityID](#) () const
Returns the unique ID associated with this entity.
- void [clearComponents](#) ()
Removes all components currently attached to the entity.
- template<typename T, typename... Args>
std::shared_ptr< T > [addComponent](#) (Args &&... args)
Adds a new component of type T to the entity.
- template<typename T>
void [removeComponent](#) ()
Removes the component of type T from the entity.
- template<typename T>
bool [hasComponent](#) () const noexcept
Checks whether the entity has a component of type T.
- template<typename T>
std::shared_ptr< T > [getComponent](#) () const
Retrieves the component of type T attached to the entity.

- `template<typename T>`
`std::shared_ptr< T > requireComponent (const std::string &context="") const`
Retrieves the component of type T and throws if it's missing.
- `template<typename Interface>`
`std::shared_ptr< Interface > getInterfaceComponent () const`
Returns the first component that implements the specified interface.

Additional Inherited Members

Static Public Member Functions inherited from [wEngine::Entity](#)

- static void [resetEntityIDCounter](#) ()
Resets the global entity ID counter to zero.

7.5.1 Detailed Description

Entity for plotting raw data points as a connected polyline.

Stores a vector of raw (x,y) points and draws straight line segments between them. Each segment is rendered using the current line style (Solid, Dashed, Dotted), with configurable color, thickness, dash length, and gap length.

Note

Unlike [FunctionEntity](#), this class does not evaluate a function — it directly uses the provided data points. The points are still transformed by the entity's origin and scale before rendering.

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.5.2 Constructor & Destructor Documentation

7.5.2.1 DataPlotEntity()

```
wPlot2D::DataPlotEntity::DataPlotEntity (
    const sf::Vector2f origin,
    const sf::Vector2f scale,
    const std::vector< sf::Vector2f > & dataPoints)
```

Constructs a [DataPlotEntity](#) with given origin, scale, and raw data points.

Parameters

<i>origin</i>	Origin of the plot in window coordinates.
<i>scale</i>	Scaling factors applied to x and y values.
<i>dataPoints</i>	Vector of raw (x,y) points to plot.

7.5.2.2 ~DataPlotEntity()

```
virtual wPlot2D::DataPlotEntity::~~DataPlotEntity () [virtual], [default]
```

Virtual destructor.

7.5.3 Member Function Documentation

7.5.3.1 getColor()

```
sf::Color wPlot2D::DataPlotEntity::getColor () const [nodiscard]
```

Get the current line color.

Returns

The SFML color used for rendering the polyline.

7.5.3.2 getThickness()

```
float wPlot2D::DataPlotEntity::getThickness () [nodiscard]
```

Get the line thickness in pixels.

Returns

Current thickness value.

7.5.3.3 getLineStyle()

```
wEngine::LineStyleComponent::LineStyle wPlot2D::DataPlotEntity::getLineStyle () [nodiscard]
```

Get the current line style.

Returns

Solid, Dashed, or Dotted.

7.5.3.4 getDashLength()

```
float wPlot2D::DataPlotEntity::getDashLength () [nodiscard]
```

Get the dash length for dashed lines.

Returns

Dash length in pixels.

7.5.3.5 getGapLength()

```
float wPlot2D::DataPlotEntity::getGapLength () [nodiscard]
```

Get the gap length for dashed/dotted lines.

Returns

Gap length in pixels.

7.5.3.6 setColor()

```
void wPlot2D::DataPlotEntity::setColor (
    sf::Color color)
```

Sets the color of the plotted line.

Parameters

<i>color</i>	New SFML color.
--------------	-----------------

7.5.3.7 setThickness()

```
void wPlot2D::DataPlotEntity::setThickness (
    float thickness)
```

Sets the line thickness in pixels.

Parameters

<i>thickness</i>	Line width.
------------------	-------------

7.5.3.8 setLineStyle()

```
void wPlot2D::DataPlotEntity::setLineStyle (
    wEngine::LineStyleComponent::LineStyle style)
```

Sets the line style (Solid, Dashed, or Dotted).

Parameters

<i>style</i>	New line style.
--------------	-----------------

7.5.3.9 setDashLength()

```
void wPlot2D::DataPlotEntity::setDashLength (
    float dashLength)
```

Sets the dash length for dashed lines.

Parameters

<i>dashLength</i>	Length of each dash in pixels.
-------------------	--------------------------------

7.5.3.10 setGapLength()

```
void wPlot2D::DataPlotEntity::setGapLength (
    float gapLength)
```

Sets the gap length between dashes or dots.

Parameters

<i>gapLength</i>	Length of the gap in pixels.
------------------	------------------------------

7.5.3.11 drawDataPlot()

```
void wPlot2D::DataPlotEntity::drawDataPlot (
    sf::RenderWindow & window)
```

Draws the connected data points to the window.

The data points are transformed by scale and origin, then connected with styled line segments using `LineDrawer::drawLine`.

Parameters

<i>window</i>	Target SFML render window.
---------------	----------------------------

The documentation for this class was generated from the following files:

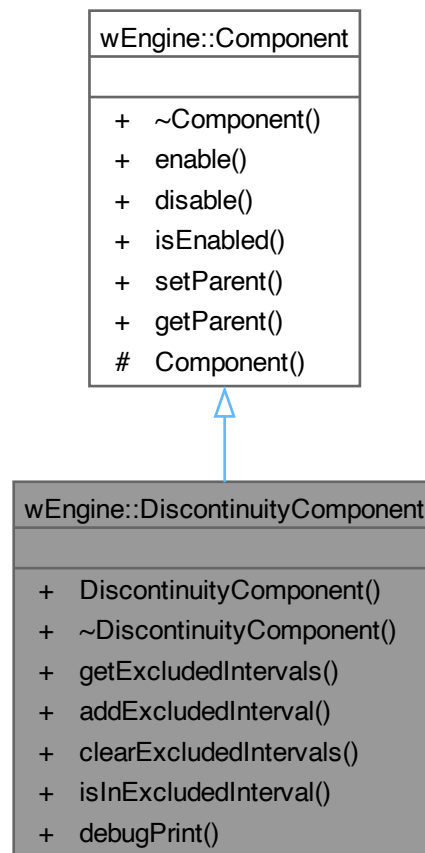
- [wDataPlotEntity.hpp](#)
- [wDataPlotEntity.cpp](#)

7.6 wEngine::DiscontinuityComponent Class Reference

ECS component that manages excluded intervals for function plotting.

```
#include <wDiscontinuityComponent.hpp>
```

Inheritance diagram for wEngine::DiscontinuityComponent:



Public Member Functions

- [DiscontinuityComponent](#) ()=default
- virtual [~DiscontinuityComponent](#) ()=default
- const std::vector< std::pair< double, double > > & [getExcludedIntervals](#) () const
Gives read-only access to the list of excluded intervals.
- void [addExcludedInterval](#) (double min, double max)
Adds an excluded interval to the list.
- void [clearExcludedIntervals](#) ()
Removes all excluded intervals.
- bool [isInExcludedInterval](#) (double x) const
Checks if a value falls into one of the excluded intervals.
- void [debugPrint](#) () const

Public Member Functions inherited from [wEngine::Component](#)

- virtual [~Component](#) ()=default

- virtual void `enable` ()
- virtual void `disable` ()
- bool `isEnabled` () const
Checks whether the component is currently active.
- void `setParent` (Entity *parent)
Sets the parent entity of this component.
- Entity * `getParent` () const
Returns the parent entity of this component.

Additional Inherited Members

Protected Member Functions inherited from `wEngine::Component`

- `Component` ()
Protected constructor to restrict instantiation to derived classes.

7.6.1 Detailed Description

ECS component that manages excluded intervals for function plotting.

This component allows the user to explicitly define intervals of the domain where a function should not be drawn (e.g., around asymptotes or undefined values). During rendering, points falling inside these intervals are skipped to avoid unwanted connections across discontinuities.

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.6.2 Constructor & Destructor Documentation

7.6.2.1 `DiscontinuityComponent()`

```
wEngine::DiscontinuityComponent::DiscontinuityComponent () [default]
```

7.6.2.2 `~DiscontinuityComponent()`

```
virtual wEngine::DiscontinuityComponent::~~DiscontinuityComponent () [virtual], [default]
```


7.6.3 Member Function Documentation

7.6.3.1 getExcludedIntervals()

```
const std::vector< std::pair< double, double > > & wEngine::DiscontinuityComponent::getExcludedIntervals () const [nodiscard]
```

Gives read-only access to the list of excluded intervals.

Returns

A constant reference to the vector of (min, max) pairs.

7.6.3.2 addExcludedInterval()

```
void wEngine::DiscontinuityComponent::addExcludedInterval (
    double min,
    double max)
```

Adds an excluded interval to the list.

Parameters

<i>min</i>	Lower bound of the interval.
<i>max</i>	Upper bound of the interval.

Exceptions

<i>std::invalid_argument</i>	if min >= max.
------------------------------	----------------

7.6.3.3 clearExcludedIntervals()

```
void wEngine::DiscontinuityComponent::clearExcludedIntervals ()
```

Removes all excluded intervals.

7.6.3.4 isInExcludedInterval()

```
bool wEngine::DiscontinuityComponent::isInExcludedInterval (
    double x) const [nodiscard]
```

Checks if a value falls into one of the excluded intervals.

Parameters

<i>x</i>	Value to test.
----------	----------------

Returns

True if x is inside any excluded interval, false otherwise.

Static Public Member Functions

- static void `resetEntityIDCounter ()`
Resets the global entity ID counter to zero.

7.7.1 Detailed Description

Represents an entity in the ECS (Entity-Component System) architecture.

Each entity is uniquely identified and can dynamically manage a collection of components. Components are stored in a type-safe map and accessed by type.

The class provides utility methods to add, remove, retrieve and query components, as well as retrieve components through interfaces.

Note

Entities do not define behavior directly: behavior is defined by the components attached to them.

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.7.2 Constructor & Destructor Documentation

7.7.2.1 Entity()

```
wEngine::Entity::Entity ()
```

7.7.2.2 ~Entity()

```
wEngine::Entity::~Entity () [virtual]
```

7.7.3 Member Function Documentation

7.7.3.1 getEntityID()

```
unsigned int wEngine::Entity::getEntityID () const [nodiscard]
```

Returns the unique ID associated with this entity.

Returns

Unsigned integer representing the entity's ID.

7.7.3.2 clearComponents()

```
void wEngine::Entity::clearComponents ()
```

Removes all components currently attached to the entity.

7.7.3.3 resetEntityIDCounter()

```
void wEngine::Entity::resetEntityIDCounter () [static]
```

Resets the global entity ID counter to zero.

This affects all subsequently created entities. Use with caution, especially in multi-entity systems.

7.7.3.4 addComponent()

```
template<typename T, typename... Args>
std::shared_ptr< T > wEngine::Entity::addComponent (
    Args &&... args) [inline]
```

Adds a new component of type T to the entity.

Constructs the component using the provided arguments and attaches it to the entity.

Template Parameters

<i>T</i>	Component type, must inherit from wEngine::Component .
<i>Args</i>	Variadic arguments used to construct the component.

Parameters

<i>args</i>	Constructor arguments forwarded to the component.
-------------	---

Returns

Shared pointer to the newly created component.

Exceptions

<i>std::runtime_error</i>	if a component of the same type already exists in the entity.
---------------------------	---

7.7.3.5 removeComponent()

```
template<typename T>
void wEngine::Entity::removeComponent () [inline]
```

Removes the component of type T from the entity.

If no such component exists, this operation does nothing.

Template Parameters

<i>T</i>	Component type to remove.
----------	---

7.7.3.6 hasComponent()

```
template<typename T>
bool wEngine::Entity::hasComponent () const [inline], [nodiscard], [noexcept]
```

Checks whether the entity has a component of type T.

Template Parameters

<i>T</i>	Component type to check.
----------	--

Returns

True if the component is present, false otherwise.

7.7.3.7 GetComponent()

```
template<typename T>
std::shared_ptr< T > wEngine::Entity::GetComponent () const [inline], [nodiscard]
```

Retrieves the component of type T attached to the entity.

Template Parameters

<i>T</i>	Component type to retrieve.
----------	---

Returns

Shared pointer to the component if found, or nullptr otherwise.

7.7.3.8 requireComponent()

```
template<typename T>
std::shared_ptr< T > wEngine::Entity::requireComponent (
    const std::string & context = "") const [inline], [nodiscard]
```

Retrieves the component of type T and throws if it's missing.

This method is similar to [GetComponent\(\)](#), but throws a `std::runtime_error` if the component is not found. Useful for critical systems where components must be present.

Template Parameters

<i>T</i>	The type of the component.
----------	----------------------------

Parameters

<i>context</i>	Optional string to specify the context of the call (e.g., method name).
----------------	---

Returns

A shared pointer to the required component.

Exceptions

<i>std::runtime_error</i>	if the component is not found.
---------------------------	--------------------------------

7.7.3.9 getInterfaceComponent()

```
template<typename Interface>
std::shared_ptr< Interface > wEngine::Entity::getInterfaceComponent () const [inline], [nodiscard]
```

Returns the first component that implements the specified interface.

Checks all components attached to the entity using dynamic casting. If a component matches the given interface type, it is returned.

Template Parameters

<i>Interface</i>	The desired interface type.
------------------	-----------------------------

Returns

A shared pointer to the matching component, or nullptr if none found.

The documentation for this class was generated from the following files:

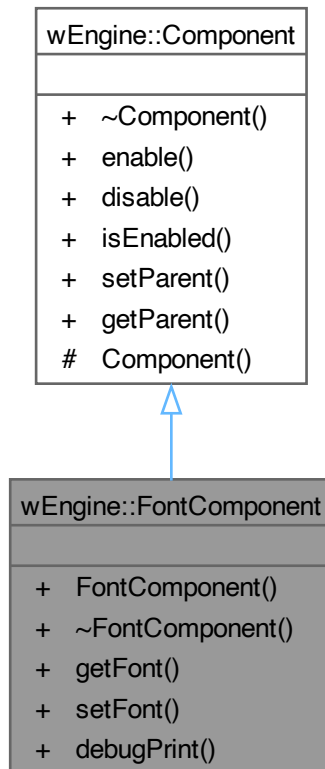
- [wEntity.hpp](#)
- [wEntity.cpp](#)

7.8 wEngine::FontComponent Class Reference

Holds a reference to an SFML font for rendering text.

```
#include <wFontComponent.hpp>
```

Inheritance diagram for wEngine::FontComponent:



Public Member Functions

- `FontComponent` (const sf::Font &font)
Constructs the `FontComponent` with a reference to the font.
- `~FontComponent` () override=default
Virtual destructor.
- const sf::Font & `getFont` () const
Returns the stored font reference.
- void `setFont` (const sf::Font &font)
Updates the stored font reference.
- void `debugPrint` () const
Prints debug information about the stored font.

Public Member Functions inherited from [wEngine::Component](#)

- virtual [~Component](#) ()=default
- virtual void [enable](#) ()
- virtual void [disable](#) ()
- bool [isEnabled](#) () const
Checks whether the component is currently active.
- void [setParent](#) ([Entity](#) *parent)
Sets the parent entity of this component.
- [Entity](#) * [getParent](#) () const
Returns the parent entity of this component.

Additional Inherited Members

Protected Member Functions inherited from [wEngine::Component](#)

- [Component](#) ()
Protected constructor to restrict instantiation to derived classes.

7.8.1 Detailed Description

Holds a reference to an SFML font for rendering text.

This component allows entities to store and access an `sf::Font` reference without needing to pass the [AssetManager](#) explicitly.

Note

The font must outlive the entity that uses it.

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.8.2 Constructor & Destructor Documentation

7.8.2.1 FontComponent()

```
wEngine::FontComponent::FontComponent (  
    const sf::Font & font) [explicit]
```

Constructs the [FontComponent](#) with a reference to the font.

Parameters

<i>font</i>	Reference to an externally managed sf::Font.
-------------	--

7.8.2.2 ~FontComponent()

```
wEngine::FontComponent::~~FontComponent () [override], [default]
```

Virtual destructor.

7.8.3 Member Function Documentation

7.8.3.1 getFont()

```
const sf::Font & wEngine::FontComponent::getFont () const
```

Returns the stored font reference.

Returns

A constant reference to the sf::Font.

7.8.3.2 setFont()

```
void wEngine::FontComponent::setFont (
    const sf::Font & font)
```

Updates the stored font reference.

Parameters

<i>font</i>	Reference to an externally managed sf::Font.
-------------	--

Note

The font must outlive this component.

7.8.3.3 debugPrint()

```
void wEngine::FontComponent::debugPrint () const
```

Prints debug information about the stored font.

The documentation for this class was generated from the following files:

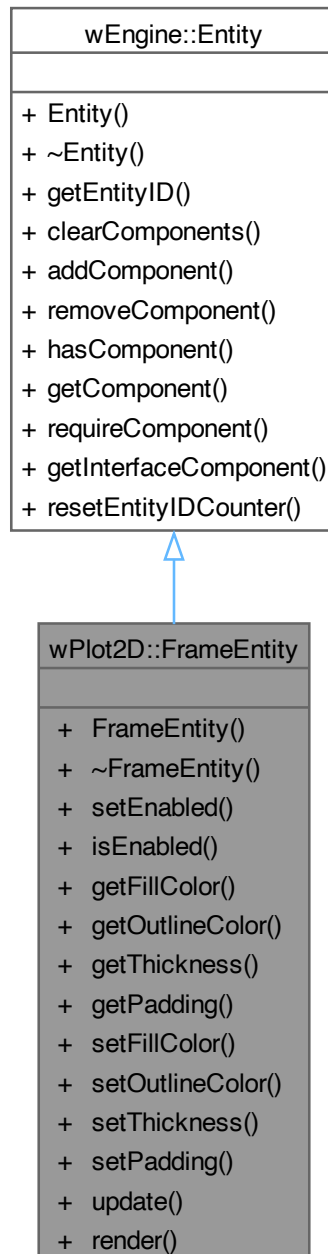
- [wFontComponent.hpp](#)
- [wFontComponent.cpp](#)

7.9 wPlot2D::FrameEntity Class Reference

Entity representing a rectangular frame around content.

```
#include <wFrameEntity.hpp>
```

Inheritance diagram for wPlot2D::FrameEntity:



Public Member Functions

- [FrameEntity](#) (bool enabled=true)
Constructs a frame entity.
- virtual [~FrameEntity](#) ()=default
Virtual destructor.
- void [setEnabled](#) (bool enabled)
Enables or disables the frame.
- bool [isEnabled](#) () const
Checks whether the frame is currently enabled.
- sf::Color [getFillColor](#) () const
Gets the current fill color of the frame.
- sf::Color [getOutlineColor](#) () const
Gets the current outline color of the frame.
- float [getThickness](#) () const
Gets the current outline thickness of the frame.
- sf::Vector2f [getPadding](#) () const
Gets the current padding applied around the content.
- void [setFillColor](#) (const sf::Color &color)
Sets the fill color of the frame.
- void [setOutlineColor](#) (const sf::Color &color)
Sets the outline color of the frame.
- void [setThickness](#) (float thickness)
Sets the outline thickness of the frame.
- void [setPadding](#) (const sf::Vector2f &padding)
Sets the padding around the content.
- void [update](#) (const sf::FloatRect &contentBounds, const sf::Vector2f &position)
Updates the size and position of the frame based on content bounds.
- void [render](#) (sf::RenderWindow &window)
Renders the frame to the given render window.

Public Member Functions inherited from [wEngine::Entity](#)

- [Entity](#) ()
- virtual [~Entity](#) ()
- unsigned int [getEntityID](#) () const
Returns the unique ID associated with this entity.
- void [clearComponents](#) ()
Removes all components currently attached to the entity.
- template<typename T, typename... Args>
std::shared_ptr< T > [addComponent](#) (Args &&... args)
Adds a new component of type T to the entity.
- template<typename T>
void [removeComponent](#) ()
Removes the component of type T from the entity.
- template<typename T>
bool [hasComponent](#) () const noexcept
Checks whether the entity has a component of type T.
- template<typename T>
std::shared_ptr< T > [getComponent](#) () const
Retrieves the component of type T attached to the entity.

- `template<typename T>`
`std::shared_ptr< T > requireComponent (const std::string &context= "") const`
*Retrieves the component of type *T* and throws if it's missing.*
- `template<typename Interface>`
`std::shared_ptr< Interface > getInterfaceComponent () const`
Returns the first component that implements the specified interface.

Additional Inherited Members

Static Public Member Functions inherited from [wEngine::Entity](#)

- static void [resetEntityIDCounter](#) ()
Resets the global entity ID counter to zero.

7.9.1 Detailed Description

Entity representing a rectangular frame around content.

A [FrameEntity](#) is typically used to visually surround titles, legends, or other graphical content. It supports:

- Toggle visibility (`enabled` flag),
- Fill and outline colors,
- Outline thickness,
- Padding around the content.

The size of the frame is dynamically updated from the content bounds (see [update \(\)](#)).

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.9.2 Constructor & Destructor Documentation

7.9.2.1 FrameEntity()

```
wPlot2D::FrameEntity::FrameEntity (
    bool enabled = true)
```

Constructs a frame entity.

Parameters

<i>enabled</i>	Whether the frame should be enabled initially.
----------------	--

7.9.2.2 ~FrameEntity()

```
virtual wPlot2D::FrameEntity::~~FrameEntity () [virtual], [default]
```

Virtual destructor.

7.9.3 Member Function Documentation

7.9.3.1 setEnabled()

```
void wPlot2D::FrameEntity::setEnabled (  
    bool enabled)
```

Enables or disables the frame.

Parameters

<i>enabled</i>	New enabled state.
----------------	--------------------

7.9.3.2 isEnabled()

```
bool wPlot2D::FrameEntity::isEnabled () const [nodiscard]
```

Checks whether the frame is currently enabled.

Returns

True if the frame is enabled, false otherwise.

7.9.3.3 getFillColor()

```
sf::Color wPlot2D::FrameEntity::getFillColor () const [nodiscard]
```

Gets the current fill color of the frame.

Returns

Fill color.

7.9.3.4 getOutlineColor()

```
sf::Color wPlot2D::FrameEntity::getOutlineColor () const [nodiscard]
```

Gets the current outline color of the frame.

Returns

Outline color.

7.9.3.5 getThickness()

```
float wPlot2D::FrameEntity::getThickness () const [nodiscard]
```

Gets the current outline thickness of the frame.

Returns

Outline thickness (in pixels).

7.9.3.6 getPadding()

```
sf::Vector2f wPlot2D::FrameEntity::getPadding () const [nodiscard]
```

Gets the current padding applied around the content.

Returns

Padding as (x, y).

7.9.3.7 setFillColor()

```
void wPlot2D::FrameEntity::setFillColor (
    const sf::Color & color)
```

Sets the fill color of the frame.

Parameters

<i>color</i>	New fill color.
--------------	-----------------

7.9.3.8 setOutlineColor()

```
void wPlot2D::FrameEntity::setOutlineColor (
    const sf::Color & color)
```

Sets the outline color of the frame.

Parameters

<i>color</i>	New outline color.
--------------	--------------------

7.9.3.9 setThickness()

```
void wPlot2D::FrameEntity::setThickness (
    float thickness)
```

Sets the outline thickness of the frame.

Parameters

<i>thickness</i>	New outline thickness (must be > 0).
------------------	--------------------------------------

7.9.3.10 setPadding()

```
void wPlot2D::FrameEntity::setPadding (
    const sf::Vector2f & padding)
```

Sets the padding around the content.

Parameters

<i>padding</i>	Padding as (x, y).
----------------	--------------------

7.9.3.11 update()

```
void wPlot2D::FrameEntity::update (
    const sf::FloatRect & contentBounds,
    const sf::Vector2f & position)
```

Updates the size and position of the frame based on content bounds.

Parameters

<i>contentBounds</i>	The bounding box of the content (width/height).
<i>position</i>	The position of the frame's center in pixels.

7.9.3.12 render()

```
void wPlot2D::FrameEntity::render (
    sf::RenderWindow & window)
```

Renders the frame to the given render window.

Parameters

<i>window</i>	Target render window.
---------------	-----------------------

The documentation for this class was generated from the following files:

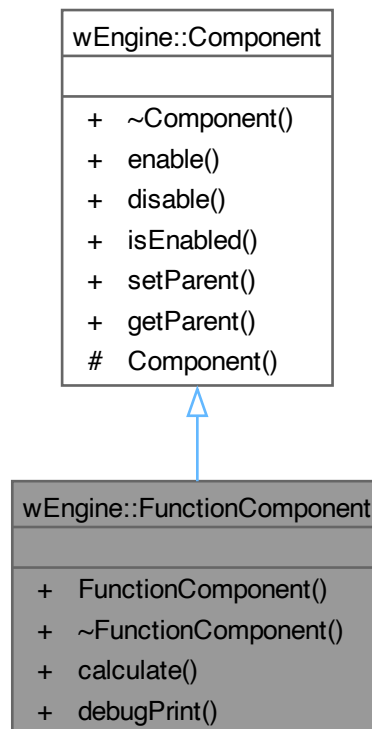
- [wFrameEntity.hpp](#)
- [wFrameEntity.cpp](#)

7.10 wEngine::FunctionComponent Class Reference

ECS component that stores a mathematical function $f(x)$.

```
#include <wFunctionComponent.hpp>
```

Inheritance diagram for wEngine::FunctionComponent:



Public Member Functions

- [FunctionComponent](#) (std::function< double(double) > function)
Constructs a [FunctionComponent](#) with a given function.
- virtual [~FunctionComponent](#) ()=default
- double [calculate](#) (double x) const
Evaluates the stored function at a given x.
- void [debugPrint](#) () const

Public Member Functions inherited from wEngine::Component

- virtual `~Component()`=default
- virtual void `enable()`
- virtual void `disable()`
- bool `isEnabled()` const
Checks whether the component is currently active.
- void `setParent(Entity *parent)`
Sets the parent entity of this component.
- `Entity *` `getParent()` const
Returns the parent entity of this component.

Additional Inherited Members

Protected Member Functions inherited from wEngine::Component

- `Component()`
Protected constructor to restrict instantiation to derived classes.

7.10.1 Detailed Description

ECS component that stores a mathematical function $f(x)$.

This component wraps a `std::function< double( double ) >` and provides an interface to evaluate the function at any given x-coordinate. It is mainly used by FunctionEntity to render mathematical curves.

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.10.2 Constructor & Destructor Documentation

7.10.2.1 FunctionComponent()

```
wEngine::FunctionComponent::FunctionComponent (
    std::function< double(double) > function)
```

Constructs a `FunctionComponent` with a given function.

Parameters

<i>function</i>	A callable object of type <code>double(double)</code> .
-----------------	---

Exceptions

<code>std::invalid_argument</code>	if the provided function is empty.
------------------------------------	------------------------------------

7.10.2.2 ~FunctionComponent()

```
virtual wEngine::FunctionComponent::~~FunctionComponent () [virtual], [default]
```

7.10.3 Member Function Documentation

7.10.3.1 calculate()

```
double wEngine::FunctionComponent::calculate (  
    double x) const [nodiscard]
```

Evaluates the stored function at a given x.

Parameters

<code>x</code>	The input value.
----------------	------------------

Returns

The result $f(x)$.

Exceptions

<code>std::runtime_error</code>	if no function is set.
---------------------------------	------------------------

7.10.3.2 debugPrint()

```
void wEngine::FunctionComponent::debugPrint () const
```

The documentation for this class was generated from the following files:

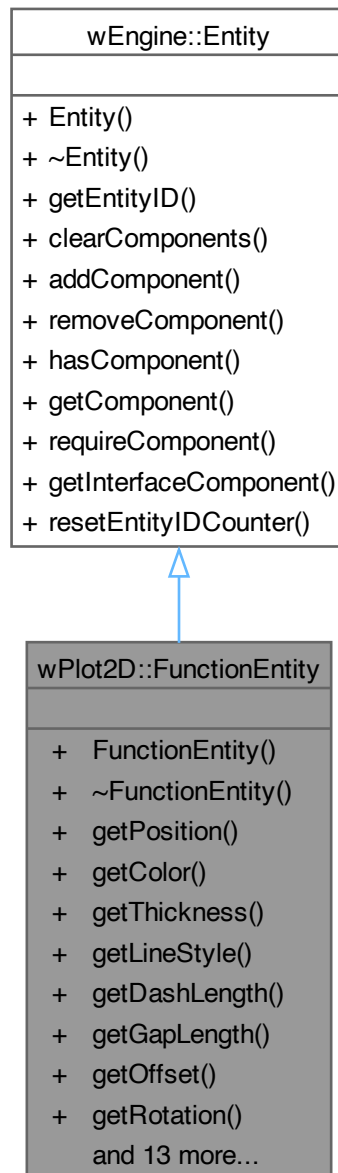
- [wFunctionComponent.hpp](#)
- [wFunctionComponent.cpp](#)

7.11 wPlot2D::FunctionEntity Class Reference

Represents a mathematical function as a drawable entity in a 2D plot.

```
#include <wFunctionEntity.hpp>
```

Inheritance diagram for wPlot2D::FunctionEntity:



Public Member Functions

- [FunctionEntity](#) (const sf::Vector2f origin, const sf::Vector2f scale, std::function< double(double) > func)

- Construct a new [FunctionEntity](#).
- virtual [~FunctionEntity](#) ()=default
Virtual destructor.
- sf::Vector2f [getPosition](#) () const
Get the position (origin) of the function in pixel space.
- sf::Color [getColor](#) () const
Get the color of the function curve.
- float [getThickness](#) () const
Get the line thickness of the function curve.
- wEngine::LineStyleComponent::LineStyle [getLineStyle](#) () const
Get the line style of the function curve.
- float [getDashLength](#) () const
Get the dash length when the line style is Dashed.
- float [getGapLength](#) () const
Get the gap length between dashes or dots.
- sf::Vector2f [getOffset](#) () const
Get the current offset applied to the function curve.
- float [getRotation](#) () const
Get the current rotation angle of the function curve.
- void [setPosition](#) (sf::Vector2f position)
Set the position (origin) of the function in pixel space.
- void [setColor](#) (sf::Color color)
Set the color of the function curve.
- void [setThickness](#) (float thickness)
Set the line thickness of the function curve.
- void [setLineStyle](#) (wEngine::LineStyleComponent::LineStyle style)
Set the line style of the function curve.
- void [setDashLength](#) (float dashLength)
Set the length of each dash when the line style is Dashed.
- void [setGapLength](#) (float gapLength)
Set the length of the gap between dashes or dots.
- void [setOffset](#) (float offsetX, float offsetY)
Set an offset applied to the function curve.
- void [setRotation](#) (float angleDegrees)
Set the rotation angle of the function curve.
- void [setScale](#) (sf::Vector2f scale)
Sets the scaling factors for the function graph.
- void [addExcludedInterval](#) (double min, double max)
Add an excluded interval where the function should not be drawn.
- void [clearExcludedIntervals](#) ()
Clear all excluded intervals.
- void [alignToYAxis](#) (float normalizedOffsetX=0.0f, float normalizedOffsetY=0.0f)
Rotates the function by 90 degrees and swaps scales accordingly.
- void [drawFunction](#) (sf::RenderWindow &>window, double startX, double endX, size_t nbPoints=1000)
Draw the function on the target window.

Public Member Functions inherited from wEngine::Entity

- [Entity](#) ()
- virtual [~Entity](#) ()
- unsigned int [getEntityID](#) () const
Returns the unique ID associated with this entity.
- void [clearComponents](#) ()
Removes all components currently attached to the entity.
- template<typename T, typename... Args>
std::shared_ptr< T > [addComponent](#) (Args &&... args)
Adds a new component of type T to the entity.
- template<typename T>
void [removeComponent](#) ()
Removes the component of type T from the entity.
- template<typename T>
bool [hasComponent](#) () const noexcept
Checks whether the entity has a component of type T.
- template<typename T>
std::shared_ptr< T > [getComponent](#) () const
Retrieves the component of type T attached to the entity.
- template<typename T>
std::shared_ptr< T > [requireComponent](#) (const std::string &context="") const
Retrieves the component of type T and throws if it's missing.
- template<typename Interface>
std::shared_ptr< Interface > [getInterfaceComponent](#) () const
Returns the first component that implements the specified interface.

Additional Inherited Members

Static Public Member Functions inherited from wEngine::Entity

- static void [resetEntityIDCounter](#) ()
Resets the global entity ID counter to zero.

7.11.1 Detailed Description

Represents a mathematical function as a drawable entity in a 2D plot.

A [FunctionEntity](#) manages all components required to render a curve:

- Origin and scale (mapping logical space to pixels).
- Color, thickness, and line style (solid, dashed, dotted).
- Offset and rotation of the curve.
- Discontinuities handled by excluded intervals.

The function is sampled at evenly spaced x-values and rendered as a polyline. Excluded intervals and invalid values (NaN, Inf) split the curve into separate segments.

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.11.2 Constructor & Destructor Documentation

7.11.2.1 FunctionEntity()

```
wPlot2D::FunctionEntity::FunctionEntity (
    const sf::Vector2f origin,
    const sf::Vector2f scale,
    std::function< double(double) > func)
```

Construct a new [FunctionEntity](#).

Parameters

<i>origin</i>	Logical origin of the coordinate system (pixels).
<i>scale</i>	Scale factors for x and y (pixels per unit).
<i>func</i>	Function of type double(double) to be plotted.

7.11.2.2 ~FunctionEntity()

```
virtual wPlot2D::FunctionEntity::~~FunctionEntity () [virtual], [default]
```

Virtual destructor.

7.11.3 Member Function Documentation

7.11.3.1 getPosition()

```
sf::Vector2f wPlot2D::FunctionEntity::getPosition () const [nodiscard]
```

Get the position (origin) of the function in pixel space.

Returns

The current origin as an sf::Vector2f.

7.11.3.2 getColor()

```
sf::Color wPlot2D::FunctionEntity::getColor () const [nodiscard]
```

Get the color of the function curve.

Returns

The current curve color.

7.11.3.3 getThickness()

```
float wPlot2D::FunctionEntity::getThickness () const [nodiscard]
```

Get the line thickness of the function curve.

Returns

The thickness in pixels.

7.11.3.4 getLineStyle()

```
wEngine::LineStyleComponent::LineStyle wPlot2D::FunctionEntity::getLineStyle () const [nodiscard]
```

Get the line style of the function curve.

Returns

The current line style (Solid, Dashed, or Dotted).

7.11.3.5 getDashLength()

```
float wPlot2D::FunctionEntity::getDashLength () const [nodiscard]
```

Get the dash length when the line style is Dashed.

Returns

Dash length in pixels.

Note

This value has no effect if the style is not Dashed.

7.11.3.6 getGapLength()

```
float wPlot2D::FunctionEntity::getGapLength () const [nodiscard]
```

Get the gap length between dashes or dots.

Returns

Gap length in pixels.

Note

This setting affects both Dashed and Dotted line styles.

7.11.3.7 getOffset()

```
sf::Vector2f wPlot2D::FunctionEntity::getOffset () const [nodiscard]
```

Get the current offset applied to the function curve.

Returns

A 2D vector containing the (x,y) offset in pixels.

7.11.3.8 getRotation()

```
float wPlot2D::FunctionEntity::getRotation () const [nodiscard]
```

Get the current rotation angle of the function curve.

Returns

The rotation angle in degrees.

7.11.3.9 setPosition()

```
void wPlot2D::FunctionEntity::setPosition (
    sf::Vector2f position)
```

Set the position (origin) of the function in pixel space.

Parameters

<i>position</i>	The new origin as an sf::Vector2f.
-----------------	------------------------------------

7.11.3.10 setColor()

```
void wPlot2D::FunctionEntity::setColor (
    sf::Color color)
```

Set the color of the function curve.

Parameters

<i>color</i>	The new color as an sf::Color.
--------------	--------------------------------

7.11.3.11 setThickness()

```
void wPlot2D::FunctionEntity::setThickness (
    float thickness)
```

Set the line thickness of the function curve.

Parameters

<i>thickness</i>	The new line thickness in pixels.
------------------	-----------------------------------

7.11.3.12 setLineStyle()

```
void wPlot2D::FunctionEntity::setLineStyle (
    wEngine::LineStyleComponent::LineStyle style)
```

Set the line style of the function curve.

Parameters

<i>style</i>	The new style (Solid, Dashed, or Dotted).
--------------	---

7.11.3.13 setDashLength()

```
void wPlot2D::FunctionEntity::setDashLength (
    float dashLength)
```

Set the length of each dash when the line style is Dashed.

Parameters

<i>dashLength</i>	The dash length in pixels (must be > 0).
-------------------	--

Exceptions

<i>std::invalid_argument</i>	if dashLength <= 0.
------------------------------	---------------------

Note

This setting has no effect if the line style is not Dashed.

7.11.3.14 setGapLength()

```
void wPlot2D::FunctionEntity::setGapLength (
    float gapLength)
```

Set the length of the gap between dashes or dots.

Parameters

<i>gapLength</i>	The gap length in pixels (must be >= 0).
------------------	--

Exceptions

<code>std::invalid_argument</code>	if <code>gapLength < 0</code> .
------------------------------------	------------------------------------

Note

This setting affects both Dashed and Dotted line styles.

7.11.3.15 **setOffset()**

```
void wPlot2D::FunctionEntity::setOffset (
    float offsetX,
    float offsetY)
```

Set an offset applied to the function curve.

The offset is applied after scaling and rotation, allowing the curve to be shifted horizontally and vertically relative to its logical origin.

Parameters

<i>offsetX</i>	Horizontal offset in pixels.
<i>offsetY</i>	Vertical offset in pixels.

7.11.3.16 **setRotation()**

```
void wPlot2D::FunctionEntity::setRotation (
    float angleDegrees)
```

Set the rotation angle of the function curve.

The rotation is applied around the logical origin of the graph. The angle is expressed in degrees, with positive values corresponding to counter-clockwise rotation.

Parameters

<i>angleDegrees</i>	Rotation angle in degrees.
---------------------	----------------------------

7.11.3.17 **setScale()**

```
void wPlot2D::FunctionEntity::setScale (
    sf::Vector2f scale)
```

Sets the scaling factors for the function graph.

This method updates the ScaleComponent so the user can control how much the x and y coordinates are stretched on screen.

Parameters

<i>scale</i>	The new scaling vector (scale.x, scale.y).
--------------	--

7.11.3.18 addExcludedInterval()

```
void wPlot2D::FunctionEntity::addExcludedInterval (
    double min,
    double max)
```

Add an excluded interval where the function should not be drawn.

Useful to handle discontinuities such as vertical asymptotes.

Parameters

<i>min</i>	Left bound of the interval.
<i>max</i>	Right bound of the interval.

7.11.3.19 clearExcludedIntervals()

```
void wPlot2D::FunctionEntity::clearExcludedIntervals ()
```

Clear all excluded intervals.

7.11.3.20 alignToYAxis()

```
void wPlot2D::FunctionEntity::alignToYAxis (
    float normalizedOffsetX = 0.0f,
    float normalizedOffsetY = 0.0f)
```

Rotates the function by 90 degrees and swaps scales accordingly.

This is a common operation when we want to interpret the function's values along the X-axis instead of the Y-axis (or vice versa). The method:

- Sets a 90 degrees rotation.
- Swaps scale.x and scale.y to preserve unit consistency.
- Applies an optional normalized offset for alignment.

Parameters

<i>normalizedOffsetX</i>	Relative offset along X after rotation, expressed in units of (oldScaleX / oldScaleY).
<i>normalizedOffsetY</i>	Relative offset along Y after rotation, expressed in units of (oldScaleY / oldScaleX).

7.11.3.21 drawFunction()

```
void wPlot2D::FunctionEntity::drawFunction (
    sf::RenderWindow & window,
    double startX,
    double endX,
    size_t nbPoints = 1000)
```

Draw the function on the target window.

Parameters

<i>window</i>	Render target.
<i>startX</i>	Start of the logical x-range.
<i>endX</i>	End of the logical x-range.
<i>nbPoints</i>	Number of points to sample (default: 1000).

The documentation for this class was generated from the following files:

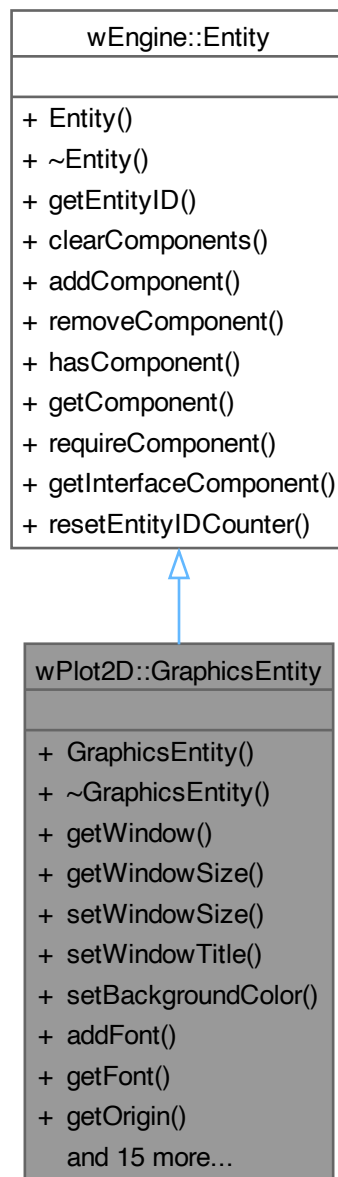
- [wFunctionEntity.hpp](#)
- [wFunctionEntity.cpp](#)

7.12 wPlot2D::GraphicsEntity Class Reference

Central entity responsible for graphical rendering in [wPlot2D](#).

```
#include <wGraphicsEntity.hpp>
```

Inheritance diagram for wPlot2D::GraphicsEntity:



Public Member Functions

- [GraphicsEntity](#) (const std::string &>windowTitle="wPlot2D", const sf::Vector2u &>windowSize={ 1600, 1600 }, const sf::Vector2f &originFactor={ 0.5f, 0.5f }, const sf::Vector2f &scaleFactor={ 0.1f, 0.1f })
Constructs the graphics entity and initializes the rendering window and components.
- virtual [~GraphicsEntity](#) ()=default
Virtual destructor.
- sf::RenderWindow & [getWindow](#) ()
Gives access to the internal SFML window.

- `sf::Vector2u getWindowSize () const`
Retrieves the current window size.
- `void setWindowSize (const sf::Vector2u &newSize)`
Sets a new window size.
- `void setWindowTitle (const std::string &title)`
Updates the window title.
- `void setBackgroundColor (const sf::Color &color)`
Clears the window with a background color.
- `void addFont (const std::string &name, const std::string &fileName)`
Adds a font to the AssetManager.
- `sf::Font &getFont (const std::string name)`
Retrieves a previously loaded font.
- `sf::Vector2f getOrigin () const`
Returns the current logical origin (in pixels).
- `void setOrigin (sf::Vector2f originFactor)`
Sets a new logical origin (normalized).
- `sf::Vector2f getScale () const`
Returns the scale factors (pixels per logical unit).
- `void setScale (sf::Vector2f scaleFactor)`
Sets new scale factors (normalized).
- `sf::Vector2f getOffset () const`
Returns the current logical offset.
- `void setOffset (sf::Vector2f offset)`
Sets the logical offset applied to the axes.
- `AxisEntity * addAxis (AxisType type, sf::Vector2f axisRange)`
Adds an axis (X or Y) to the scene.
- `TitleEntity * addTitle (const std::string &title, TitleAlignment alignment=TitleAlignment::Bottom)`
Adds a main plot title (top or bottom).
- `TitleEntity * addTitle (const std::wstring &title, TitleAlignment alignment=TitleAlignment::Bottom)`
Adds a main plot title (top or bottom).
- `FunctionEntity * addFunction (std::function< double(double)> func, double startX, double endX, size_t nbPoints=1000)`
Adds a mathematical function to the scene.
- `DataPlotEntity * addDataPlot (const std::vector< sf::Vector2f > &dataPoints)`
Adds a raw data plot (connected points).
- `LegendEntity * addLegend (const sf::Vector2f &position, bool hasFrame=true)`
Adds a legend box at a given position.
- `TitleEntity * addText (const std::string &text, sf::Vector2f position)`
Adds arbitrary text to the scene.
- `TitleEntity * addText (const std::wstring &text, sf::Vector2f position)`
Adds arbitrary text to the scene.
- `LineEntity * addLine (const sf::Vector2f &start, const sf::Vector2f &end, bool withArrow=false)`
Adds a line segment to the scene.
- `void saveToFile (const std::string &filename)`
Saves a screenshot of the current window.

Public Member Functions inherited from [wEngine::Entity](#)

- [Entity](#) ()
- virtual [~Entity](#) ()
- unsigned int [getEntityID](#) () const
Returns the unique ID associated with this entity.
- void [clearComponents](#) ()
Removes all components currently attached to the entity.
- template<typename T, typename... Args>
std::shared_ptr< T > [addComponent](#) (Args &&... args)
Adds a new component of type T to the entity.
- template<typename T>
void [removeComponent](#) ()
Removes the component of type T from the entity.
- template<typename T>
bool [hasComponent](#) () const noexcept
Checks whether the entity has a component of type T.
- template<typename T>
std::shared_ptr< T > [getComponent](#) () const
Retrieves the component of type T attached to the entity.
- template<typename T>
std::shared_ptr< T > [requireComponent](#) (const std::string &context="") const
Retrieves the component of type T and throws if it's missing.
- template<typename Interface>
std::shared_ptr< Interface > [getInterfaceComponent](#) () const
Returns the first component that implements the specified interface.

Additional Inherited Members

Static Public Member Functions inherited from [wEngine::Entity](#)

- static void [resetEntityIDCounter](#) ()
Resets the global entity ID counter to zero.

7.12.1 Detailed Description

Central entity responsible for graphical rendering in [wPlot2D](#).

The [GraphicsEntity](#) manages the creation and control of the SFML rendering window. It provides high-level methods to add and configure graphical entities:

- Axes (X and Y),
- Titles (main or custom text),
- Functions and raw data plots,
- Legends,
- Lines (with or without arrows).

It also handles window configuration (title, size, background color) and allows exporting the final rendering to an image file.

Note

This class is intended to be the main entry point for user interaction with the rendering system.

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.12.2 Constructor & Destructor Documentation

7.12.2.1 GraphicsEntity()

```
wPlot2D::GraphicsEntity::GraphicsEntity (
    const std::string & windowTitle = "wPlot2D",
    const sf::Vector2u & windowSize = { 1600, 1600 },
    const sf::Vector2f & originFactor = { 0.5f, 0.5f },
    const sf::Vector2f & scaleFactor = { 0.1f, 0.1f })
```

Constructs the graphics entity and initializes the rendering window and components.

Parameters

<i>windowTitle</i>	The title displayed on the window (default: "wPlot2D").
<i>windowSize</i>	Window dimensions in pixels (default: 1600×1600).
<i>originFactor</i>	Normalized factor in [0,1]×[0,1] specifying the origin's relative position (default: (0.5f, 0.5f)).
<i>scaleFactor</i>	Normalized factor specifying the size of one logical unit relative to window dimensions (default: (0.1f, 0.1f)).

Exceptions

<i>std::invalid_argument</i>	if originFactor not in [0,1].
------------------------------	-------------------------------

7.12.2.2 ~GraphicsEntity()

```
virtual wPlot2D::GraphicsEntity::~~GraphicsEntity () [virtual], [default]
```

Virtual destructor.

7.12.3 Member Function Documentation

7.12.3.1 getWindow()

```
sf::RenderWindow & wPlot2D::GraphicsEntity::getWindow () [nodiscard]
```

Gives access to the internal SFML window.

Returns

Reference to the internal `sf::RenderWindow`.

7.12.3.2 getWindowSize()

```
sf::Vector2u wPlot2D::GraphicsEntity::getWindowSize () const [nodiscard]
```

Retrieves the current window size.

Returns

Window size in pixels.

7.12.3.3 setWindowSize()

```
void wPlot2D::GraphicsEntity::setWindowSize (
    const sf::Vector2u & newSize)
```

Sets a new window size.

Parameters

<i>newSize</i>	Window dimensions in pixels.
----------------	------------------------------

7.12.3.4 setWindowTitle()

```
void wPlot2D::GraphicsEntity::setWindowTitle (
    const std::string & title)
```

Updates the window title.

Parameters

<i>title</i>	New window title.
--------------	-------------------

7.12.3.5 setBackgroundColor()

```
void wPlot2D::GraphicsEntity::setBackgroundColor (
    const sf::Color & color)
```

Clears the window with a background color.

Parameters

<i>color</i>	Background fill color.
--------------	------------------------

7.12.3.6 addFont()

```
void wPlot2D::GraphicsEntity::addFont (
    const std::string & name,
    const std::string & fileName)
```

Adds a font to the AssetManager.

Parameters

<i>name</i>	Identifier string for the font.
<i>fileName</i>	Path to the font file.

7.12.3.7 getFont()

```
sf::Font & wPlot2D::GraphicsEntity::getFont (
    const std::string name)
```

Retrieves a previously loaded font.

Parameters

<i>name</i>	Identifier of the font.
-------------	-------------------------

Returns

Reference to the `sf::Font`.

Exceptions

<i>std::runtime_error</i>	if the font is not found.
---------------------------	---------------------------

7.12.3.8 getOrigin()

```
sf::Vector2f wPlot2D::GraphicsEntity::getOrigin () const [nodiscard]
```

Returns the current logical origin (in pixels).

Returns

Origin position in pixel coordinates.

Exceptions

<i>std::runtime_error</i>	if the PositionComponent is missing.
---------------------------	--------------------------------------

7.12.3.9 setOrigin()

```
void wPlot2D::GraphicsEntity::setOrigin (
    sf::Vector2f originFactor)
```

Sets a new logical origin (normalized).

Parameters

<i>originFactor</i>	in [0,1]×[0,1] new relative origin.
---------------------	-------------------------------------

Exceptions

<i>std::invalid_argument</i>	if originFactor is outside [0,1].
------------------------------	-----------------------------------

7.12.3.10 getScale()

```
sf::Vector2f wPlot2D::GraphicsEntity::getScale () const [nodiscard]
```

Returns the scale factors (pixels per logical unit).

Returns

Scaling vector.

Exceptions

<i>std::runtime_error</i>	if the ScaleComponent is missing.
---------------------------	-----------------------------------

7.12.3.11 setScale()

```
void wPlot2D::GraphicsEntity::setScale (
    sf::Vector2f scaleFactor)
```

Sets new scale factors (normalized).

Parameters

<i>scaleFactor</i>	new scaling factor
--------------------	--------------------

7.12.3.12 getOffset()

```
sf::Vector2f wPlot2D::GraphicsEntity::getOffset () const [nodiscard]
```

Returns the current logical offset.

Returns

Offset vector in logical units.

Exceptions

<code>std::runtime_error</code>	if the <code>OffsetComponent</code> is missing.
---------------------------------	---

7.12.3.13 setOffset()

```
void wPlot2D::GraphicsEntity::setOffset (
    sf::Vector2f offset)
```

Sets the logical offset applied to the axes.

Parameters

<i>offset</i>	Displacement in logical units.
---------------	--------------------------------

7.12.3.14 addAxis()

```
AxisEntity * wPlot2D::GraphicsEntity::addAxis (
    AxisType type,
    sf::Vector2f axisRange) [nodiscard]
```

Adds an axis (X or Y) to the scene.

Parameters

<i>type</i>	Axis type.
<i>axisRange</i>	Logical range for the axis.

Returns

Pointer to the created [AxisEntity](#).

7.12.3.15 addTitle() [1/2]

```
TitleEntity * wPlot2D::GraphicsEntity::addTitle (
    const std::string & title,
    TitleAlignment alignment = TitleAlignment::Bottom) [nodiscard]
```

Adds a main plot title (top or bottom).

Parameters

<i>title</i>	Title text (UTF-8).
<i>alignment</i>	Vertical alignment (default: bottom).

Returns

Pointer to the created [TitleEntity](#).

7.12.3.16 addTitle() [2/2]

```
TitleEntity * wPlot2D::GraphicsEntity::addTitle (
    const std::wstring & title,
    TitleAlignment alignment = TitleAlignment::Bottom) [nodiscard]
```

Adds a main plot title (top or bottom).

Parameters

<i>title</i>	Title text (UTF-16/32).
<i>alignment</i>	Vertical alignment (default: bottom).

Returns

Pointer to the created [TitleEntity](#).

7.12.3.17 addFunction()

```
FunctionEntity * wPlot2D::GraphicsEntity::addFunction (
    std::function< double(double)> func,
    double startX,
    double endX,
    size_t nbPoints = 1000) [nodiscard]
```

Adds a mathematical function to the scene.

Parameters

<i>func</i>	Function of type double(double).
<i>startX</i>	Domain start (logical).
<i>endX</i>	Domain end (logical).
<i>nbPoints</i>	Sampling resolution (default 1000).

Returns

Pointer to the created [FunctionEntity](#).

7.12.3.18 addDataPlot()

```
DataPlotEntity * wPlot2D::GraphicsEntity::addDataPlot (
    const std::vector< sf::Vector2f > & dataPoints) [nodiscard]
```

Adds a raw data plot (connected points).

Parameters

<i>dataPoints</i>	Vector of (x,y) coordinates.
-------------------	------------------------------

Returns

Pointer to the created [DataPlotEntity](#).

7.12.3.19 addLegend()

```
LegendEntity * wPlot2D::GraphicsEntity::addLegend (
    const sf::Vector2f & position,
    bool hasFrame = true) [nodiscard]
```

Adds a legend box at a given position.

Parameters

<i>position</i>	Normalized position inside window [0,1]x[0,1].
<i>hasFrame</i>	Whether the legend frame is visible (default true).

Returns

Pointer to the created [LegendEntity](#).

7.12.3.20 addText() [1/2]

```
TitleEntity * wPlot2D::GraphicsEntity::addText (
    const std::string & text,
    sf::Vector2f position) [nodiscard]
```

Adds arbitrary text to the scene.

Parameters

<i>text</i>	Text string (UTF-8).
<i>position</i>	Normalized position in [0,1]x[0,1].

Returns

Pointer to the created [TitleEntity](#).

7.12.3.21 addText() [2/2]

```
TitleEntity * wPlot2D::GraphicsEntity::addText (
    const std::wstring & text,
    sf::Vector2f position) [nodiscard]
```

Adds arbitrary text to the scene.

Parameters

<i>text</i>	Text string (UTF-16/32).
<i>position</i>	Normalized position in [0,1]x[0,1].

Returns

Pointer to the created [TitleEntity](#).

7.12.3.22 addLine()

```
LineEntity * wPlot2D::GraphicsEntity::addLine (
    const sf::Vector2f & start,
    const sf::Vector2f & end,
    bool withArrow = false) [nodiscard]
```

Adds a line segment to the scene.

Parameters

<i>start</i>	Start point in logical coordinates.
<i>end</i>	End point in logical coordinates.
<i>withArrow</i>	Whether to render an arrowhead at the end.

Returns

Pointer to the created [LineEntity](#).

7.12.3.23 saveToFile()

```
void wPlot2D::GraphicsEntity::saveToFile (  
    const std::string & filename)
```

Saves a screenshot of the current window.

Parameters

<i>filename</i>	Output file path (supported: png, bmp, tga, jpg).
-----------------	---

Exceptions

<i>std::runtime_error</i>	if saving fails.
---------------------------	------------------

The documentation for this class was generated from the following files:

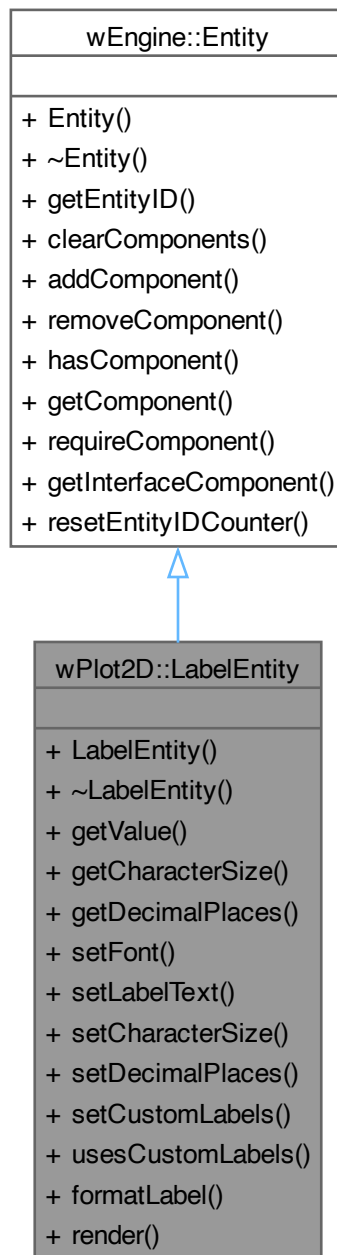
- [wGraphicsEntity.hpp](#)
- [wGraphicsEntity.cpp](#)

7.13 wPlot2D::LabelEntity Class Reference

Represents a textual label or a collection of axis labels.

```
#include <wLabelEntity.hpp>
```

Inheritance diagram for wPlot2D::LabelEntity:



Public Member Functions

- [LabelEntity](#) (const sf::Font &font, [AxisType](#) type, sf::Vector2f initialPosition)
Constructs a [LabelEntity](#) with a given font, axis orientation and initial position.
- virtual [~LabelEntity](#) ()=default
Virtual destructor.
- float [getValue](#) () const

- Returns the numeric value associated with the label.*
- unsigned int [getCharacterSize](#) () const
Returns the current character size of the label text.
- int [getDecimalPlaces](#) () const
Returns the number of decimal places currently used for numeric formatting.
- void [setFont](#) (const sf::Font &font)
Sets a new font for the label.
- void [setLabelText](#) (std::string text)
Defines the text content of the label.
- void [setCharacterSize](#) (unsigned int newSize)
Sets a new character size for the labels.
- void [setDecimalPlaces](#) (int places)
Sets the number of decimal places for numeric labels.
- void [setCustomLabels](#) (const std::string &labels)
Sets a custom label string.
- bool [usesCustomLabels](#) () const
Indicates whether the entity is currently using custom labels.
- std::string [formatLabel](#) (float value)
Formats a numeric value into a label string.
- void [render](#) (sf::RenderWindow &window)
Renders the label on the given SFML window.

Public Member Functions inherited from [wEngine::Entity](#)

- [Entity](#) ()
- virtual [~Entity](#) ()
- unsigned int [getEntityID](#) () const
Returns the unique ID associated with this entity.
- void [clearComponents](#) ()
Removes all components currently attached to the entity.
- template<typename T, typename... Args>
std::shared_ptr< T > [addComponent](#) (Args &&... args)
Adds a new component of type T to the entity.
- template<typename T>
void [removeComponent](#) ()
Removes the component of type T from the entity.
- template<typename T>
bool [hasComponent](#) () const noexcept
Checks whether the entity has a component of type T.
- template<typename T>
std::shared_ptr< T > [getComponent](#) () const
Retrieves the component of type T attached to the entity.
- template<typename T>
std::shared_ptr< T > [requireComponent](#) (const std::string &context="") const
Retrieves the component of type T and throws if it's missing.
- template<typename Interface>
std::shared_ptr< Interface > [getInterfaceComponent](#) () const
Returns the first component that implements the specified interface.

Additional Inherited Members

Static Public Member Functions inherited from [wEngine::Entity](#)

- static void [resetEntityIDCounter](#) ()
Resets the global entity ID counter to zero.

7.13.1 Detailed Description

Represents a textual label or a collection of axis labels.

A [LabelEntity](#) manages the rendering of formatted text associated with axis notches. Labels can be generated dynamically (from numeric values, with controlled precision) or defined manually via custom strings.

The class relies on SFML's `sf::Text` for rendering and provides customization of style (font, color, character size) and placement (axis orientation, offset relative to the axis).

Note

Typically, a [LabelEntity](#) is aggregated inside an [AxisEntity](#) to display labels alongside axis notches.

See also

[AxisEntity](#)

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.13.2 Constructor & Destructor Documentation

7.13.2.1 LabelEntity()

```
wPlot2D::LabelEntity::LabelEntity (
    const sf::Font & font,
    AxisType type,
    sf::Vector2f initialPosition)
```

Constructs a [LabelEntity](#) with a given font, axis orientation and initial position.

Parameters

<i>font</i>	Reference to an SFML font (must remain valid during the lifetime of the entity).
<i>type</i>	Axis orientation (AxisType::X_AXIS or AxisType::Y_AXIS).
<i>initialPosition</i>	Position where the label will be anchored.

7.13.2.2 ~LabelEntity()

```
virtual wPlot2D::LabelEntity::~~LabelEntity () [virtual], [default]
```

Virtual destructor.

7.13.3 Member Function Documentation

7.13.3.1 getValue()

```
float wPlot2D::LabelEntity::getValue () const [nodiscard]
```

Returns the numeric value associated with the label.

Returns

The value stored in the label.

7.13.3.2 getCharacterSize()

```
unsigned int wPlot2D::LabelEntity::getCharacterSize () const [nodiscard]
```

Returns the current character size of the label text.

Returns

Character size in pixels.

7.13.3.3 getDecimalPlaces()

```
int wPlot2D::LabelEntity::getDecimalPlaces () const [nodiscard]
```

Returns the number of decimal places currently used for numeric formatting.

Returns

Number of digits after the decimal point.

7.13.3.4 setFont()

```
void wPlot2D::LabelEntity::setFont (  
    const sf::Font & font)
```

Sets a new font for the label.

Parameters

<i>font</i>	Reference to an SFML font (must remain valid during the lifetime of the entity).
-------------	--

7.13.3.5 setLabelText()

```
void wPlot2D::LabelEntity::setLabelText (
    std::string text)
```

Defines the text content of the label.

If custom labels are enabled, this method updates the string that will be rendered. Otherwise, it is generally managed internally via numeric formatting.

Parameters

<i>text</i>	The new string to assign to the label.
-------------	--

7.13.3.6 setCharacterSize()

```
void wPlot2D::LabelEntity::setCharacterSize (
    unsigned int newSize)
```

Sets a new character size for the labels.

Parameters

<i>newSize</i>	Character size in pixels.
----------------	---------------------------

7.13.3.7 setDecimalPlaces()

```
void wPlot2D::LabelEntity::setDecimalPlaces (
    int places)
```

Sets the number of decimal places for numeric labels.

Parameters

<i>places</i>	Digits after the decimal point (must be ≥ 0).
---------------	---

7.13.3.8 setCustomLabels()

```
void wPlot2D::LabelEntity::setCustomLabels (
    const std::string & labels)
```

Sets a custom label string.

This enables "custom label mode". When active, numeric formatting is ignored and the provided string is displayed instead.

Parameters

<i>labels</i>	Custom string to display as a label.
---------------	--------------------------------------

7.13.3.9 usesCustomLabels()

```
bool wPlot2D::LabelEntity::usesCustomLabels () const [nodiscard]
```

Indicates whether the entity is currently using custom labels.

Returns

True if custom labels are active, false if numeric formatting is used.

7.13.3.10 formatLabel()

```
std::string wPlot2D::LabelEntity::formatLabel (  
    float value)
```

Formats a numeric value into a label string.

If custom labels are active, the stored custom string is returned. Otherwise, the numeric value is converted using the current number of decimal places.

Parameters

<i>value</i>	Numeric value to format.
--------------	--------------------------

Returns

A string ready to be displayed as a label.

7.13.3.11 render()

```
void wPlot2D::LabelEntity::render (  
    sf::RenderWindow & window)
```

Renders the label on the given SFML window.

Parameters

<i>window</i>	Reference to the render window.
---------------	---------------------------------

The documentation for this class was generated from the following files:

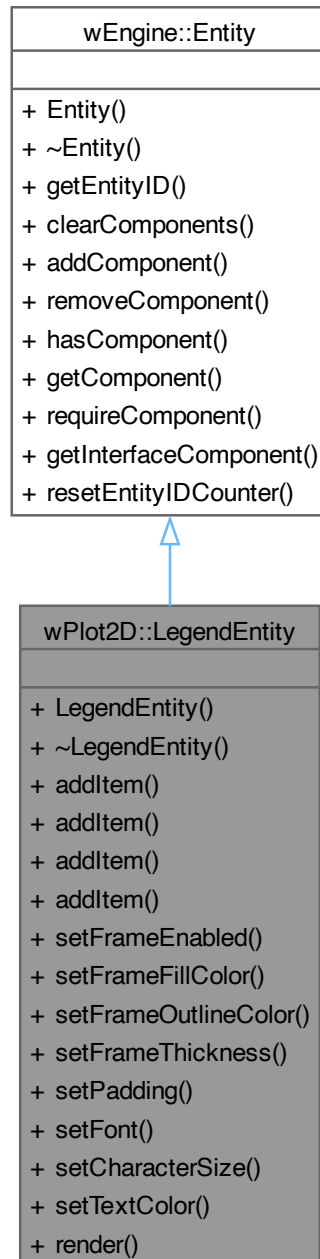
- [wLabelEntity.hpp](#)
- [wLabelEntity.cpp](#)

7.14 wPlot2D::LegendEntity Class Reference

Represents a legend box that describes functions and data plots.

```
#include <wLegendEntity.hpp>
```

Inheritance diagram for wPlot2D::LegendEntity:



Public Member Functions

- [LegendEntity](#) (const sf::Font &font, const sf::Vector2f &position, bool hasFrame=true)
Constructs a [LegendEntity](#).
- virtual [~LegendEntity](#) ()=default
Virtual destructor.
- void [addItem](#) (const std::string &label, [FunctionEntity](#) *function)
Adds a new legend item associated with a function.
- void [addItem](#) (const std::wstring &label, [FunctionEntity](#) *function)
Adds a new legend item associated with a function (wide string).
- void [addItem](#) (const std::string &label, [DataPlotEntity](#) *plot)
Adds a new legend item associated with a data plot.
- void [addItem](#) (const std::wstring &label, [DataPlotEntity](#) *plot)
Adds a new legend item associated with a data plot (wide string).
- void [setFrameEnabled](#) (bool enabled)
Enables or disables the surrounding frame of the legend.
- void [setFrameFillColor](#) (const sf::Color &color)
Sets the fill color of the legend frame.
- void [setFrameOutlineColor](#) (const sf::Color &color)
Sets the outline color of the legend frame.
- void [setFrameThickness](#) (float thickness)
Sets the outline thickness of the legend frame.
- void [setPadding](#) (const sf::Vector2f &padding)
Sets the internal padding between items and the frame borders.
- void [setFont](#) (const sf::Font &font)
Updates the font used for all legend labels.
- void [setCharacterSize](#) (unsigned int size)
Sets the character size of the legend text.
- void [setTextColor](#) (const sf::Color &color)
Sets the color of the legend labels.
- void [render](#) (sf::RenderWindow &window)
Renders the legend (all items and optional frame) to the target window.

Public Member Functions inherited from [wEngine::Entity](#)

- [Entity](#) ()
- virtual [~Entity](#) ()
- unsigned int [getEntityID](#) () const
Returns the unique ID associated with this entity.
- void [clearComponents](#) ()
Removes all components currently attached to the entity.
- template<typename T, typename... Args>
std::shared_ptr< T > [addComponent](#) (Args &&... args)
Adds a new component of type T to the entity.
- template<typename T>
void [removeComponent](#) ()
Removes the component of type T from the entity.
- template<typename T>
bool [hasComponent](#) () const noexcept
Checks whether the entity has a component of type T.

- `template<typename T>`
`std::shared_ptr< T > getComponent () const`
Retrieves the component of type T attached to the entity.
- `template<typename T>`
`std::shared_ptr< T > requireComponent (const std::string &context="") const`
Retrieves the component of type T and throws if it's missing.
- `template<typename Interface>`
`std::shared_ptr< Interface > getInterfaceComponent () const`
Returns the first component that implements the specified interface.

Additional Inherited Members

Static Public Member Functions inherited from [wEngine::Entity](#)

- static void [resetEntityIDCounter](#) ()
Resets the global entity ID counter to zero.

7.14.1 Detailed Description

Represents a legend box that describes functions and data plots.

A [LegendEntity](#) provides a visual legend for plotted entities such as [FunctionEntity](#) and [DataPlotEntity](#). Each legend item is composed of:

- A sample line ([LineEntity](#)) with the same style, thickness, and color as the source entity.
- A text label (`sf::Text`) describing the entity.

The legend can optionally be surrounded by a frame ([FrameEntity](#)) with configurable padding, outline color, thickness, and fill color.

7.14.1.0.1 Components and Features:

- Configurable font and text size.
- Support for UTF-8 and wide string labels.
- Dynamic addition of items from existing plotted entities.
- Automatic alignment of line + text pairs inside the legend box.

See also

[FunctionEntity](#), [DataPlotEntity](#), [LineEntity](#), [FrameEntity](#)

Note

The font passed in the constructor must remain valid during the lifetime of the legend, as SFML does not copy font data internally.

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch

7.14.2 Constructor & Destructor Documentation

7.14.2.1 LegendEntity()

```
wPlot2D::LegendEntity::LegendEntity (
    const sf::Font & font,
    const sf::Vector2f & position,
    bool hasFrame = true)
```

Constructs a [LegendEntity](#).

Initializes the legend with a given font, anchor position, and optional frame.

Parameters

<i>font</i>	Reference to the font used for labels (must remain valid).
<i>position</i>	Anchor position of the legend box in window coordinates.
<i>hasFrame</i>	Whether to display a surrounding frame (default: true).

7.14.2.2 ~LegendEntity()

```
virtual wPlot2D::LegendEntity::~~LegendEntity () [virtual], [default]
```

Virtual destructor.

7.14.3 Member Function Documentation

7.14.3.1 addItem() [1/4]

```
void wPlot2D::LegendEntity::addItem (
    const std::string & label,
    FunctionEntity * function)
```

Adds a new legend item associated with a function.

Parameters

<i>label</i>	Label text (UTF-8 string).
<i>function</i>	Pointer to the source FunctionEntity .

7.14.3.2 addItem() [2/4]

```
void wPlot2D::LegendEntity::addItem (
    const std::wstring & label,
    FunctionEntity * function)
```

Adds a new legend item associated with a function (wide string).

Parameters

<i>label</i>	Label text (wide string).
<i>function</i>	Pointer to the source FunctionEntity .

7.14.3.3 addItem() [3/4]

```
void wPlot2D::LegendEntity::addItem (
    const std::string & label,
    DataPlotEntity * plot)
```

Adds a new legend item associated with a data plot.

Parameters

<i>label</i>	Label text (UTF-8 string).
<i>plot</i>	Pointer to the source DataPlotEntity .

7.14.3.4 addItem() [4/4]

```
void wPlot2D::LegendEntity::addItem (
    const std::wstring & label,
    DataPlotEntity * plot)
```

Adds a new legend item associated with a data plot (wide string).

Parameters

<i>label</i>	Label text (wide string).
<i>plot</i>	Pointer to the source DataPlotEntity .

7.14.3.5 setFrameEnabled()

```
void wPlot2D::LegendEntity::setFrameEnabled (
    bool enabled)
```

Enables or disables the surrounding frame of the legend.

Parameters

<i>enabled</i>	True to display the frame, false to hide it.
----------------	--

7.14.3.6 setFrameFillColor()

```
void wPlot2D::LegendEntity::setFrameFillColor (
    const sf::Color & color)
```

Sets the fill color of the legend frame.

Parameters

<i>color</i>	SFML color applied to the frame's background.
--------------	---

7.14.3.7 setFrameOutlineColor()

```
void wPlot2D::LegendEntity::setFrameOutlineColor (
    const sf::Color & color)
```

Sets the outline color of the legend frame.

Parameters

<i>color</i>	SFML color applied to the frame's border.
--------------	---

7.14.3.8 setFrameThickness()

```
void wPlot2D::LegendEntity::setFrameThickness (
    float thickness)
```

Sets the outline thickness of the legend frame.

Parameters

<i>thickness</i>	Thickness in pixels (positive for outside expansion).
------------------	---

7.14.3.9 setPadding()

```
void wPlot2D::LegendEntity::setPadding (
    const sf::Vector2f & padding)
```

Sets the internal padding between items and the frame borders.

Padding defines horizontal and vertical margins in pixels.

Parameters

<i>padding</i>	Vector (x, y) where: • <i>x</i> = horizontal padding (left/right), • <i>y</i> = vertical padding (top/bottom).
----------------	---

7.14.3.10 setFont()

```
void wPlot2D::LegendEntity::setFont (
    const sf::Font & font)
```

Updates the font used for all legend labels.

Note

The font must remain valid during the legend's lifetime, as SFML does not copy font data internally.

Parameters

<i>font</i>	Reference to an externally managed sf::Font.
-------------	--

7.14.3.11 setCharacterSize()

```
void wPlot2D::LegendEntity::setCharacterSize (
    unsigned int size)
```

Sets the character size of the legend text.

Parameters

<i>size</i>	Font size in pixels.
-------------	----------------------

7.14.3.12 setTextColor()

```
void wPlot2D::LegendEntity::setTextColor (
    const sf::Color & color)
```

Sets the color of the legend labels.

Parameters

<i>color</i>	SFML color applied to all legend text.
--------------	--

7.14.3.13 render()

```
void wPlot2D::LegendEntity::render (
    sf::RenderWindow & window)
```

Renders the legend (all items and optional frame) to the target window.

Each item is drawn with its sample line and label text, aligned inside the legend box. The optional frame is drawn behind all items.

Parameters

<i>window</i>	The target SFML render window.
---------------	--------------------------------

The documentation for this class was generated from the following files:

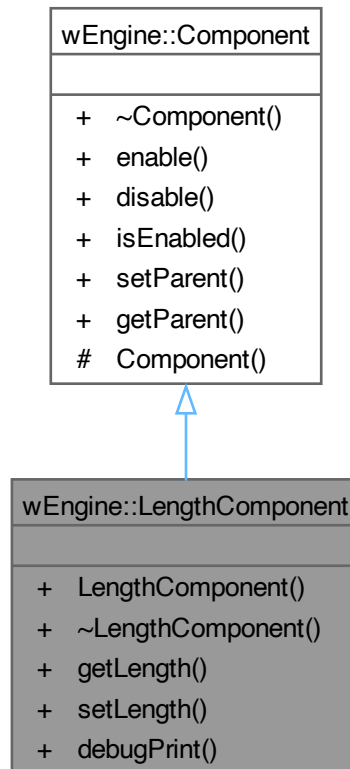
- [wLegendEntity.hpp](#)
- [wLegendEntity.cpp](#)

7.15 wEngine::LengthComponent Class Reference

ECS component that defines the length of a drawable object.

```
#include <wLengthComponent.hpp>
```

Inheritance diagram for wEngine::LengthComponent:



Public Member Functions

- `LengthComponent` (float length=2.0f)
Constructs the component with an initial positive length.
- virtual `~LengthComponent` ()=default
- float `getLength` () const
Returns the current length.
- void `setLength` (float newLength)
Sets a new length value.
- void `debugPrint` () const
Outputs the current length value to the console for debugging.

Public Member Functions inherited from `wEngine::Component`

- virtual `~Component()`=default
- virtual void `enable()`
- virtual void `disable()`
- bool `isEnabled()` const
Checks whether the component is currently active.
- void `setParent(Entity *parent)`
Sets the parent entity of this component.
- `Entity *` `getParent()` const
Returns the parent entity of this component.

Additional Inherited Members

Protected Member Functions inherited from `wEngine::Component`

- `Component()`
Protected constructor to restrict instantiation to derived classes.

7.15.1 Detailed Description

ECS component that defines the length of a drawable object.

This component stores a positive float value representing the length (in pixels) of lines or shapes. The value must be strictly positive.

Exceptions

<code>std::invalid_argument</code>	if the value is zero or negative.
------------------------------------	-----------------------------------

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.15.2 Constructor & Destructor Documentation

7.15.2.1 `LengthComponent()`

```
wEngine::LengthComponent::LengthComponent (
    float length = 2.0f)
```

Constructs the component with an initial positive length.

Parameters

<i>length</i>	Initial length value (default is 2.0f).
---------------	---

Exceptions

<i>std::invalid_argument</i>	if the value is zero or negative.
------------------------------	-----------------------------------

7.15.2.2 ~LengthComponent()

```
virtual wEngine::LengthComponent::~~LengthComponent () [virtual], [default]
```

7.15.3 Member Function Documentation

7.15.3.1 getLength()

```
float wEngine::LengthComponent::getLength () const [nodiscard]
```

Returns the current length.

Returns

A positive float representing the line length (in pixels).

7.15.3.2 setLength()

```
void wEngine::LengthComponent::setLength (
    float newLength)
```

Sets a new length value.

Parameters

<i>newLength</i>	A strictly positive float.
------------------	----------------------------

Exceptions

<i>std::invalid_argument</i>	if the value is zero or negative.
------------------------------	-----------------------------------

7.15.3.3 debugPrint()

```
void wEngine::LengthComponent::debugPrint () const
```

Outputs the current length value to the console for debugging.

The documentation for this class was generated from the following files:

- [wLengthComponent.hpp](#)
- [wLengthComponent.cpp](#)

7.16 wEngine::LineDrawer Class Reference

Utility class for rendering thick lines and polylines with style support.

```
#include <wLineDrawer.hpp>
```

Static Public Member Functions

- static float [drawLine](#) (sf::RenderWindow &window, const sf::Vector2f &point1, const sf::Vector2f &point2, const sf::Color &color, float thickness, [LineStyleComponent::LineStyle](#) style=[LineStyleComponent::LineStyle::Solid](#), float dashLength=20.0f, float gapLength=5.0f, float patternOffset=0.0f)

Draws a single thick line segment between two points.

- static void [drawPolylineRound](#) (sf::RenderWindow &window, const std::vector< sf::Vector2f > &points, const sf::Color &color, float thickness, [LineStyleComponent::LineStyle](#) style=[LineStyleComponent::LineStyle::Solid](#), float dashLength=20.0f, float gapLength=5.0f, unsigned int arcResolution=12)

Draws a polyline (sequence of connected line segments) with optional round joins.

7.16.1 Detailed Description

Utility class for rendering thick lines and polylines with style support.

The [LineDrawer](#) provides static methods to draw line segments and polylines with configurable thickness, color, and style (Solid, Dashed, Dotted).

7.16.1.0.1 Features:

- Thick line rendering via quads (two triangles per segment).
- Support for dashed and dotted patterns using configurable dash/gap lengths.
- Dash/dot continuity across multiple connected segments using a shared pattern offset.
- Optional round joins at corners of polylines (applied only when style == Solid).

7.16.1.0.2 Usage:

- Use [drawLine\(\)](#) to render a single thick segment.
- Use [drawPolylineRound\(\)](#) to render a sequence of connected points with optional round joins.
- To maintain consistent dash/dot alignment across segments, pass the returned `patternOffset` from [drawLine\(\)](#) into the next segment.

Warning

Round joins are currently only applied for `Solid` style. For dashed or dotted lines, joins would produce inconsistent results and are therefore omitted.

See also

[LineStyleComponent](#) for configuring line style options.

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.16.2 Member Function Documentation

7.16.2.1 drawLine()

```
float wEngine::LineDrawer::drawLine (
    sf::RenderWindow & window,
    const sf::Vector2f & point1,
    const sf::Vector2f & point2,
    const sf::Color & color,
    float thickness,
    LineStyleComponent::LineStyle style = LineStyleComponent::LineStyle::Solid,
    float dashLength = 20.0f,
    float gapLength = 5.0f,
    float patternOffset = 0.0f) [static]
```

Draws a single thick line segment between two points.

7.16.2.1.1 Style behavior:

- **Solid:** Renders a single quad covering the full segment.
- **Dashed:** Repeats a dash/gap pattern along the segment.
- **Dotted:** Places successive dots along the segment, using thickness as dot length.

7.16.2.1.2 Pattern control:

- `dashLength` sets the visible length of each dash (Dashed style).
- `gapLength` sets the empty space between dashes or dots.
- `thickness` is reused as the dot length if `style == Dotted`.
- `patternOffset` maintains pattern alignment between consecutive calls.

Parameters

<i>window</i>	Render target.
<i>point1</i>	First endpoint of the line.
<i>point2</i>	Second endpoint of the line.
<i>color</i>	Line color.
<i>thickness</i>	Line thickness in pixels.
<i>style</i>	Line style (Solid, Dashed, Dotted).
<i>dashLength</i>	Dash length (used if <code>style == Dashed</code>).
<i>gapLength</i>	Gap length between dashes or dots.
<i>patternOffset</i>	Initial offset within the dash/dot pattern.

Returns

Updated pattern offset after this segment (pass to next segment for continuity).

7.16.2.2 drawPolylineRound()

```
void wEngine::LineDrawer::drawPolylineRound (
    sf::RenderWindow & window,
    const std::vector< sf::Vector2f > & points,
    const sf::Color & color,
    float thickness,
    LineStyleComponent::LineStyle style = LineStyleComponent::LineStyle::Solid,
    float dashLength = 20.0f,
    float gapLength = 5.0f,
    unsigned int arcResolution = 12) [static]
```

Draws a polyline (sequence of connected line segments) with optional round joins.

- Each segment [p1, p2] is rendered using [drawLine\(\)](#), with pattern continuity preserved.
- If style == Solid and a next segment exists: a circular arc is approximated using triangles to smooth the corner at [p2].

Parameters

<i>window</i>	Render target.
<i>points</i>	List of polyline points (must contain at least 2).
<i>color</i>	Polyline color.
<i>thickness</i>	Line thickness in pixels.
<i>style</i>	Line style (Solid, Dashed, Dotted).
<i>dashLength</i>	Dash length (used if style == Dashed).
<i>gapLength</i>	Gap length between dashes or dots.
<i>arcResolution</i>	Number of triangles used to approximate each round join (higher = smoother).

Note

For Dashed or Dotted styles, round joins are skipped.

The documentation for this class was generated from the following files:

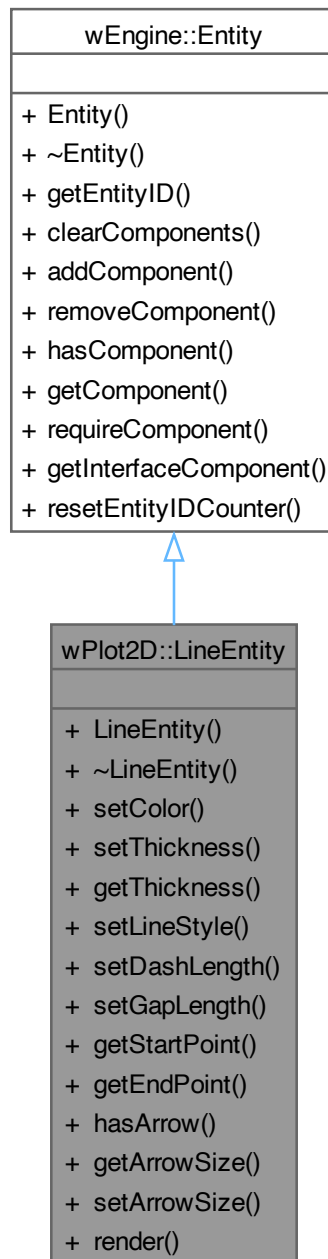
- [wLineDrawer.hpp](#)
- [wLineDrawer.cpp](#)

7.17 wPlot2D::LineEntity Class Reference

Entity representing a straight line segment with optional arrowhead.

```
#include <wLineEntity.hpp>
```

Inheritance diagram for wPlot2D::LineEntity:



Public Member Functions

- [LineEntity](#) (const sf::Vector2f &origin, const sf::Vector2f &scale, const sf::Vector2f &start, const sf::Vector2f &end, bool withArrow=false)
Construct a line entity between two points.
- virtual [~LineEntity](#) ()=default
Virtual destructor.

- void [setColor](#) (sf::Color color)
Sets the color of the line and arrowhead.
- void [setThickness](#) (float thickness)
Sets the thickness of the line.
- float [getThickness](#) () const
Returns the current thickness of the line.
- void [setLineStyle](#) (wEngine::LineStyleComponent::LineStyle style)
Sets the visual style of the line.
- void [setDashLength](#) (float dashLength)
Sets the dash length for dashed lines.
- void [setGapLength](#) (float gapLength)
Sets the gap length between dashes or dots.
- sf::Vector2f [getStartPoint](#) () const
Returns the starting point of the line.
- sf::Vector2f [getEndPoint](#) () const
Returns the ending point of the line.
- bool [hasArrow](#) () const
Checks if the line has an arrowhead.
- float [getArrowSize](#) () const
Returns the arrowhead size factor.
- void [setArrowSize](#) (float arrowSize)
Sets the arrowhead size factor.
- void [render](#) (sf::RenderWindow &window)
Renders the line (and optional arrowhead).

Public Member Functions inherited from [wEngine::Entity](#)

- [Entity](#) ()
- virtual [~Entity](#) ()
- unsigned int [getEntityID](#) () const
Returns the unique ID associated with this entity.
- void [clearComponents](#) ()
Removes all components currently attached to the entity.
- template<typename T, typename... Args>
std::shared_ptr< T > [addComponent](#) (Args &&... args)
Adds a new component of type T to the entity.
- template<typename T>
void [removeComponent](#) ()
Removes the component of type T from the entity.
- template<typename T>
bool [hasComponent](#) () const noexcept
Checks whether the entity has a component of type T.
- template<typename T>
std::shared_ptr< T > [getComponent](#) () const
Retrieves the component of type T attached to the entity.
- template<typename T>
std::shared_ptr< T > [requireComponent](#) (const std::string &context="") const
Retrieves the component of type T and throws if it's missing.
- template<typename Interface>
std::shared_ptr< Interface > [getInterfaceComponent](#) () const
Returns the first component that implements the specified interface.

Additional Inherited Members

Static Public Member Functions inherited from [wEngine::Entity](#)

- static void [resetEntityIDCounter](#) ()
Resets the global entity ID counter to zero.

7.17.1 Detailed Description

Entity representing a straight line segment with optional arrowhead.

This entity provides configurable line rendering within the plot area:

- Supports Solid, Dashed, and Dotted styles (via [LineStyleComponent](#)).
- Customizable color, thickness, dash length, and gap length.
- Optional arrowhead at the end (useful for axes or vectors).

Coordinates are expressed in logical units and transformed by the entity's origin and scale before being rendered.

See also

[wEngine::LineDrawer](#) for the rendering implementation

[wEngine::LineStyleComponent](#) for style configuration

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.17.2 Constructor & Destructor Documentation

7.17.2.1 LineEntity()

```
wPlot2D::LineEntity::LineEntity (  
    const sf::Vector2f & origin,  
    const sf::Vector2f & scale,  
    const sf::Vector2f & start,  
    const sf::Vector2f & end,  
    bool withArrow = false)
```

Construct a line entity between two points.

Parameters

<i>origin</i>	Origin of the plot (reference point).
<i>scale</i>	Scaling factor to convert logical coordinates into pixels.
<i>start</i>	Line starting point (logical coordinates).
<i>end</i>	Line ending point (logical coordinates).
<i>withArrow</i>	Whether to draw an arrowhead at the end.

7.17.2.2 ~LineEntity()

```
virtual wPlot2D::LineEntity::~~LineEntity () [virtual], [default]
```

Virtual destructor.

7.17.3 Member Function Documentation**7.17.3.1 setColor()**

```
void wPlot2D::LineEntity::setColor (
    sf::Color color)
```

Sets the color of the line and arrowhead.

Parameters

<i>color</i>	New color to apply.
--------------	---------------------

7.17.3.2 setThickness()

```
void wPlot2D::LineEntity::setThickness (
    float thickness)
```

Sets the thickness of the line.

Parameters

<i>thickness</i>	Thickness in pixels.
------------------	----------------------

7.17.3.3 getThickness()

```
float wPlot2D::LineEntity::getThickness () const [nodiscard]
```

Returns the current thickness of the line.

Returns

Thickness in pixels.

7.17.3.4 setLineStyle()

```
void wPlot2D::LineEntity::setLineStyle (
    wEngine::LineStyleComponent::LineStyle style)
```

Sets the visual style of the line.

Parameters

<i>style</i>	Solid, Dashed, or Dotted.
--------------	---------------------------

7.17.3.5 setDashLength()

```
void wPlot2D::LineEntity::setDashLength (
    float dashLength)
```

Sets the dash length for dashed lines.

Parameters

<i>dashLength</i>	Length of each dash in pixels.
-------------------	--------------------------------

7.17.3.6 setGapLength()

```
void wPlot2D::LineEntity::setGapLength (
    float gapLength)
```

Sets the gap length between dashes or dots.

Parameters

<i>gapLength</i>	Length of the gap in pixels.
------------------	------------------------------

7.17.3.7 getStartPoint()

```
sf::Vector2f wPlot2D::LineEntity::getStartPoint () const [nodiscard]
```

Returns the starting point of the line.

Returns

Start point in logical coordinates.

7.17.3.8 getEndPoint()

```
sf::Vector2f wPlot2D::LineEntity::getEndPoint () const [nodiscard]
```

Returns the ending point of the line.

Returns

End point in logical coordinates.

7.17.3.9 hasArrow()

```
bool wPlot2D::LineEntity::hasArrow () const [nodiscard]
```

Checks if the line has an arrowhead.

Returns

True if an arrowhead is drawn, false otherwise.

7.17.3.10 getArrowSize()

```
float wPlot2D::LineEntity::getArrowSize () const [nodiscard]
```

Returns the arrowhead size factor.

Returns

Arrow size relative to line thickness.

7.17.3.11 setArrowSize()

```
void wPlot2D::LineEntity::setArrowSize (
    float arrowSize)
```

Sets the arrowhead size factor.

Parameters

<i>arrowSize</i>	Arrow size relative to line thickness.
------------------	--

7.17.3.12 render()

```
void wPlot2D::LineEntity::render (
    sf::RenderWindow & window)
```

Renders the line (and optional arrowhead).

Parameters

<i>window</i>	Target render window.
---------------	-----------------------

The documentation for this class was generated from the following files:

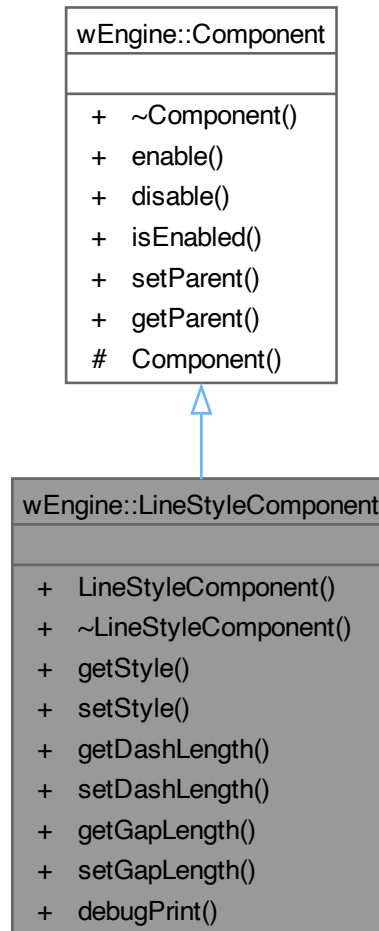
- [wLineEntity.hpp](#)
- [wLineEntity.cpp](#)

7.18 wEngine::LineStyleComponent Class Reference

ESC component that defines the style of a line (solid, dotted, dashed).

```
#include <wLineStyleComponent.hpp>
```

Inheritance diagram for wEngine::LineStyleComponent:



Public Types

- enum class `LineStyle` { `Solid` , `Dotted` , `Dashed` }

Available styles for line rendering.

Public Member Functions

- `LineStyleComponent` (`LineStyle` style=`LineStyle::Solid`)

Constructs a `LineStyleComponent` with an optional style.

- virtual `~LineStyleComponent()`=default
- `LineStyle` `getStyle()` const
Returns the current line style.
- void `setStyle` (`LineStyle` style)
Sets the current line style.
- float `getDashLength()` const
Returns the dash length (used for Dashed style).
- void `setDashLength` (float dashLength)
Sets the dash length.
- float `getGapLength()` const
Returns the gap length (used for Dotted and Dashed styles).
- void `setGapLength` (float gapLength)
Sets the gap length.
- void `debugPrint()` const

Public Member Functions inherited from `wEngine::Component`

- virtual `~Component()`=default
- virtual void `enable()`
- virtual void `disable()`
- bool `isEnabled()` const
Checks whether the component is currently active.
- void `setParent` (`Entity *`parent)
Sets the parent entity of this component.
- `Entity *` `getParent()` const
Returns the parent entity of this component.

Additional Inherited Members

Protected Member Functions inherited from `wEngine::Component`

- `Component()`
Protected constructor to restrict instantiation to derived classes.

7.18.1 Detailed Description

ESC component that defines the style of a line (solid, dotted, dashed).

This component controls how lines are drawn in the rendering pipeline. For dotted and dashed styles, both dash length and gap length can be configured.

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.18.2 Member Enumeration Documentation

7.18.2.1 LineStyle

```
enum class wEngine::LineStyleComponent::LineStyle [strong]
```

Available styles for line rendering.

Enumerator

Solid	
Dotted	
Dashed	

7.18.3 Constructor & Destructor Documentation

7.18.3.1 LineStyleComponent()

```
wEngine::LineStyleComponent::LineStyleComponent (  
    LineStyle style = LineStyle::Solid)
```

Constructs a [LineStyleComponent](#) with an optional style.

Parameters

<i>style</i>	Line style to use (default: Solid).
--------------	-------------------------------------

7.18.3.2 ~LineStyleComponent()

```
virtual wEngine::LineStyleComponent::~~LineStyleComponent () [virtual], [default]
```

7.18.4 Member Function Documentation

7.18.4.1 getStyle()

```
LineStyleComponent::LineStyle wEngine::LineStyleComponent::getStyle () const [nodiscard]
```

Returns the current line style.

Returns

The style as [LineStyle](#).

7.18.4.2 setStyle()

```
void wEngine::LineStyleComponent::setStyle (  
    LineStyle style)
```

Sets the current line style.

Parameters

<i>style</i>	New line style to apply.
--------------	--------------------------

7.18.4.3 `getDashLength()`

```
float wEngine::LineStyleComponent::getDashLength () const [nodiscard]
```

Returns the dash length (used for Dashed style).

Returns

Dash length in pixels.

7.18.4.4 `setDashLength()`

```
void wEngine::LineStyleComponent::setDashLength (  
    float dashLength)
```

Sets the dash length.

Parameters

<i>dashLength</i>	Dash length in pixels.
-------------------	------------------------

Exceptions

<i>std::invalid_argument</i>	if <i>dashLength</i> ≤ 0.
------------------------------	---------------------------

7.18.4.5 `getGapLength()`

```
float wEngine::LineStyleComponent::getGapLength () const [nodiscard]
```

Returns the gap length (used for Dotted and Dashed styles).

Returns

Gap length in pixels.

7.18.4.6 `setGapLength()`

```
void wEngine::LineStyleComponent::setGapLength (  
    float gapLength)
```

Sets the gap length.

Parameters

<i>gapLength</i>	Gap length in pixels.
------------------	-----------------------

Exceptions

<code>std::invalid_argument</code>	if <code>gapLength < 0</code> .
------------------------------------	------------------------------------

7.18.4.7 debugPrint()

```
void wEngine::LineStyleComponent::debugPrint () const
```

The documentation for this class was generated from the following files:

- [wLineStyleComponent.hpp](#)
- [wLineStyleComponent.cpp](#)

7.19 wEngine::MathUtils Class Reference

Provides common mathematical helper functions for plotting and geometry.

```
#include <wMathUtils.hpp>
```

Static Public Member Functions

- static `std::vector< double > linspace` (double start, double end, size_t nbPoints)
Generates a linearly spaced vector of values between two bounds.

7.19.1 Detailed Description

Provides common mathematical helper functions for plotting and geometry.

This utility class groups static methods that are frequently needed when working with numerical data, discretization, and rendering curves.

Note

All methods are static and do not require instantiation.

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.19.2 Member Function Documentation

7.19.2.1 linspace()

```
std::vector< double > wEngine::MathUtils::linspace (
    double start,
    double end,
    size_t nbPoints) [static], [nodiscard]
```

Generates a linearly spaced vector of values between two bounds.

This function produces a vector of evenly spaced points between `start` and `end` (inclusive).

Parameters

<i>start</i>	Starting value.
<i>end</i>	Ending value.
<i>nbPoints</i>	Number of points to generate (must be ≥ 2).

Returns

A `std::vector< double >` containing evenly spaced values.

Exceptions

<i>std::invalid_argument</i>	if <code>start $\geq$ end</code> or <code>nbPoints $<$ 2</code> .
------------------------------	--

```
auto values = MathUtils::linspace( 0.0, 1.0, 5 );  
// values = { 0.0, 0.25, 0.5, 0.75, 1.0 }
```

The documentation for this class was generated from the following files:

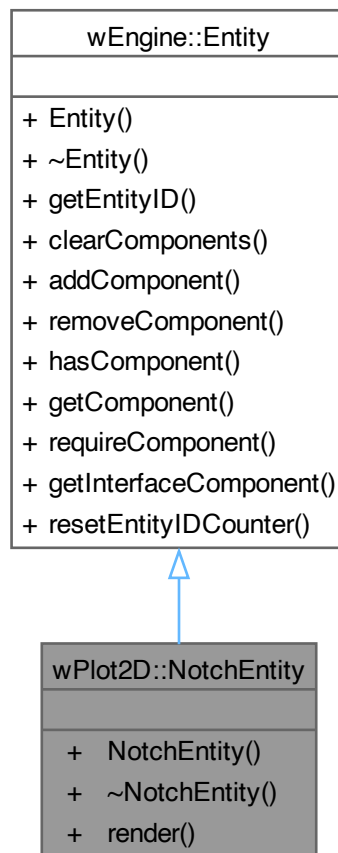
- [wMathUtils.hpp](#)
- [wMathUtils.cpp](#)

7.20 wPlot2D::NotchEntity Class Reference

Represents a single tick mark ("notch") on a 2D axis.

```
#include <wNotchEntity.hpp>
```

Inheritance diagram for wPlot2D::NotchEntity:



Public Member Functions

- `NotchEntity` (`AxisType` type)
Constructs a `NotchEntity` aligned to a given axis.
- virtual `~NotchEntity` ()=default
Virtual destructor.
- void `render` (`sf::RenderWindow &window`)
Renders the notch using SFML.

Public Member Functions inherited from `wEngine::Entity`

- `Entity` ()
- virtual `~Entity` ()
- unsigned int `getEntityID` () const
Returns the unique ID associated with this entity.
- void `clearComponents` ()
Removes all components currently attached to the entity.

- `template<typename T, typename... Args>`
`std::shared_ptr< T > addComponent (Args &&... args)`
Adds a new component of type T to the entity.
- `template<typename T>`
`void removeComponent ()`
Removes the component of type T from the entity.
- `template<typename T>`
`bool hasComponent () const noexcept`
Checks whether the entity has a component of type T.
- `template<typename T>`
`std::shared_ptr< T > getComponent () const`
Retrieves the component of type T attached to the entity.
- `template<typename T>`
`std::shared_ptr< T > requireComponent (const std::string &context="") const`
Retrieves the component of type T and throws if it's missing.
- `template<typename Interface>`
`std::shared_ptr< Interface > getInterfaceComponent () const`
Returns the first component that implements the specified interface.

Additional Inherited Members

Static Public Member Functions inherited from [wEngine::Entity](#)

- `static void resetEntityIDCounter ()`
Resets the global entity ID counter to zero.

7.20.1 Detailed Description

Represents a single tick mark ("notch") on a 2D axis.

A [NotchEntity](#) is a visual element used to mark intervals along a coordinate axis, helping users interpret scale in a plot. It is rendered as a small filled rectangle, oriented perpendicularly to its associated axis (`X_AXIS` or `Y_AXIS`).

7.20.1.0.1 Components required:

- `PositionComponent`: specifies the top-left pixel position.
- `ColorComponent`: defines the notch color.
- `ThicknessComponent`: defines the thickness (along the axis).
- `LengthComponent`: defines the length (perpendicular to the axis).

7.20.1.0.2 Orientation:

- `AxisType::X_AXIS` : vertical notch (aligned with Y),
- `AxisType::Y_AXIS` : horizontal notch (aligned with X).

The actual rendering logic is handled internally using `sf::RectangleShape`.

Note

Components must be added externally (typically by `AxisEntity::addNotches()`).

See also

[AxisEntity](#), [AxisType](#), [PositionComponent](#), [LengthComponent](#), [ThicknessComponent](#)

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.20.2 Constructor & Destructor Documentation

7.20.2.1 NotchEntity()

```
wPlot2D::NotchEntity::NotchEntity (
    AxisType type)
```

Constructs a `NotchEntity` aligned to a given axis.

Parameters

<i>type</i>	The axis type (X or Y) which determines notch orientation.
-------------	--

7.20.2.2 ~NotchEntity()

```
virtual wPlot2D::NotchEntity::~~NotchEntity () [virtual], [default]
```

Virtual destructor.

7.20.3 Member Function Documentation

7.20.3.1 render()

```
void wPlot2D::NotchEntity::render (
    sf::RenderWindow & window)
```

Renders the notch using SFML.

Builds a rectangle from ECS components (position, thickness, length, color) and draws it in the render window.

Exceptions

<code>std::runtime_error</code>	if any required component is missing.
---------------------------------	---------------------------------------

Parameters

<code>window</code>	The render window to draw onto.
---------------------	---------------------------------

The documentation for this class was generated from the following files:

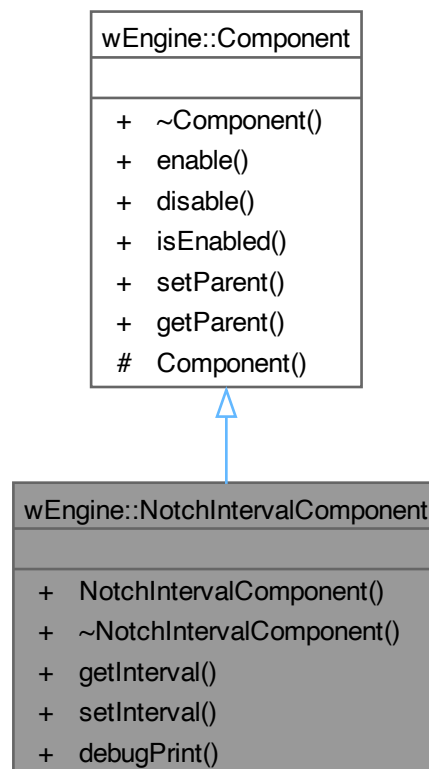
- [wNotchEntity.hpp](#)
- [wNotchEntity.cpp](#)

7.21 wEngine::NotchIntervalComponent Class Reference

ECS component that defines the interval between notches on an axis.

```
#include <wNotchIntervalComponent.hpp>
```

Inheritance diagram for wEngine::NotchIntervalComponent:



Public Member Functions

- [NotchIntervalComponent](#) (float interval=1.0f)
Constructs a [NotchIntervalComponent](#) with a given interval.
- virtual [~NotchIntervalComponent](#) ()=default
- float [getInterval](#) () const
Returns the current spacing between notches.
- void [setInterval](#) (float newInterval)
Sets a new interval between notches.
- void [debugPrint](#) () const
Outputs the current interval to standard output for debugging.

Public Member Functions inherited from [wEngine::Component](#)

- virtual [~Component](#) ()=default
- virtual void [enable](#) ()
- virtual void [disable](#) ()
- bool [isEnabled](#) () const
Checks whether the component is currently active.
- void [setParent](#) ([Entity](#) *parent)
Sets the parent entity of this component.
- [Entity](#) * [getParent](#) () const
Returns the parent entity of this component.

Additional Inherited Members

Protected Member Functions inherited from [wEngine::Component](#)

- [Component](#) ()
Protected constructor to restrict instantiation to derived classes.

7.21.1 Detailed Description

ECS component that defines the interval between notches on an axis.

This component stores the spacing (in logical units) between visual ticks or markers on an axis, such as those used in coordinate grids or charts.

It enables the placement of regular visual markers and may be queried by rendering systems.

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.21.2 Constructor & Destructor Documentation

7.21.2.1 NotchIntervalComponent()

```
wEngine::NotchIntervalComponent::NotchIntervalComponent (
    float interval = 1.0f)
```

Constructs a [NotchIntervalComponent](#) with a given interval.

Parameters

<i>interval</i>	Distance between notches in logical units (must be > 0). Default is 1.0f.
-----------------	---

Exceptions

<i>std::invalid_argument</i>	if interval is not strictly positive.
------------------------------	---------------------------------------

7.21.2.2 ~NotchIntervalComponent()

```
virtual wEngine::NotchIntervalComponent::~~NotchIntervalComponent () [virtual], [default]
```

7.21.3 Member Function Documentation**7.21.3.1 getInterval()**

```
float wEngine::NotchIntervalComponent::getInterval () const [nodiscard]
```

Returns the current spacing between notches.

Returns

The interval, expressed in logical units.

7.21.3.2 setInterval()

```
void wEngine::NotchIntervalComponent::setInterval (
    float newInterval)
```

Sets a new interval between notches.

Parameters

<i>newInterval</i>	New spacing value (must be > 0).
--------------------	----------------------------------

Exceptions

<i>std::invalid_argument</i>	if newInterval is not strictly positive.
------------------------------	--

7.21.3.3 debugPrint()

```
void wEngine::NotchIntervalComponent::debugPrint () const
```

Outputs the current interval to standard output for debugging.

The documentation for this class was generated from the following files:

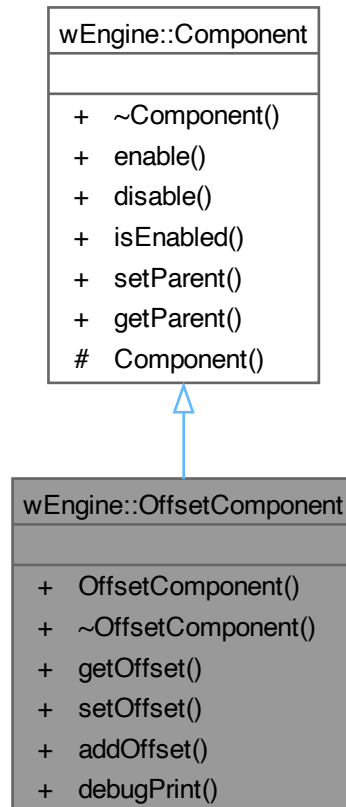
- [wNotchIntervalComponent.hpp](#)
- [wNotchIntervalComponent.cpp](#)

7.22 wEngine::OffsetComponent Class Reference

ECS component that defines a logical coordinate offset.

```
#include <wOffsetComponent.hpp>
```

Inheritance diagram for wEngine::OffsetComponent:



Public Member Functions

- `OffsetComponent` (`sf::Vector2f` offset=`sf::Vector2f(0.0f, 0.0f)`)
Constructs the component with an optional offset.
- virtual `~OffsetComponent` ()=default
- `sf::Vector2f` `getOffset` () const
Returns the current offset.
- void `setOffset` (`sf::Vector2f` offset)
Updates the logical offset value.
- void `addOffset` (`sf::Vector2f` delta)
Increments the current offset by a given vector.
- void `debugPrint` () const
Outputs the current offset value to the console for debugging.

Public Member Functions inherited from `wEngine::Component`

- virtual `~Component()`=default
- virtual void `enable()`
- virtual void `disable()`
- bool `isEnabled()` const
Checks whether the component is currently active.
- void `setParent(Entity *parent)`
Sets the parent entity of this component.
- `Entity * getParent()` const
Returns the parent entity of this component.

Additional Inherited Members

Protected Member Functions inherited from `wEngine::Component`

- `Component()`
Protected constructor to restrict instantiation to derived classes.

7.22.1 Detailed Description

ECS component that defines a logical coordinate offset.

This component stores a 2D offset (in logical units) applied to graphical elements such as axes, curves, titles, labels and data points. It allows changing the visual reference frame without affecting pixel-based positioning.

7.22.1.0.1 Usage Examples:

- With offset = (0, 0), logical coordinates are drawn as-is.
- With offset = (-20, 0), the visual origin is shifted 20 units to the right.

The offset is often combined with the origin and scale to compute actual pixel positions.

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.22.2 Constructor & Destructor Documentation

7.22.2.1 OffsetComponent()

```
wEngine::OffsetComponent::OffsetComponent (
    sf::Vector2f offset = sf::Vector2f( 0.0f, 0.0f ))
```

Constructs the component with an optional offset.

Parameters

<i>offset</i>	A 2D vector representing the logical offset (default is (0.0f, 0.0f)).
---------------	--

7.22.2.2 ~OffsetComponent()

```
virtual wEngine::OffsetComponent::~~OffsetComponent () [virtual], [default]
```

7.22.3 Member Function Documentation**7.22.3.1 getOffset()**

```
sf::Vector2f wEngine::OffsetComponent::getOffset () const [nodiscard]
```

Returns the current offset.

Returns

A 2D vector representing the logical offset.

7.22.3.2 setOffset()

```
void wEngine::OffsetComponent::setOffset (
    sf::Vector2f offset)
```

Updates the logical offset value.

Parameters

<i>offset</i>	The new offset to apply.
---------------	--------------------------

7.22.3.3 addOffset()

```
void wEngine::OffsetComponent::addOffset (
    sf::Vector2f delta)
```

Increments the current offset by a given vector.

Parameters

<i>delta</i>	The additional offset to add to the current value.
--------------	--

7.22.3.4 debugPrint()

```
void wEngine::OffsetComponent::debugPrint () const
```

Outputs the current offset value to the console for debugging.

The documentation for this class was generated from the following files:

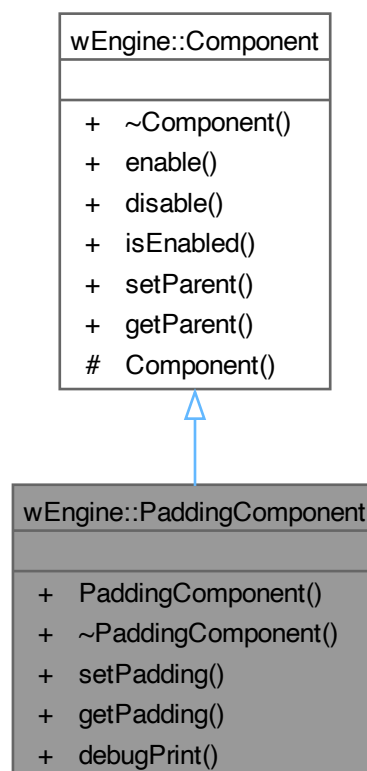
- [wOffsetComponent.hpp](#)
- [wOffsetComponent.cpp](#)

7.23 wEngine::PaddingComponent Class Reference

ECS component representing internal padding for UI-like elements.

```
#include <wPaddingComponent.hpp>
```

Inheritance diagram for wEngine::PaddingComponent:



Public Member Functions

- **PaddingComponent** (sf::Vector2f padding=sf::Vector2f(0.0f, 0.0f))
*Constructs a **PaddingComponent** with optional initial padding.*
- virtual **~PaddingComponent** ()=default
- void **setPadding** (sf::Vector2f padding)
Sets the padding vector.
- sf::Vector2f **getPadding** () const
Returns the current padding values.
- void **debugPrint** () const
Prints the current padding values to standard output for debugging.

Public Member Functions inherited from wEngine::Component

- virtual **~Component** ()=default
- virtual void **enable** ()
- virtual void **disable** ()
- bool **isEnabled** () const
Checks whether the component is currently active.
- void **setParent** (Entity *parent)
Sets the parent entity of this component.
- Entity * **getParent** () const
Returns the parent entity of this component.

Additional Inherited Members

Protected Member Functions inherited from wEngine::Component

- **Component** ()
Protected constructor to restrict instantiation to derived classes.

7.23.1 Detailed Description

ECS component representing internal padding for UI-like elements.

This component encapsulates a 2D padding vector (horizontal and vertical) that can be used to add internal spacing between a visual element (e.g., a title or a frame) and its boundary.

7.23.1.0.1 Padding Convention:

- x corresponds to horizontal padding (left and right),
- y corresponds to vertical padding (top and bottom).

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.23.2 Constructor & Destructor Documentation

7.23.2.1 PaddingComponent()

```
wEngine::PaddingComponent::PaddingComponent (
    sf::Vector2f padding = sf::Vector2f( 0.0f, 0.0f ))
```

Constructs a [PaddingComponent](#) with optional initial padding.

Parameters

<i>padding</i>	Padding vector (x, y) (default is (0.0f, 0.0f)).
----------------	--

7.23.2.2 ~PaddingComponent()

```
virtual wEngine::PaddingComponent::~~PaddingComponent () [virtual], [default]
```

7.23.3 Member Function Documentation

7.23.3.1 setPadding()

```
void wEngine::PaddingComponent::setPadding (
    sf::Vector2f padding)
```

Sets the padding vector.

Parameters

<i>padding</i>	New padding values (x, y).
----------------	----------------------------

7.23.3.2 getPadding()

```
sf::Vector2f wEngine::PaddingComponent::getPadding () const [nodiscard]
```

Returns the current padding values.

Returns

Padding vector (x, y).

7.23.3.3 debugPrint()

```
void wEngine::PaddingComponent::debugPrint () const
```

Prints the current padding values to standard output for debugging.

The documentation for this class was generated from the following files:

- [wPaddingComponent.hpp](#)
- [wPaddingComponent.cpp](#)

7.24 wEngine::PathUtils Class Reference

Utility class providing static functions for managing executable and resource paths across platforms.

```
#include <wPathUtils.hpp>
```

Static Public Member Functions

- static std::string [getExecutablePath](#) ()
Returns the absolute path to the current executable.
- static std::string [getExecutableDir](#) ()
Returns the directory containing the current executable.

7.24.1 Detailed Description

Utility class providing static functions for managing executable and resource paths across platforms.

This class retrieves the absolute path of the current executable or its parent directory, in a portable way (macOS, Windows, Linux). It is particularly useful for locating resources such as fonts, images, or configuration files relative to the application binary.

All methods are static and do not require instantiation.

Note

If the path cannot be determined, a `std::runtime_error` is thrown.

7.24.1.0.1 Notes on unusual headers:

- `<mach-o/dyld.h>` (macOS): provides `_NSGetExecutablePath`, part of the Mach-O dynamic loader API. Documentation: see Apple Developer docs (man page: `man 3 dyld`).
- `<windows.h>` (Windows): provides `GetModuleFileNameA`, part of the Win32 API. Documentation: see Microsoft Learn (GetModuleFileName function).
- `<unistd.h>` + `/proc/self/exe` (Linux): `/proc/self/exe` is a symbolic link exposing the running executable, documented in `man 5 proc`.

These APIs are not C++ standard and must be used with care, as they are platform-dependent.

See also

`std::filesystem` for path manipulations.

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.24.2 Member Function Documentation

7.24.2.1 `getExecutablePath()`

```
std::string wEngine::PathUtils::getExecutablePath () [static], [nodiscard]
```

Returns the absolute path to the current executable.

Platform-specific APIs are used:

- macOS: `_NSGetExecutablePath`
- Windows: `GetModuleFileNameA`
- Linux: `/proc/self/exe`

Returns

Absolute path to the binary (e.g., `"/path/to/MyApp.app/Contents/MacOS/MyApp"`).

Exceptions

<code>std::runtime_error</code>	if the path cannot be resolved.
---------------------------------	---------------------------------

7.24.2.2 `getExecutableDir()`

```
std::string wEngine::PathUtils::getExecutableDir () [static], [nodiscard]
```

Returns the directory containing the current executable.

This is often used as a base path to resolve relative resource paths.

Returns

Directory containing the binary (e.g., `"/path/to/MyApp.app/Contents/MacOS"`).

Exceptions

<code>std::runtime_error</code>	if the path cannot be resolved.
---------------------------------	---------------------------------

The documentation for this class was generated from the following files:

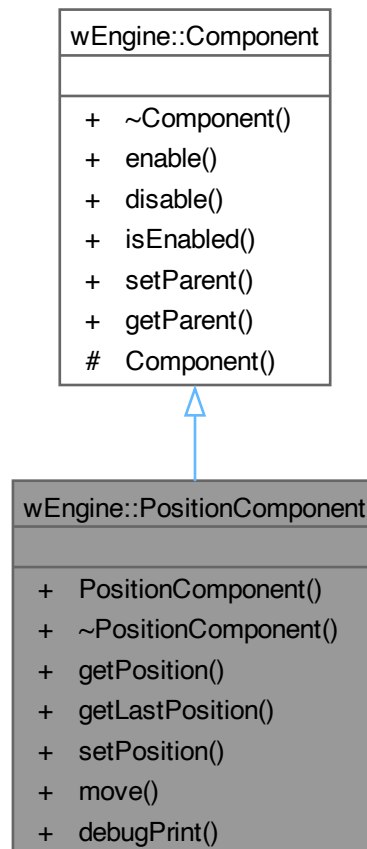
- [wPathUtils.hpp](#)
- [wPathUtils.cpp](#)

7.25 wEngine::PositionComponent Class Reference

ECS component storing the position of an entity in 2D space and supports movement tracking.

```
#include <wPositionComponent.hpp>
```

Inheritance diagram for wEngine::PositionComponent:



Public Member Functions

- `PositionComponent` (sf::Vector2f position=sf::Vector2f(0.0f, 0.0f))
Constructs a `PositionComponent` with the given position.
- virtual `~PositionComponent` ()=default
- sf::Vector2f `getPosition` () const
Gets the current position of the component.
- sf::Vector2f `getLastPosition` () const
Returns the previous position (before the last move).
- void `setPosition` (sf::Vector2f newPosition)
Sets the 2D position to a new value.
- void `move` (const sf::Vector2f &offset)
Moves the position by an offset.
- void `debugPrint` () const
Outputs the position (x, y) to standard output for debugging.

Public Member Functions inherited from `wEngine::Component`

- virtual `~Component()`=default
- virtual void `enable()`
- virtual void `disable()`
- bool `isEnabled()` const
Checks whether the component is currently active.
- void `setParent(Entity *parent)`
Sets the parent entity of this component.
- `Entity * getParent()` const
Returns the parent entity of this component.

Additional Inherited Members

Protected Member Functions inherited from `wEngine::Component`

- `Component()`
Protected constructor to restrict instantiation to derived classes.

7.25.1 Detailed Description

ECS component storing the position of an entity in 2D space and supports movement tracking.

This component holds a 2D vector representing the current and previous spatial position of its parent entity.

7.25.1.0.1 Usage Examples:

- Default position at origin:

```
addComponent< wEngine::PositionComponent >( );
```
- Custom position:

```
addComponent< wEngine::PositionComponent >( sf::Vector2f( 100.0f, 200.0f ) );
```

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.25.2 Constructor & Destructor Documentation

7.25.2.1 PositionComponent()

```
wEngine::PositionComponent::PositionComponent (
    sf::Vector2f position = sf::Vector2f( 0.0f, 0.0f ) )
```

Constructs a `PositionComponent` with the given position.

Parameters

<i>position</i>	Initial position (default is (0.0f, 0.0f)).
-----------------	---

7.25.2.2 ~PositionComponent()

```
virtual wEngine::PositionComponent::~~PositionComponent () [virtual], [default]
```

7.25.3 Member Function Documentation

7.25.3.1 getPosition()

```
sf::Vector2f wEngine::PositionComponent::getPosition () const [nodiscard]
```

Gets the current position of the component.

Returns

The current 2D position vector.

7.25.3.2 getLastPosition()

```
sf::Vector2f wEngine::PositionComponent::getLastPosition () const [nodiscard]
```

Returns the previous position (before the last move).

Returns

The last recorded position.

7.25.3.3 setPosition()

```
void wEngine::PositionComponent::setPosition (
    sf::Vector2f newPosition)
```

Sets the 2D position to a new value.

Parameters

<i>newPosition</i>	The new position.
--------------------	-------------------

7.25.3.4 move()

```
void wEngine::PositionComponent::move (
    const sf::Vector2f & offset)
```

Moves the position by an offset.

Parameters

<i>offset</i>	The offset to add to the current position.
---------------	--

Stores the current position as the last position before applying the offset.

7.25.3.5 debugPrint()

```
void wEngine::PositionComponent::debugPrint () const
```

Outputs the position (x, y) to standard output for debugging.

The documentation for this class was generated from the following files:

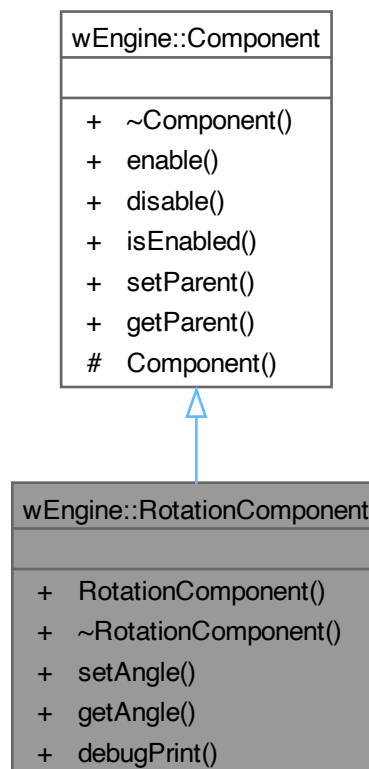
- [wPositionComponent.hpp](#)
- [wPositionComponent.cpp](#)

7.26 wEngine::RotationComponent Class Reference

ECS component that stores a rotation angle (in degrees).

```
#include <wRotationComponent.hpp>
```

Inheritance diagram for wEngine::RotationComponent:



Public Member Functions

- [RotationComponent](#) (float angle=0.0f)
Construct a [RotationComponent](#) with an initial angle.
- virtual [~RotationComponent](#) ()=default
- void [setAngle](#) (float angle)
Set the rotation angle.
- float [getAngle](#) () const
Get the current rotation angle.
- void [debugPrint](#) () const
Outputs the current angle value to the console for debugging.

Public Member Functions inherited from [wEngine::Component](#)

- virtual [~Component](#) ()=default
- virtual void [enable](#) ()
- virtual void [disable](#) ()
- bool [isEnabled](#) () const
Checks whether the component is currently active.
- void [setParent](#) ([Entity](#) *parent)
Sets the parent entity of this component.
- [Entity](#) * [getParent](#) () const
Returns the parent entity of this component.

Additional Inherited Members**Protected Member Functions inherited from [wEngine::Component](#)**

- [Component](#) ()
Protected constructor to restrict instantiation to derived classes.

7.26.1 Detailed Description

ECS component that stores a rotation angle (in degrees).

This component can be attached to any entity that needs to be rotated relative to its logical origin. The angle is expressed in degrees (positive values correspond to counter-clockwise rotation).

7.26.1.0.1 Usage Example:

```
addComponent< wEngine::RotationComponent > ( 45.0f ); // Rotate 45° counter-clockwise
```

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.26.2 Constructor & Destructor Documentation**7.26.2.1 RotationComponent()**

```
wEngine::RotationComponent::RotationComponent (
    float angle = 0.0f)
```

Construct a [RotationComponent](#) with an initial angle.

Parameters

<i>angle</i>	Initial rotation angle in degrees (default: 0.0f).
--------------	--

7.26.2.2 ~RotationComponent()

```
virtual wEngine::RotationComponent::~~RotationComponent () [virtual], [default]
```

7.26.3 Member Function Documentation**7.26.3.1 setAngle()**

```
void wEngine::RotationComponent::setAngle (  
    float angle)
```

Set the rotation angle.

Parameters

<i>angle</i>	New rotation angle in degrees.
--------------	--------------------------------

7.26.3.2 getAngle()

```
float wEngine::RotationComponent::getAngle () const [nodiscard]
```

Get the current rotation angle.

Returns

The stored angle in degrees.

7.26.3.3 debugPrint()

```
void wEngine::RotationComponent::debugPrint () const
```

Outputs the current angle value to the console for debugging.

The documentation for this class was generated from the following files:

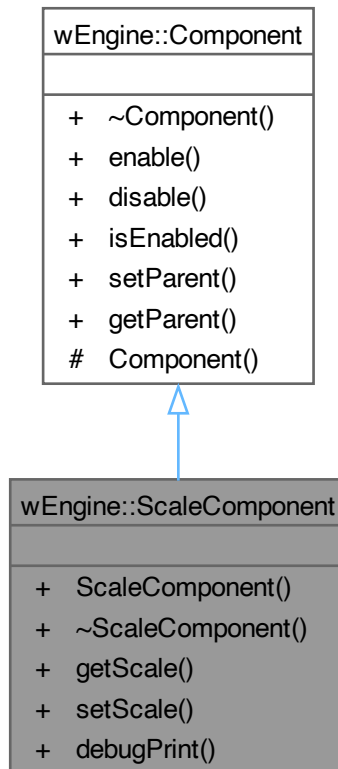
- [wRotationComponent.hpp](#)
- [wRotationComponent.cpp](#)

7.27 wEngine::ScaleComponent Class Reference

ECS component that defines the scaling factor for an entity in 2D space.

```
#include <wScaleComponent.hpp>
```

Inheritance diagram for wEngine::ScaleComponent:



Public Member Functions

- `ScaleComponent` (`sf::Vector2f` scale={ 1.0f, 1.0f })
Constructs a `ScaleComponent` with the given scale.
- virtual `~ScaleComponent` ()=default
- `sf::Vector2f` `getScale` () const
Retrieves the current scale factor.
- void `setScale` (`sf::Vector2f` newScale)
Sets a new scale factor.
- void `debugPrint` () const
Outputs the scale to standard output for debugging.

Public Member Functions inherited from [wEngine::Component](#)

- virtual [~Component](#) ()=default
- virtual void [enable](#) ()
- virtual void [disable](#) ()
- bool [isEnabled](#) () const
Checks whether the component is currently active.
- void [setParent](#) ([Entity](#) *parent)
Sets the parent entity of this component.
- [Entity](#) * [getParent](#) () const
Returns the parent entity of this component.

Additional Inherited Members

Protected Member Functions inherited from [wEngine::Component](#)

- [Component](#) ()
Protected constructor to restrict instantiation to derived classes.

7.27.1 Detailed Description

ECS component that defines the scaling factor for an entity in 2D space.

The scale determines how much the entity is scaled along the X and Y axes. It is typically used to transform logical coordinates into pixel coordinates.

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.27.2 Constructor & Destructor Documentation

7.27.2.1 ScaleComponent()

```
wEngine::ScaleComponent::ScaleComponent (
    sf::Vector2f scale = { 1.0f, 1.0f })
```

Constructs a [ScaleComponent](#) with the given scale.

Parameters

<i>scale</i>	Initial scale vector (must be strictly positive, defaults is (1.0f, 1.0f).
--------------	--

Exceptions

<code>std::invalid_argument</code>	if any scale component is non-positive.
------------------------------------	---

7.27.2.2 ~ScaleComponent()

```
virtual wEngine::ScaleComponent::~~ScaleComponent () [virtual], [default]
```

7.27.3 Member Function Documentation**7.27.3.1 getScale()**

```
sf::Vector2f wEngine::ScaleComponent::getScale () const [nodiscard]
```

Retrieves the current scale factor.

Returns

The scale as an `sf::Vector2f` (X and Y scale).

7.27.3.2 setScale()

```
void wEngine::ScaleComponent::setScale (
    sf::Vector2f newScale)
```

Sets a new scale factor.

Parameters

<i>newScale</i>	New scale vector (must be strictly positive).
-----------------	---

Exceptions

<code>std::invalid_argument</code>	if newScale has non-positive values.
------------------------------------	--------------------------------------

7.27.3.3 debugPrint()

```
void wEngine::ScaleComponent::debugPrint () const
```

Outputs the scale to standard output for debugging.

The documentation for this class was generated from the following files:

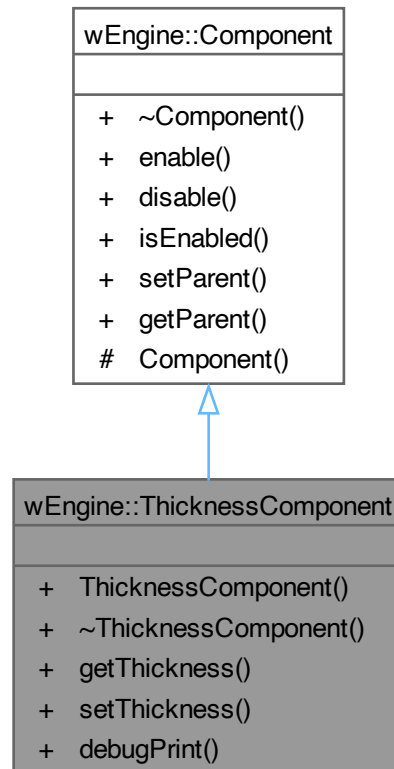
- [wScaleComponent.hpp](#)
- [wScaleComponent.cpp](#)

7.28 wEngine::ThicknessComponent Class Reference

ECS component that defines the thickness of a drawable object.

```
#include <wThicknessComponent.hpp>
```

Inheritance diagram for wEngine::ThicknessComponent:



Public Member Functions

- `ThicknessComponent` (float thickness=2.0f)
Constructs the component with an initial positive thickness.
- virtual `~ThicknessComponent` ()=default
- float `getThickness` () const
Returns the current thickness.
- void `setThickness` (float newThickness)
Sets a new thickness value.
- void `debugPrint` () const
Outputs the current thickness value to the console for debugging.

Public Member Functions inherited from wEngine::Component

- virtual `~Component()`=default
- virtual void `enable()`
- virtual void `disable()`
- bool `isEnabled()` const
Checks whether the component is currently active.
- void `setParent(Entity *parent)`
Sets the parent entity of this component.
- `Entity *` `getParent()` const
Returns the parent entity of this component.

Additional Inherited Members

Protected Member Functions inherited from wEngine::Component

- `Component()`
Protected constructor to restrict instantiation to derived classes.

7.28.1 Detailed Description

ECS component that defines the thickness of a drawable object.

This component stores a positive float value representing the thickness (in pixels) of lines or shapes (e.g., axes, borders). The value must be strictly positive.

Examples:

- A thickness of 1.0f draws a thin line.
- A thickness of 4.0f produces a bold axis.

Exceptions

<code>std::invalid_argument</code>	if the value is zero or negative.
------------------------------------	-----------------------------------

7.28.1.0.1 Usage Examples:

- A thickness of 1.0f draws a thin line.
- A thickness of 4.0f produces a bold axis.

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.28.2 Constructor & Destructor Documentation

7.28.2.1 ThicknessComponent()

```
wEngine::ThicknessComponent::ThicknessComponent (
    float thickness = 2.0f)
```

Constructs the component with an initial positive thickness.

Parameters

<i>thickness</i>	Initial thickness value (default is 2.0f).
------------------	--

Exceptions

<i>std::invalid_argument</i>	if the value is zero or negative.
------------------------------	-----------------------------------

7.28.2.2 ~ThicknessComponent()

```
virtual wEngine::ThicknessComponent::~~ThicknessComponent () [virtual], [default]
```

7.28.3 Member Function Documentation

7.28.3.1 getThickness()

```
float wEngine::ThicknessComponent::getThickness () const [nodiscard]
```

Returns the current thickness.

Returns

A positive float representing the line thickness (in pixels).

7.28.3.2 setThickness()

```
void wEngine::ThicknessComponent::setThickness (  
    float newThickness)
```

Sets a new thickness value.

Parameters

<i>newThickness</i>	A strictly positive float.
---------------------	----------------------------

Exceptions

<i>std::invalid_argument</i>	if the value is zero or negative.
------------------------------	-----------------------------------

7.28.3.3 debugPrint()

```
void wEngine::ThicknessComponent::debugPrint () const
```

Outputs the current thickness value to the console for debugging.

The documentation for this class was generated from the following files:

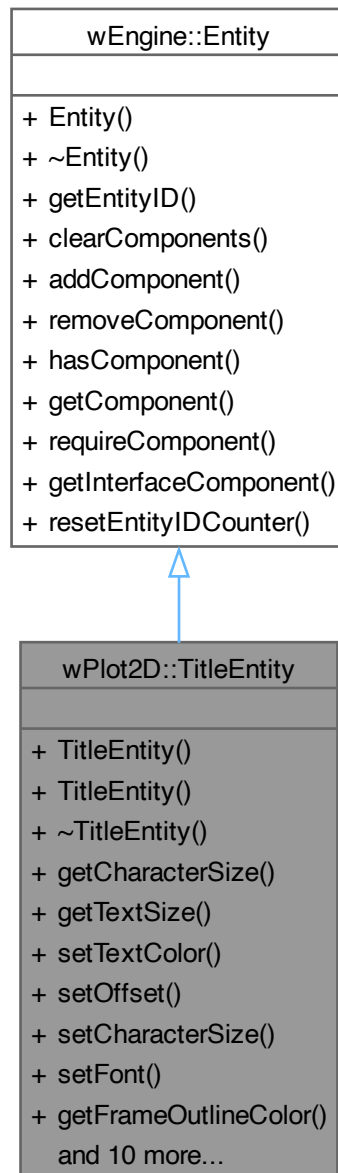
- [wThicknessComponent.hpp](#)
- [wThicknessComponent.cpp](#)

7.29 wPlot2D::TitleEntity Class Reference

Represents a textual label (typically an axis title or main plot title) in a 2D plot.

```
#include <wTitleEntity.hpp>
```

Inheritance diagram for wPlot2D::TitleEntity:



Public Member Functions

- [TitleEntity](#) (const sf::Font &font, const std::string &title, bool hasFrame=false)

- Constructs a title entity with specified font and text (UTF-8).*

 - [TitleEntity](#) (const sf::Font &font, const std::wstring &title, bool hasFrame=false)

Constructs a title entity with specified font and wide string (UTF-16/32).
- virtual [~TitleEntity](#) ()=default
- Virtual destructor.*
- unsigned int [getCharacterSize](#) () const
- Returns the current character size.*
- sf::FloatRect [getTextSize](#) () const
- Returns the local bounding box of the title text.*
- void [setTextColor](#) (sf::Color textColor)
- Sets the text color.*
- void [setOffset](#) (sf::Vector2f offset)
- Sets the offset relative to the base anchor position.*
- void [setCharacterSize](#) (unsigned int size)
- Sets the font size (character size) of the title.*
- void [setFont](#) (const sf::Font &font)
- Sets the font reference for the title.*
- sf::Color [getFrameOutlineColor](#) () const
- Returns the current outline color of the frame.*
- sf::Color [getFrameFillColor](#) () const
- Returns the current fill color of the frame.*
- float [getFrameThickness](#) () const
- Returns the frame's outline thickness.*
- sf::Vector2f [getPadding](#) () const
- Returns the internal padding of the frame.*
- bool [isFrameEnabled](#) () const
- Returns whether the title has a visible frame.*
- void [setFrameEnabled](#) (bool enabled)
- Enables or disables the visual frame.*
- void [setFrameOutlineColor](#) (const sf::Color &color)
- Sets the frame's outline color.*
- void [setFrameFillColor](#) (const sf::Color &color)
- Sets the fill color of the frame.*
- void [setFrameThickness](#) (float thickness)
- Sets the thickness of the frame's outline.*
- void [setPadding](#) (sf::Vector2f padding)
- Sets the internal padding of the frame (horizontal and vertical).*
- void [render](#) (sf::RenderWindow &window)
- Renders the title and its frame (if enabled) to the window.*

Public Member Functions inherited from [wEngine::Entity](#)

- [Entity](#) ()
- virtual [~Entity](#) ()
- unsigned int [getEntityID](#) () const
- Returns the unique ID associated with this entity.*
- void [clearComponents](#) ()
- Removes all components currently attached to the entity.*
- template<typename T, typename... Args>
std::shared_ptr< T > [addComponent](#) (Args &&... args)

- Adds a new component of type T to the entity.*

 - `template<typename T>`
`void removeComponent ()`

Removes the component of type T from the entity.
- `template<typename T>`
`bool hasComponent () const noexcept`

Checks whether the entity has a component of type T.
- `template<typename T>`
`std::shared_ptr< T > getComponent () const`

Retrieves the component of type T attached to the entity.
- `template<typename T>`
`std::shared_ptr< T > requireComponent (const std::string &context="") const`

Retrieves the component of type T and throws if it's missing.
- `template<typename Interface>`
`std::shared_ptr< Interface > getInterfaceComponent () const`

Returns the first component that implements the specified interface.

Additional Inherited Members

Static Public Member Functions inherited from [wEngine::Entity](#)

- `static void resetEntityIDCounter ()`
Resets the global entity ID counter to zero.

7.29.1 Detailed Description

Represents a textual label (typically an axis title or main plot title) in a 2D plot.

A [TitleEntity](#) displays text using SFML's `sf::Text`, styled and positioned using ECS components (`PositionComponent`, `OffsetComponent`, `ColorComponent`, `FontComponent`). Optionally, it can display a surrounding rectangular frame ([FrameEntity](#)) with customizable outline, fill color, thickness, and padding.

7.29.1.0.1 Notes:

- The font passed to the constructor must remain valid during the lifetime of the entity (SFML does not copy font data).
- The frame is disabled by default unless explicitly enabled at construction or later.

See also

[FrameEntity](#), [AxisEntity](#), [GraphicsEntity](#)

Author

Wilfried Koch

Copyright

© 2025 Wilfried Koch. All rights reserved.

7.29.2 Constructor & Destructor Documentation

7.29.2.1 TitleEntity() [1/2]

```
wPlot2D::TitleEntity::TitleEntity (
    const sf::Font & font,
    const std::string & title,
    bool hasFrame = false)
```

Constructs a title entity with specified font and text (UTF-8).

Parameters

<i>font</i>	Reference to an externally managed font (must remain valid).
<i>title</i>	The string to display.
<i>hasFrame</i>	Whether a surrounding frame should be displayed.

7.29.2.2 TitleEntity() [2/2]

```
wPlot2D::TitleEntity::TitleEntity (
    const sf::Font & font,
    const std::wstring & title,
    bool hasFrame = false)
```

Constructs a title entity with specified font and wide string (UTF-16/32).

Parameters

<i>font</i>	Reference to an externally managed font (must remain valid).
<i>title</i>	The wide string to display.
<i>hasFrame</i>	Whether a surrounding frame should be displayed.

7.29.2.3 ~TitleEntity()

```
virtual wPlot2D::TitleEntity::~~TitleEntity () [virtual], [default]
```

Virtual destructor.

7.29.3 Member Function Documentation

7.29.3.1 getCharacterSize()

```
unsigned int wPlot2D::TitleEntity::getCharacterSize () const [nodiscard]
```

Returns the current character size.

Returns

Character size (font size in pixels).

7.29.3.2 getTextSize()

```
sf::FloatRect wPlot2D::TitleEntity::getTextSize () const [nodiscard]
```

Returns the local bounding box of the title text.

Returns

A FloatRect representing the size and local origin of the text.

7.29.3.3 setTextColor()

```
void wPlot2D::TitleEntity::setTextColor (
    sf::Color textColor)
```

Sets the text color.

Parameters

<i>textColor</i>	New SFML color.
------------------	-----------------

7.29.3.4 setOffset()

```
void wPlot2D::TitleEntity::setOffset (
    sf::Vector2f offset)
```

Sets the offset relative to the base anchor position.

Parameters

<i>offset</i>	Displacement vector in pixels.
---------------	--------------------------------

7.29.3.5 setCharacterSize()

```
void wPlot2D::TitleEntity::setCharacterSize (
    unsigned int size)
```

Sets the font size (character size) of the title.

Parameters

<i>size</i>	Size in pixels.
-------------	-----------------

7.29.3.6 setFont()

```
void wPlot2D::TitleEntity::setFont (
    const sf::Font & font)
```

Sets the font reference for the title.

Parameters

<i>font</i>	Reference to an externally managed font (must remain valid).
-------------	--

Note

The font must outlive this entity, otherwise rendering will be invalid.

7.29.3.7 getFrameOutlineColor()

```
sf::Color wPlot2D::TitleEntity::getFrameOutlineColor () const [nodiscard]
```

Returns the current outline color of the frame.

Returns

SFML color.

7.29.3.8 getFrameFillColor()

```
sf::Color wPlot2D::TitleEntity::getFrameFillColor () const [nodiscard]
```

Returns the current fill color of the frame.

Returns

SFML color.

7.29.3.9 getFrameThickness()

```
float wPlot2D::TitleEntity::getFrameThickness () const [nodiscard]
```

Returns the frame's outline thickness.

Returns

Thickness in pixels.

7.29.3.10 getPadding()

```
sf::Vector2f wPlot2D::TitleEntity::getPadding () const [nodiscard]
```

Returns the internal padding of the frame.

Returns

Padding vector { *x*, *y* } in pixels.

7.29.3.11 isFrameEnabled()

```
bool wPlot2D::TitleEntity::isFrameEnabled () const [nodiscard]
```

Returns whether the title has a visible frame.

Returns

True if frame is enabled.

7.29.3.12 setFrameEnabled()

```
void wPlot2D::TitleEntity::setFrameEnabled (  
    bool enabled)
```

Enables or disables the visual frame.

Parameters

<i>enabled</i>	True to show the frame, false to hide it.
----------------	---

7.29.3.13 setFrameOutlineColor()

```
void wPlot2D::TitleEntity::setFrameOutlineColor (  
    const sf::Color & color)
```

Sets the frame's outline color.

Parameters

<i>color</i>	SFML color.
--------------	-------------

7.29.3.14 setFrameFillColor()

```
void wPlot2D::TitleEntity::setFrameFillColor (  
    const sf::Color & color)
```

Sets the fill color of the frame.

Parameters

<i>color</i>	SFML color.
--------------	-------------

7.29.3.15 setFrameThickness()

```
void wPlot2D::TitleEntity::setFrameThickness (  
    float thickness)
```

Sets the thickness of the frame's outline.

Parameters

<i>thickness</i>	Outline thickness in pixels.
------------------	------------------------------

7.29.3.16 setPadding()

```
void wPlot2D::TitleEntity::setPadding (
    sf::Vector2f padding)
```

Sets the internal padding of the frame (horizontal and vertical).

This creates a margin between the text and the frame borders. Expressed as (horizontal, vertical) padding in pixels.

Parameters

<i>padding</i>	Vector of type (left/right, top/bottom) padding.
----------------	--

7.29.3.17 render()

```
void wPlot2D::TitleEntity::render (
    sf::RenderWindow & window)
```

Renders the title and its frame (if enabled) to the window.

The title position is computed from:

- `PositionComponent` (anchor point),
- `OffsetComponent` (displacement),
- the text's local bounds (centered origin).

If the frame is enabled, it is rendered behind the text, centered with the same anchor point and adjusted using the specified padding.

Parameters

<i>window</i>	The target SFML render window.
---------------	--------------------------------

The documentation for this class was generated from the following files:

- [wTitleEntity.hpp](#)
- [wTitleEntity.cpp](#)

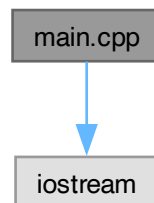
Chapter 8

File Documentation

8.1 main.cpp File Reference

```
#include <iostream>
```

Include dependency graph for main.cpp:



Functions

- int `main` ()

8.1.1 Function Documentation

8.1.1.1 `main()`

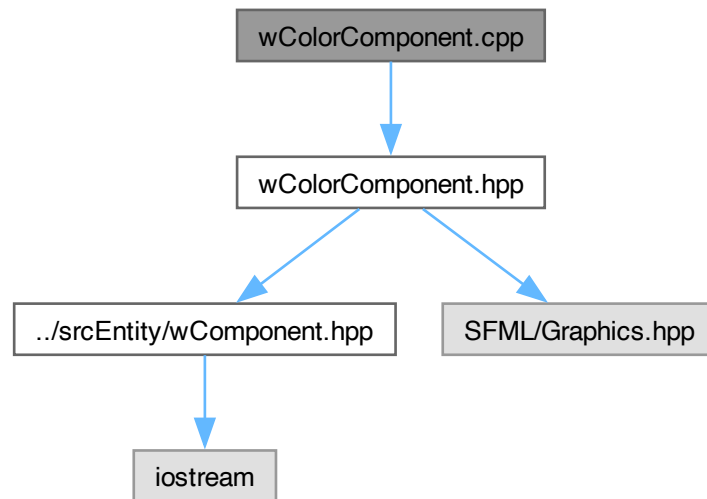
```
int main ()
```

8.2 wColorComponent.cpp File Reference

Implementation of the ColorComponent class.

```
#include "wColorComponent.hpp"
```

Include dependency graph for wColorComponent.cpp:



Namespaces

- namespace [wEngine](#)

8.2.1 Detailed Description

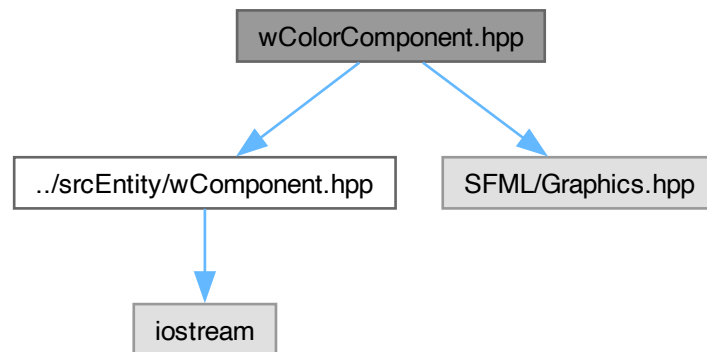
Implementation of the ColorComponent class.

8.3 wColorComponent.hpp File Reference

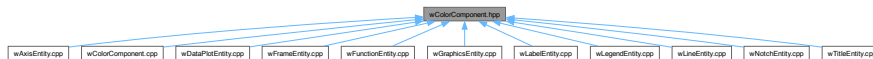
```
#include "../srcEntity/wComponent.hpp"
```

```
#include <SFML/Graphics.hpp>
```

Include dependency graph for wColorComponent.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [wEngine::ColorComponent](#)
ECS component that holds a color value.

Namespaces

- namespace [wEngine](#)

8.4 wColorComponent.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002
00003  +-----+
00004  Created by Wilfried Koch.
00005  Copyright © 2025 Wilfried Koch. All rights reserved.
00006  +-----+
00007  */
00008  #ifndef W_COLOR_COMPONENT_HPP
00009  #define W_COLOR_COMPONENT_HPP
00010
00011  #include "../srcEntity/wComponent.hpp"
00012
00013  #pragma GCC diagnostic push
00014  #pragma GCC diagnostic ignored "-Wfloat-equal"
  
```

```

00015 #pragma GCC diagnostic ignored "-Wswitch-default"
00016 #include <SFML/Graphics.hpp>
00017 #pragma GCC diagnostic pop
00018
00019 namespace wEngine
00020 {
00021
00022     class ColorComponent : public Component
00023     {
00024     public:
00025         ColorComponent( sf::Color color = sf::Color::Black );
00026
00027         /*
00028          * @brief Virtual destructor.
00029          */
00030         virtual ~ColorComponent( ) = default;
00031
00032         [[nodiscard]] sf::Color getColor( ) const;
00033
00034         void setColor( sf::Color newColor );
00035
00036         void debugPrint( ) const;
00037     private:
00038         sf::Color mColor;
00039     };
00040 } //End of namespace wEngine
00041 #endif

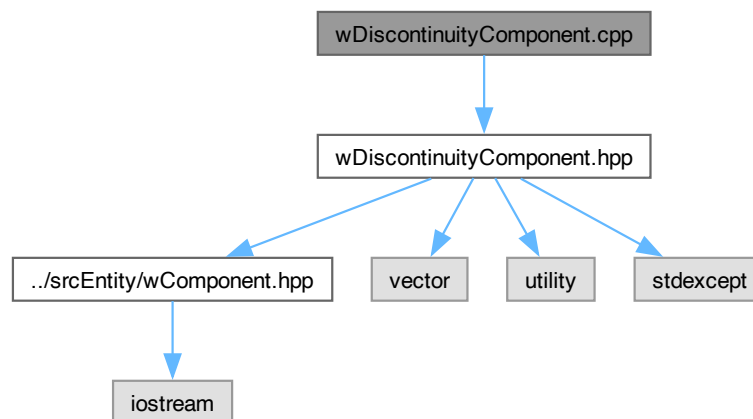
```

8.5 wDiscontinuityComponent.cpp File Reference

Implementation of the DiscontinuityComponent class.

```
#include "wDiscontinuityComponent.hpp"
```

Include dependency graph for wDiscontinuityComponent.cpp:



Namespaces

- namespace `wEngine`

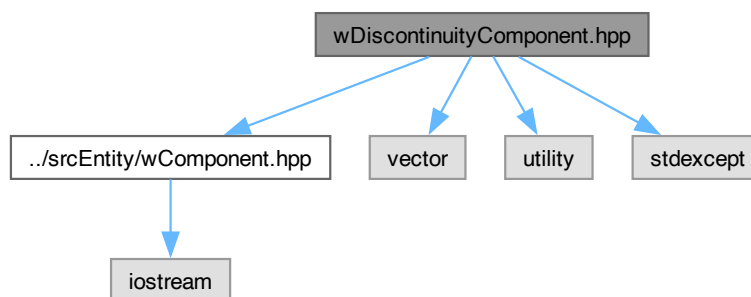
8.5.1 Detailed Description

Implementation of the DiscontinuityComponent class.

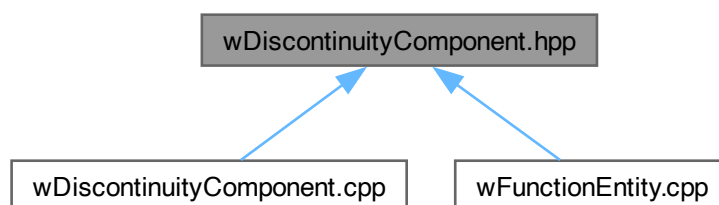
8.6 wDiscontinuityComponent.hpp File Reference

```
#include "../srcEntity/wComponent.hpp"
#include <vector>
#include <utility>
#include <stdexcept>
```

Include dependency graph for wDiscontinuityComponent.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [wEngine::DiscontinuityComponent](#)
ECS component that manages excluded intervals for function plotting.

Namespaces

- namespace [wEngine](#)

8.7 wDiscontinuityComponent.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002  +-----+
00003  Created by Wilfried Koch.
00004  Copyright @ 2025 Wilfried Koch. All rights reserved.
00005  +-----+
00006  */
00007
00008 #ifndef W_DISCONTINUITY_COMPONENT_HPP
00009 #define W_DISCONTINUITY_COMPONENT_HPP
00010
00011 #include "../srcEntity/wComponent.hpp"
00012
00013 #include <vector>
00014 #include <utility>
00015 #include <stdexcept>
00016
00017 namespace wEngine
00018 {
00019
00033     class DiscontinuityComponent : public Component
00034     {
00035     public:
00036         /*
00037          * @brief Default constructor.
00038          */
00039         DiscontinuityComponent( ) = default;
00040
00041         /*
00042          * @brief Virtual destructor.
00043          */
00044         virtual ~DiscontinuityComponent( ) = default;
00045
00050         [[nodiscard]] const std::vector< std::pair< double, double > >& getExcludedIntervals( )
00051         const;
00052
00058         void addExcludedInterval( double min, double max );
00059
00063         void clearExcludedIntervals( );
00064
00070         [[nodiscard]] bool isInExcludedInterval( double x ) const;
00071
00072         /*
00073          * @brief Outputs the excluded intervals to the console for debugging.
00074          */
00075         void debugPrint( ) const;
00076     private:
00077         std::vector< std::pair< double, double > > mExcludedIntervals;
00078     };
00079
00080 } //End of namespace wEngine
00081
00082 #endif

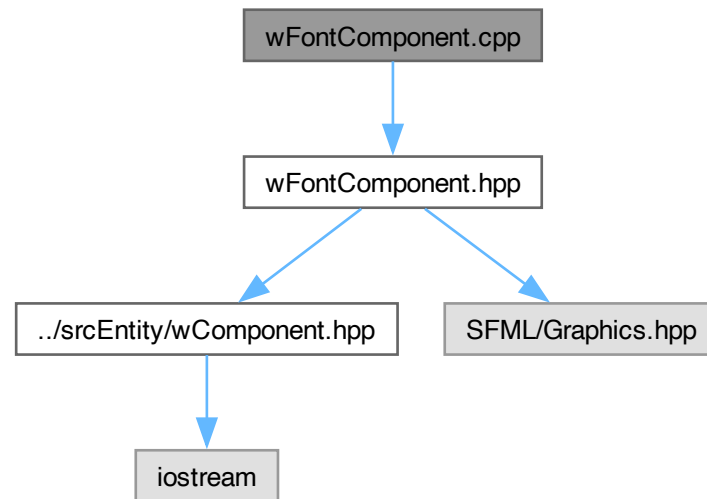
```

8.8 wFontComponent.cpp File Reference

Implementation of the FontComponent class.

```
#include "wFontComponent.hpp"
```

Include dependency graph for wFontComponent.cpp:



Namespaces

- namespace [wEngine](#)

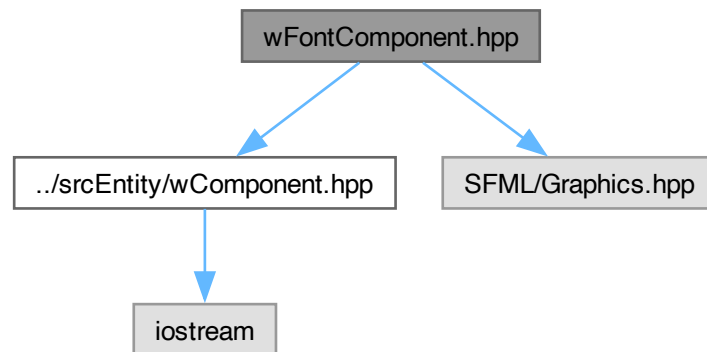
8.8.1 Detailed Description

Implementation of the FontComponent class.

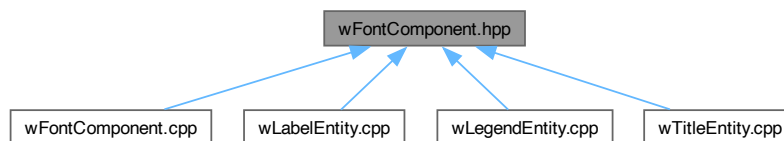
8.9 wFontComponent.hpp File Reference

```
#include "../srcEntity/wComponent.hpp"  
#include <SFML/Graphics.hpp>
```

Include dependency graph for wFontComponent.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [wEngine::FontComponent](#)
Holds a reference to an SFML font for rendering text.

Namespaces

- namespace [wEngine](#)

8.10 wFontComponent.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002
00003  +-----+
00003  Created by Wilfried Koch.
00004  Copyright @ 2025 Wilfried Koch. All rights reserved.
00005
00006  +-----+
00006  */
  
```



```

00007
00008 #ifndef W_FONT_COMPONENT_HPP
00009 #define W_FONT_COMPONENT_HPP
00010
00011 #include "../srcEntity/wComponent.hpp"
00012
00013 #pragma GCC diagnostic push
00014 #pragma GCC diagnostic ignored "-Wfloat-equal"
00015 #pragma GCC diagnostic ignored "-Wswitch-default"
00016 #include <SFML/Graphics.hpp>
00017 #pragma GCC diagnostic pop
00018
00019 namespace wEngine
00020 {
00021
00035     class FontComponent : public Component
00036     {
00037     public:
00042         explicit FontComponent( const sf::Font& font );
00043
00047         ~FontComponent( ) override = default;
00048
00053         const sf::Font& getFont( ) const;
00054
00060         void setFont( const sf::Font& font );
00061
00065         void debugPrint( ) const;
00066
00067     private:
00068         const sf::Font* mFont;
00069     };
00070
00071 } //End of namespace wEngine
00072
00073 #endif

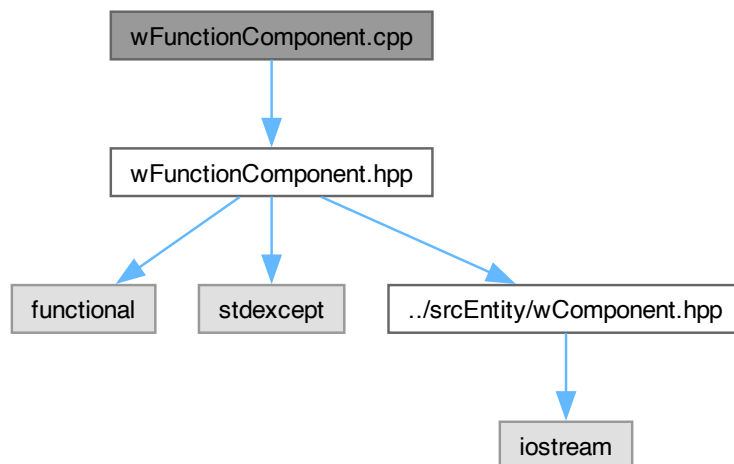
```

8.11 wFunctionComponent.cpp File Reference

Implementation of the FunctionComponent class.

#include "wFunctionComponent.hpp"

Include dependency graph for wFunctionComponent.cpp:



Namespaces

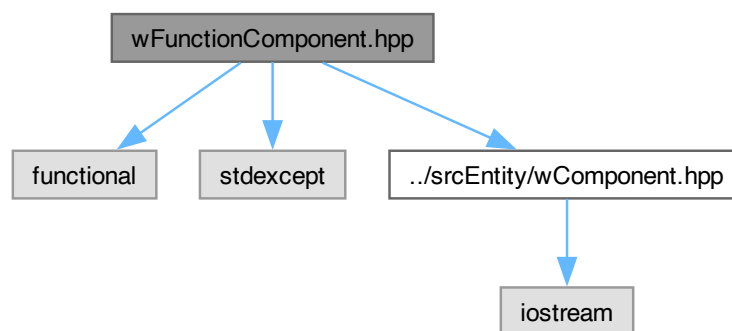
- namespace [wEngine](#)

8.11.1 Detailed Description

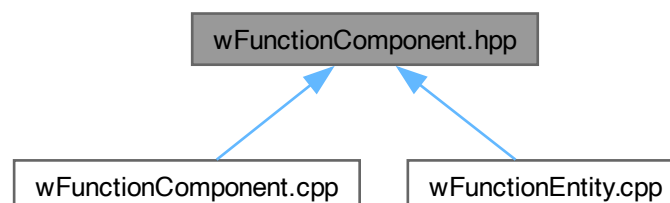
Implementation of the FunctionComponent class.

8.12 wFunctionComponent.hpp File Reference

```
#include <functional>
#include <stdexcept>
#include "../srcEntity/wComponent.hpp"
Include dependency graph for wFunctionComponent.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [wEngine::FunctionComponent](#)
ECS component that stores a mathematical function $f(x)$.

Namespaces

- namespace [wEngine](#)

8.13 wFunctionComponent.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002
00003  +-----+
00003  Created by Wilfried Koch.
00004  Copyright @ 2025 Wilfried Koch. All rights reserved.
00005
00006  +-----+
00006  */
00007
00008  #ifndef W_FUNCTION_COMPONENT_HPP
00009  #define W_FUNCTION_COMPONENT_HPP
00010
00011  #include <functional>
00012  #include <stdexcept>
00013
00014  #include "../srcEntity/wComponent.hpp"
00015
00016  namespace wEngine
00017  {
00018
00031      class FunctionComponent : public Component
00032      {
00033      public:
00039          FunctionComponent( std::function< double( double ) > function );
00040
00041          /*
00042           * @brief Virtual destructor.
00043           */
00044          virtual ~FunctionComponent( ) = default;
00045
00052          [[nodiscard]] double calculate( double x ) const;
00053
00054          /*
00055           * @brief Prints a message confirming that the function is set.
00056           */
00057          void debugPrint( ) const;
00058      private:
00059          std::function< double( double ) > mFunction;
00060      };
00061
00062  }//End of namespace wEngine
00063
00064  #endif

```

8.14 wLengthComponent.cpp File Reference

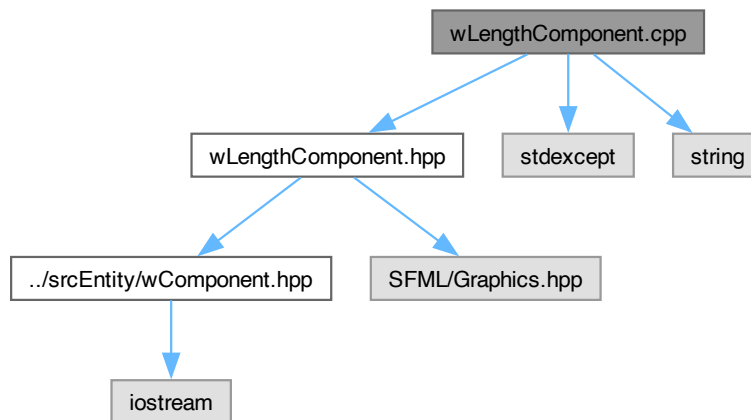
Implementation of the LengthComponent class.

```

#include "wLengthComponent.hpp"
#include <stdexcept>
#include <string>

```

Include dependency graph for wLengthComponent.cpp:



Namespaces

- namespace [wEngine](#)

8.14.1 Detailed Description

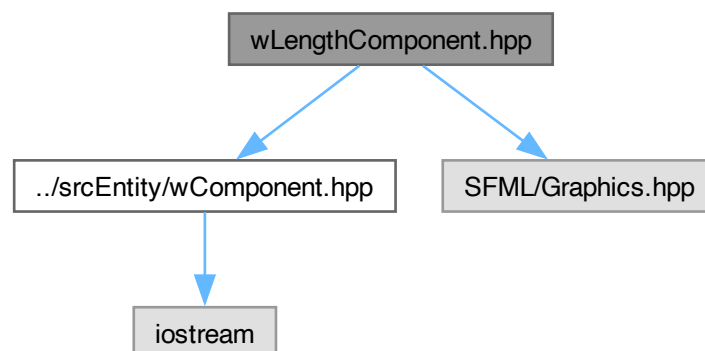
Implementation of the LengthComponent class.

8.15 wLengthComponent.hpp File Reference

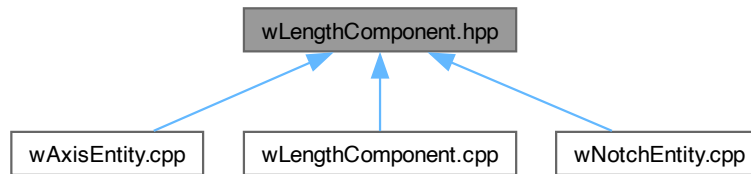
```
#include "../srcEntity/wComponent.hpp"
```

```
#include <SFML/Graphics.hpp>
```

Include dependency graph for wLengthComponent.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [wEngine::LengthComponent](#)
ECS component that defines the length of a drawable object.

Namespaces

- namespace [wEngine](#)

8.16 wLengthComponent.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002
00003  +-----+
00004  Created by Wilfried Koch.
00005  Copyright @ 2025 Wilfried Koch. All rights reserved.
00006  +-----+
00007  */
00008  #ifndef W_LENGTH_COMPONENT_HPP
00009  #define W_LENGTH_COMPONENT_HPP
00010
00011  #include "../srcEntity/wComponent.hpp"
00012
00013  #pragma GCC diagnostic push
00014  #pragma GCC diagnostic ignored "-Wfloat-equal"
00015  #pragma GCC diagnostic ignored "-Wswitch-default"
00016  #include <SFML/Graphics.hpp>
00017  #pragma GCC diagnostic pop
00018
00019  namespace wEngine
00020  {
00021
00022      class LengthComponent : public Component
00023      {
00024      public:
00025          LengthComponent( float length = 2.0f );
00026
00027          /*
00028           * @brief Virtual destructor.
00029           */
00030          virtual ~LengthComponent( ) = default;
00031
00032          [[nodiscard]] float getLength( ) const;
00033
00034          void setLength( float newLength );
00035
00036          void debugPrint( ) const;
00037      private:
  
```

```

00068         float mLength;
00069
00075         void validatePositive( float value ) const;
00076     };
00077
00078 }//End of namespace wEngine
00079
00080 #endif

```

8.17 wLineStyleComponent.cpp File Reference

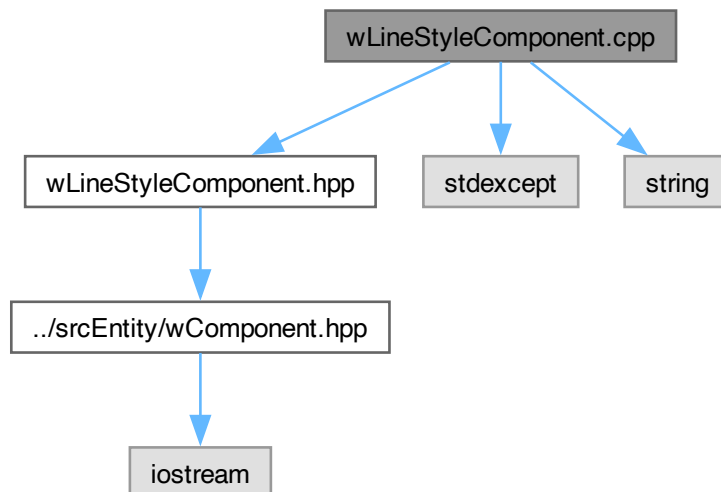
Implementation of the LineStyleComponent class.

```

#include "wLineStyleComponent.hpp"
#include <stdexcept>
#include <string>

```

Include dependency graph for wLineStyleComponent.cpp:



Namespaces

- namespace `wEngine`

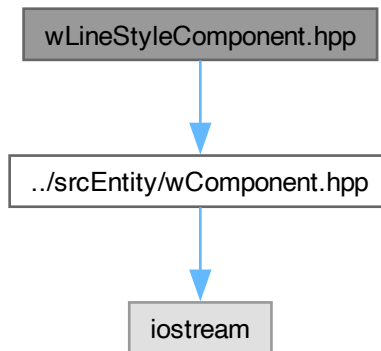
8.17.1 Detailed Description

Implementation of the LineStyleComponent class.

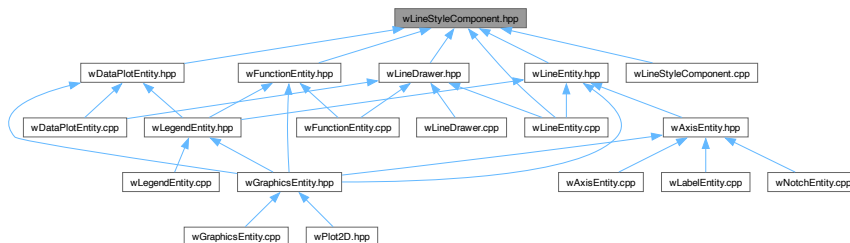
8.18 wLineStyleComponent.hpp File Reference

```
#include "../srcEntity/wComponent.hpp"
```

Include dependency graph for wLineStyleComponent.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [wEngine::LineStyleComponent](#)
ESC component that defines the style of a line (solid, dotted, dashed).

Namespaces

- namespace [wEngine](#)

8.19 wLineStyleComponent.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002  +-----+
00003  Created by Wilfried Koch.
00004  Copyright @ 2025 Wilfried Koch. All rights reserved.
00005  +-----+
00006  */
00007
00008 #ifndef W_LINE_STYLE_COMPONENT_HPP
00009 #define W_LINE_STYLE_COMPONENT_HPP
00010
00011 #include "../srcEntity/wComponent.hpp"
00012
00013 namespace wEngine
00014 {
00015
00027     class LineStyleComponent : public Component
00028     {
00029     public:
00034         enum class LineStyle
00035         {
00036             Solid,
00037             Dotted,
00038             Dashed
00039         };
00040
00045         LineStyleComponent( LineStyle style = LineStyle::Solid );
00046
00047         /*
00048          * @brief Virtual destructor.
00049          */
00050         virtual ~LineStyleComponent( ) = default;
00051
00056         [[nodiscard]] LineStyle getStyle( ) const;
00057
00062         void setStyle( LineStyle style );
00063
00068         [[nodiscard]] float getDashLength( ) const;
00069
00075         void setDashLength( float dashLength );
00076
00081         [[nodiscard]] float getGapLength( ) const;
00082
00088         void setGapLength( float gapLength );
00089
00090         /*
00091          * @brief Outputs the current style and parameters to the console.
00092          */
00093         void debugPrint( ) const;
00094     private:
00095         LineStyle mStyle;
00096         float mDashLength;
00097         float mGapLength;
00098     };
00099
00100 } //End of namespace wEngine
00101
00102 #endif

```

8.20 wNotchIntervalComponent.cpp File Reference

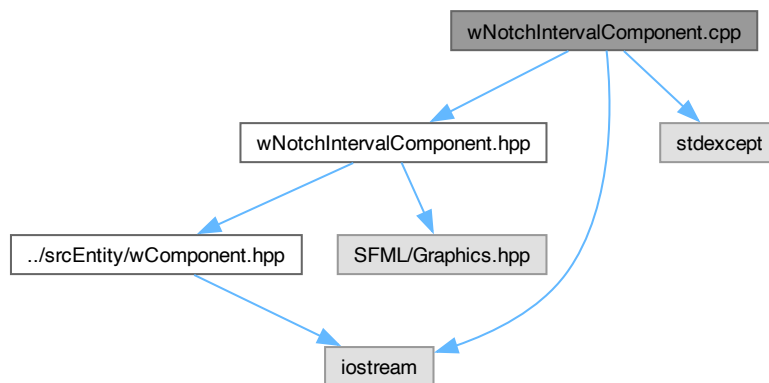
Implementation of the wNotchIntervalComponent class.

```

#include "wNotchIntervalComponent.hpp"
#include <iostream>
#include <stdexcept>

```


Include dependency graph for wNotchIntervalComponent.cpp:



Namespaces

- namespace [wEngine](#)

8.20.1 Detailed Description

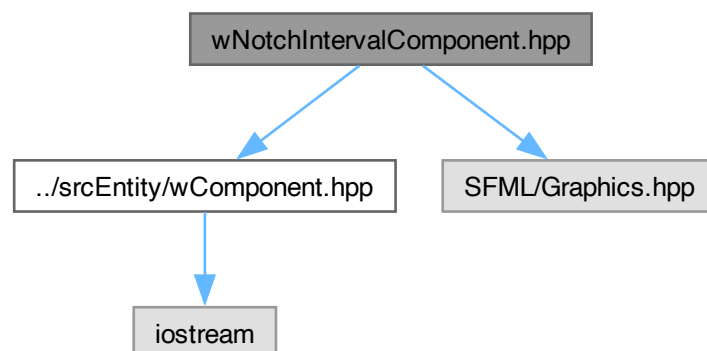
Implementation of the wNotchIntervalComponent class.

8.21 wNotchIntervalComponent.hpp File Reference

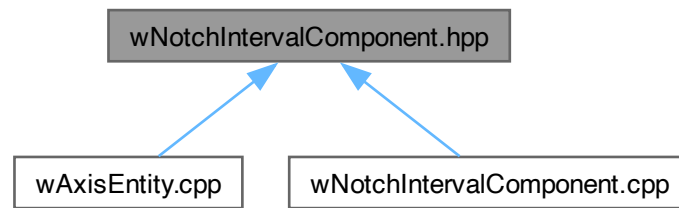
```
#include "../srcEntity/wComponent.hpp"
```

```
#include <SFML/Graphics.hpp>
```

Include dependency graph for wNotchIntervalComponent.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [wEngine::NotchIntervalComponent](#)
ECS component that defines the interval between notches on an axis.

Namespaces

- namespace [wEngine](#)

8.22 wNotchIntervalComponent.hpp

[Go to the documentation of this file.](#)

```

00001 /*
00002
00003 +-----+
00004 Created by Wilfried Koch.
00005 Copyright © 2025 Wilfried Koch. All rights reserved.
00006 +-----+
00007 */
00008 #ifndef W_NOTCH_INTERVAL_COMPONENT_HPP
00009 #define W_NOTCH_INTERVAL_COMPONENT_HPP
00010 #include "../srcEntity/wComponent.hpp"
00011 #pragma GCC diagnostic push
00012 #pragma GCC diagnostic ignored "-Wfloat-equal"
00013 #pragma GCC diagnostic ignored "-Wswitch-default"
00014 #include <SFML/Graphics.hpp>
00015 #pragma GCC diagnostic pop
00016
00017 namespace wEngine
00018 {
00019     class NotchIntervalComponent : public Component
00020     {
00021     public:
00022         NotchIntervalComponent( float interval = 1.0f );
00023
00024         /*
00025          * @brief Virtual destructor.
00026          */
00027         virtual ~NotchIntervalComponent( ) = default;
00028
00029         [[nodiscard]] float getInterval( ) const;
00030
00031         void setInterval( float newInterval );
00032     };
00033 }

```

```

00062
00066         void debugPrint( ) const;
00067
00068     private:
00069         float mInterval;
00070
00076         void validateInterval( float value ) const;
00077     };
00078
00079 }// End of namespace wEngine
00080
00081 #endif

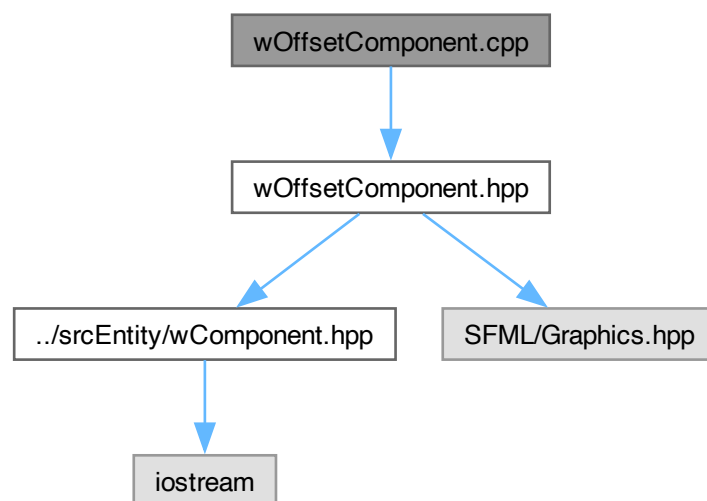
```

8.23 wOffsetComponent.cpp File Reference

Implementation of the OffsetComponent class.

```
#include "wOffsetComponent.hpp"
```

Include dependency graph for wOffsetComponent.cpp:



Namespaces

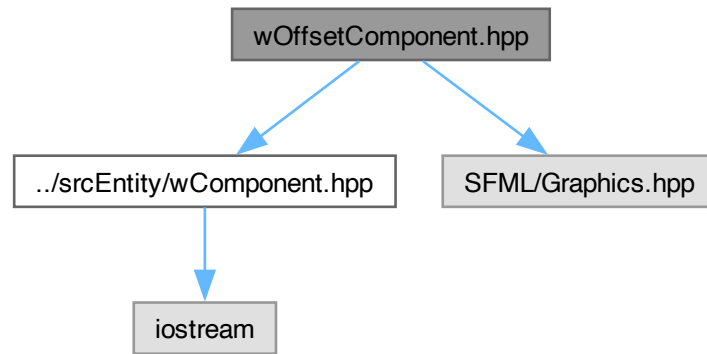
- namespace [wEngine](#)

8.23.1 Detailed Description

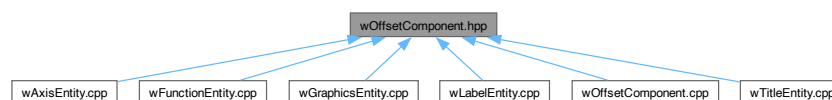
Implementation of the OffsetComponent class.

8.24 wOffsetComponent.hpp File Reference

```
#include "../srcEntity/wComponent.hpp"
#include <SFML/Graphics.hpp>
Include dependency graph for wOffsetComponent.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [wEngine::OffsetComponent](#)
ECS component that defines a logical coordinate offset.

Namespaces

- namespace [wEngine](#)

8.25 wOffsetComponent.hpp

[Go to the documentation of this file.](#)

```
00001 /*
00002
00003 -----
00003 Created by Wilfried Koch.
00004 Copyright @ 2025 Wilfried Koch. All rights reserved.
```

```

00005
00006 +-----+
00006 */
00007
00008 #ifndef W_OFFSET_COMPONENT_HPP
00009 #define W_OFFSET_COMPONENT_HPP
00010
00011 #include "../srcEntity/wComponent.hpp"
00012
00013 #pragma GCC diagnostic push
00014 #pragma GCC diagnostic ignored "-Wfloat-equal"
00015 #pragma GCC diagnostic ignored "-Wswitch-default"
00016 #include <SFML/Graphics.hpp>
00017 #pragma GCC diagnostic pop
00018
00019 namespace wEngine
00020 {
00021
00040     class OffsetComponent : public Component
00041     {
00042     public:
00047         OffsetComponent( sf::Vector2f offset = sf::Vector2f( 0.0f, 0.0f ) );
00048
00049         /*
00050          * @brief Virtual destructor.
00051          */
00052         virtual ~OffsetComponent( ) = default;
00053
00058         [[nodiscard]] sf::Vector2f getOffset( ) const;
00059
00064         void setOffset( sf::Vector2f offset );
00065
00070         void addOffset( sf::Vector2f delta );
00071
00075         void debugPrint( ) const;
00076     private:
00077         sf::Vector2f mOffset;
00078     };
00079
00080 } //End of namespace wEngine
00081
00082 #endif

```

8.26 wPaddingComponent.cpp File Reference

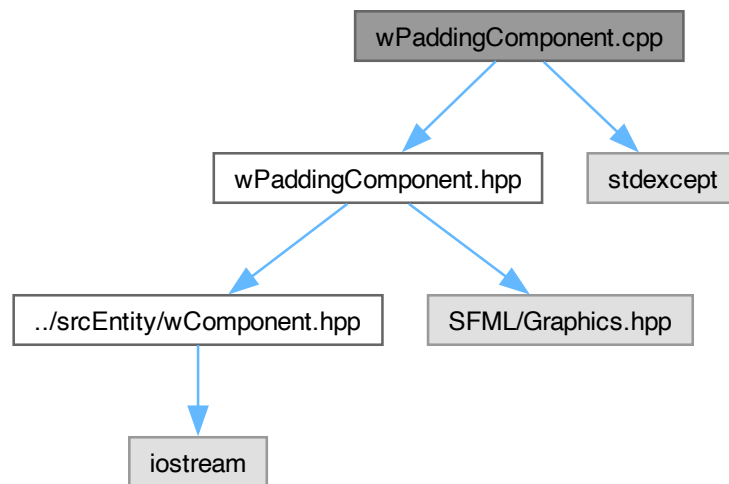
Implementation of the PaddingComponent class.

```

#include "wPaddingComponent.hpp"
#include <stdexcept>

```

Include dependency graph for wPaddingComponent.cpp:



Namespaces

- namespace [wEngine](#)

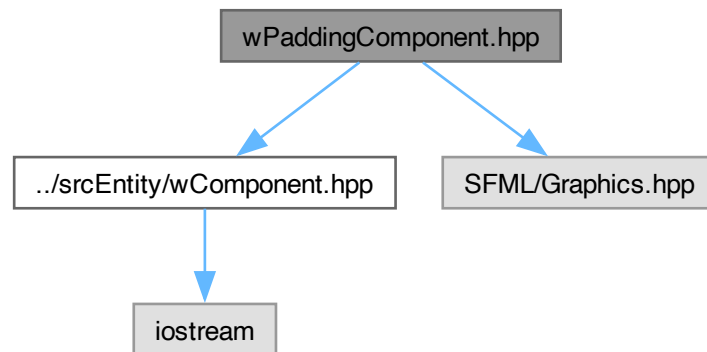
8.26.1 Detailed Description

Implementation of the PaddingComponent class.

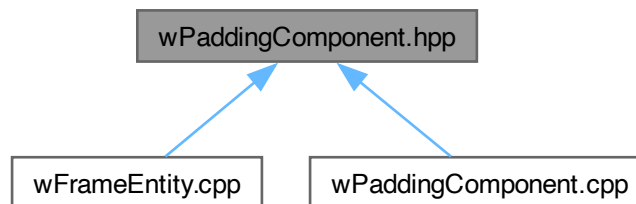
8.27 wPaddingComponent.hpp File Reference

```
#include "../srcEntity/wComponent.hpp"  
#include <SFML/Graphics.hpp>
```

Include dependency graph for wPaddingComponent.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [wEngine::PaddingComponent](#)
ECS component representing internal padding for UI-like elements.

Namespaces

- namespace [wEngine](#)

8.28 wPaddingComponent.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef W_PADDING_COMPONENT_HPP
00002 #define W_PADDING_COMPONENT_HPP

```

```

00003
00004 #include "../srcEntity/wComponent.hpp"
00005
00006 #pragma GCC diagnostic push
00007 #pragma GCC diagnostic ignored "-Wfloat-equal"
00008 #pragma GCC diagnostic ignored "-Wswitch-default"
00009 #include <SFML/Graphics.hpp>
00010 #pragma GCC diagnostic pop
00011
00012 namespace wEngine
00013 {
00014
00031     class PaddingComponent : public Component
00032     {
00033     public:
00038         PaddingComponent( sf::Vector2f padding = sf::Vector2f( 0.0f, 0.0f ) );
00039
00040         /*
00041          * @brief Virtual destructor.
00042          */
00043         virtual ~PaddingComponent( ) = default;
00044
00049         void setPadding( sf::Vector2f padding );
00050
00055         [[nodiscard]] sf::Vector2f getPadding( ) const;
00056
00060         void debugPrint( ) const;
00061     private:
00062         sf::Vector2f mPadding;
00063
00069         void validatePositive( const sf::Vector2f& value ) const;
00070     };
00071
00072 }//End of namespace wEngine
00073
00074 #endif

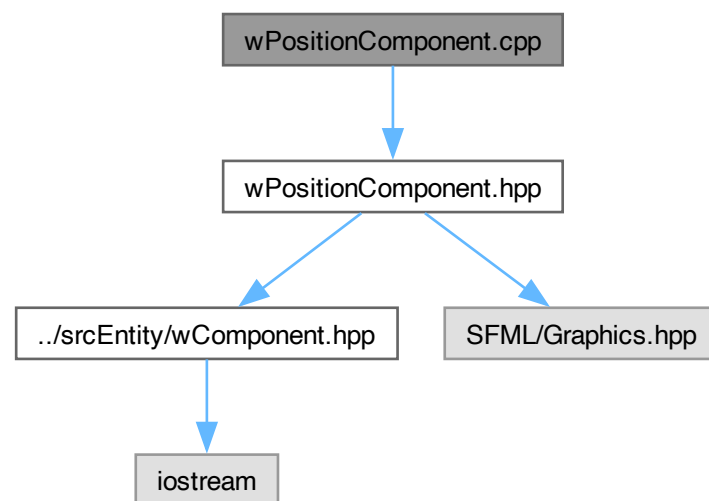
```

8.29 wPositionComponent.cpp File Reference

Implementation of the PositionComponent class.

```
#include "wPositionComponent.hpp"
```

Include dependency graph for wPositionComponent.cpp:



Namespaces

- namespace [wEngine](#)

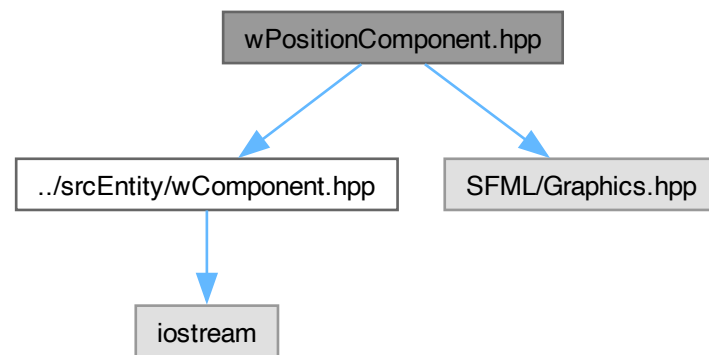
8.29.1 Detailed Description

Implementation of the PositionComponent class.

8.30 wPositionComponent.hpp File Reference

```
#include "../srcEntity/wComponent.hpp"
#include <SFML/Graphics.hpp>
```

Include dependency graph for wPositionComponent.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [wEngine::PositionComponent](#)
ECS component storing the position of an entity in 2D space and supports movement tracking.

Namespaces

- namespace [wEngine](#)

8.31 wPositionComponent.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002
00003  +-----+
00004  Created by Wilfried Koch.
00005  Copyright © 2025 Wilfried Koch. All rights reserved.
00006  +-----+
00007  */
00008  #ifndef W_POSITION_COMPONENT_HPP
00009  #define W_POSITION_COMPONENT_HPP
00010
00011  #include "../srcEntity/wComponent.hpp"
00012
00013  #pragma GCC diagnostic push
00014  #pragma GCC diagnostic ignored "-Wfloat-equal"
00015  #pragma GCC diagnostic ignored "-Wswitch-default"
00016  #include <SFML/Graphics.hpp>
00017  #pragma GCC diagnostic pop
00018
00019  namespace wEngine
00020  {
00021
00043      class PositionComponent : public Component
00044      {
00045      public:
00050          PositionComponent( sf::Vector2f position = sf::Vector2f( 0.0f, 0.0f ) );
00051
00052          /*
00053           * @brief Virtual destructor.
00054           */
00055          virtual ~PositionComponent( ) = default;
00056
00061          [[nodiscard]] sf::Vector2f getPosition( ) const;
00062
00067          [[nodiscard]] sf::Vector2f getLastPosition( ) const;
00068
00073          void setPosition( sf::Vector2f newPosition );
00074
00081          void move( const sf::Vector2f& offset );
00082
00086          void debugPrint( ) const;
00087      private:
00088          sf::Vector2f mPosition;
00089          sf::Vector2f mLastPosition;
00090      };
00091
00092  } //End of namespace wEngine
00093
00094  #endif

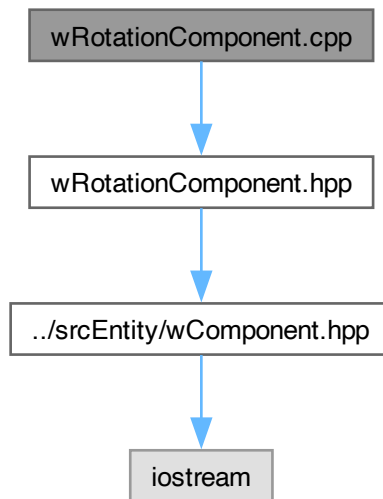
```

8.32 wRotationComponent.cpp File Reference

Implementation of the RotationComponent class.

```
#include "wRotationComponent.hpp"
```

Include dependency graph for wRotationComponent.cpp:



Namespaces

- namespace [wEngine](#)

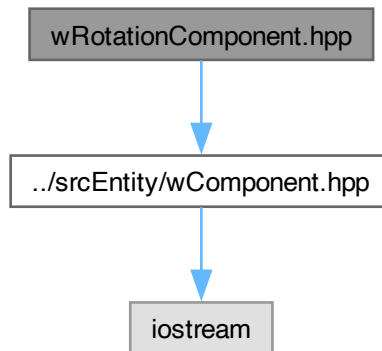
8.32.1 Detailed Description

Implementation of the `RotationComponent` class.

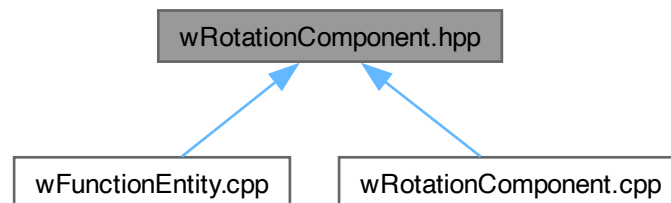
8.33 wRotationComponent.hpp File Reference

```
#include "../srcEntity/wComponent.hpp"
```

Include dependency graph for wRotationComponent.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [wEngine::RotationComponent](#)
ECS component that stores a rotation angle (in degrees).

Namespaces

- namespace [wEngine](#)

8.34 wRotationComponent.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002
00003  +-----+
00003  Created by Wilfried Koch.
00004  Copyright © 2025 Wilfried Koch. All rights reserved.
00005  +-----+
00006  */
00007
00008  #ifndef W_ROTATION_COMPONENT_HPP
00009  #define W_ROTATION_COMPONENT_HPP
00010
00011  #include "../srcEntity/wComponent.hpp"
00012
00013  namespace wEngine
00014  {
00015
00033      class RotationComponent : public Component
00034      {
00035      public:
00040          RotationComponent( float angle = 0.0f );
00041
00042          /*
00043           * @brief Virtual destructor.
00044           */
00045          virtual ~RotationComponent( ) = default;
00046
00051          void setAngle( float angle );
00052
00057          [[nodiscard]] float getAngle( ) const;
00058
00062          void debugPrint( ) const;
00063
00064      private:
00065          float mAngle;
00066      };
00067
00068  }//End of namespace wEngine
00069
00070  #endif

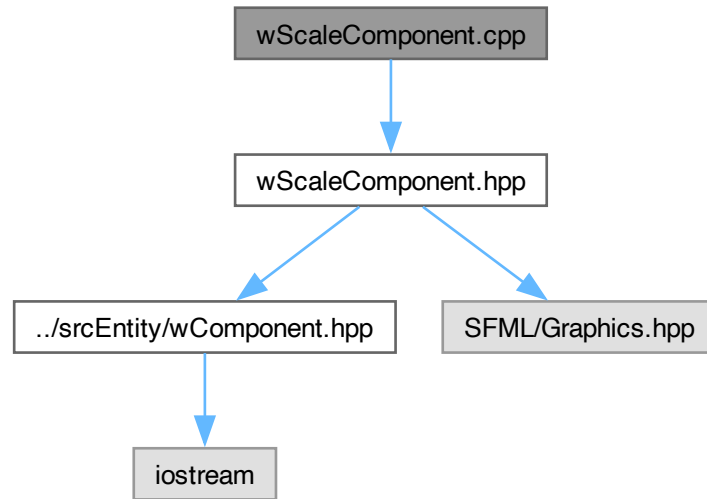
```

8.35 wScaleComponent.cpp File Reference

Implementation of the ScaleComponent class.

```
#include "wScaleComponent.hpp"
```

Include dependency graph for wScaleComponent.cpp:



Namespaces

- namespace [wEngine](#)

8.35.1 Detailed Description

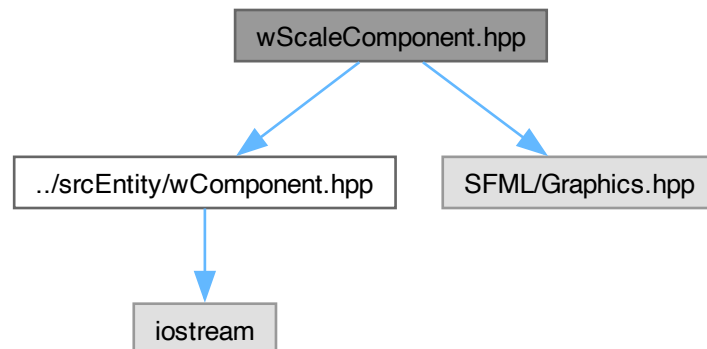
Implementation of the `ScaleComponent` class.

8.36 wScaleComponent.hpp File Reference

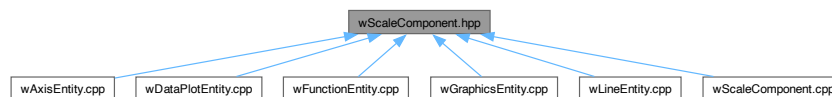
```
#include "../srcEntity/wComponent.hpp"
```

```
#include <SFML/Graphics.hpp>
```

Include dependency graph for wScaleComponent.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [wEngine::ScaleComponent](#)
ECS component that defines the scaling factor for an entity in 2D space.

Namespaces

- namespace [wEngine](#)

8.37 wScaleComponent.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002
00003  +-----+
00003  Created by Wilfried Koch.
00004  Copyright © 2025 Wilfried Koch. All rights reserved.
00005  +-----+
00006  */
00007
00008  #ifndef W_SCALE_COMPONENT_HPP
00009  #define W_SCALE_COMPONENT_HPP
00010
00011  #include "../srcEntity/wComponent.hpp"
  
```

```

00012
00013 #pragma GCC diagnostic push
00014 #pragma GCC diagnostic ignored "-Wfloat-equal"
00015 #pragma GCC diagnostic ignored "-Wswitch-default"
00016 #include <SFML/Graphics.hpp>
00017 #pragma GCC diagnostic pop
00018
00019 namespace wEngine
00020 {
00021
00022     class ScaleComponent : public Component
00023     {
00024     public:
00025         ScaleComponent( sf::Vector2f scale = { 1.0f, 1.0f } );
00026
00027         /*
00028          * @brief Virtual destructor.
00029          */
00030         virtual ~ScaleComponent( ) = default;
00031
00032         [[nodiscard]] sf::Vector2f getScale( ) const;
00033
00034         void setScale( sf::Vector2f newScale );
00035
00036         void debugPrint( ) const;
00037     private:
00038         sf::Vector2f mScale;
00039
00040         void validatePositive( const sf::Vector2f& value ) const;
00041     };
00042 } //End of namespace wEngine
00043 #endif

```

8.38 wThicknessComponent.cpp File Reference

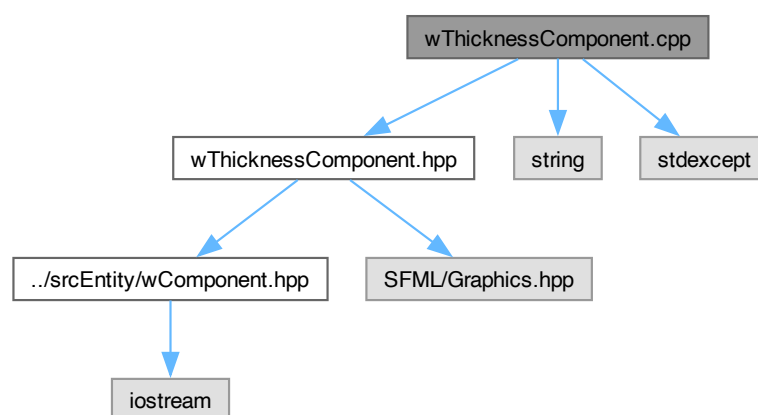
Implementation of the ThicknessComponent class.

```

#include "wThicknessComponent.hpp"
#include <string>
#include <stdexcept>

```

Include dependency graph for wThicknessComponent.cpp:



Namespaces

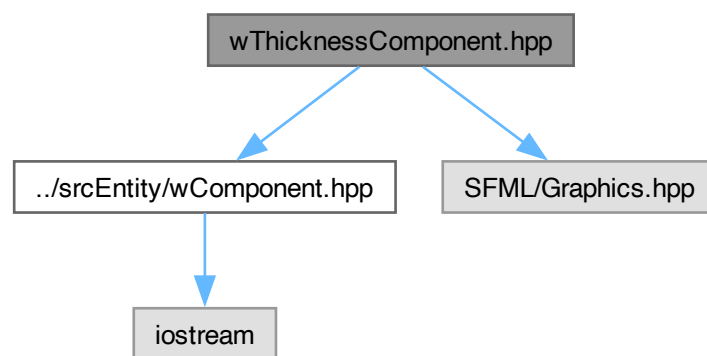
- namespace `wEngine`

8.38.1 Detailed Description

Implementation of the ThicknessComponent class.

8.39 wThicknessComponent.hpp File Reference

```
#include "../srcEntity/wComponent.hpp"
#include <SFML/Graphics.hpp>
Include dependency graph for wThicknessComponent.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [wEngine::ThicknessComponent](#)
ECS component that defines the thickness of a drawable object.

Namespaces

- namespace [wEngine](#)

8.40 wThicknessComponent.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002  +-----+
00003  Created by Wilfried Koch.
00004  Copyright © 2025 Wilfried Koch. All rights reserved.
00005  +-----+
00006  */
00007
00008 #ifndef W_THICKNESS_COMPONENT_HPP
00009 #define W_THICKNESS_COMPONENT_HPP
00010
00011 #include "../srcEntity/wComponent.hpp"
00012
00013 #pragma GCC diagnostic push
00014 #pragma GCC diagnostic ignored "-Wfloat-equal"
00015 #pragma GCC diagnostic ignored "-Wswitch-default"
00016 #include <SFML/Graphics.hpp>
00017 #pragma GCC diagnostic pop
00018
00019 namespace wEngine
00020 {
00021
00043     class ThicknessComponent : public Component
00044     {
00045     public:
00051         ThicknessComponent( float thickness = 2.0f );
00052
00053         /*
00054          * @brief Virtual destructor.
00055          */
00056         virtual ~ThicknessComponent( ) = default;
00057
00062         [[nodiscard]] float getThickness( ) const;
00063
00069         void setThickness( float newThickness );
00070
00074         void debugPrint( ) const;
00075     private:
00076         float mThickness;
00077
00078         /*
00079          * @brief Validates that the thickness value is strictly positive.
00080          * @param value The thickness value to validate.
00081          * @throws std::invalid_argument if value <= 0.
00082          */
00083         void validatePositive( float value ) const;
00084     };
00085
00086 } //End of namespace wEngine
00087
00088 #endif

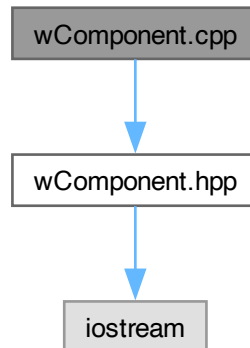
```

8.41 wComponent.cpp File Reference

Implementation of the Component class.

```
#include "wComponent.hpp"
```

Include dependency graph for wComponent.cpp:



Namespaces

- namespace [wEngine](#)

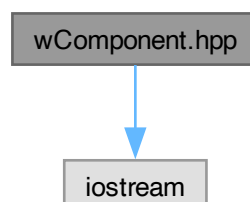
8.41.1 Detailed Description

Implementation of the Component class.

8.42 wComponent.hpp File Reference

```
#include <iostream>
```

Include dependency graph for wComponent.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class `wEngine::Component`
Abstract base class for all ECS components.

Namespaces

- namespace `wEngine`

8.43 wComponent.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002
00003  +-----+
00004  Created by Wilfried Koch.
00005  Copyright @ 2025 Wilfried Koch. All rights reserved.
00006  +-----+
00007  */
00008  #ifndef W_COMPONENT_HPP
00009  #define W_COMPONENT_HPP
00010
00011  #include <iostream>
00012
00013  namespace wEngine
00014  {
00015
00016      class Entity;
00017
00031      class Component
00032      {
00033      public:
00034          /*
00035           * @brief Virtual destructor.
00036           */
00037          virtual ~Component( ) = default;
00038
00039          /*
00040           * @brief Enables the component (makes it active).
00041           */
00042          virtual void enable( );
00043
00044          /*
00045           * @brief Disables the component (makes it inactive).
00046           */
00047          virtual void disable( );
00048
00053          [[nodiscard]] bool isEnabled( ) const;
00054
00059          void setParent( Entity* parent );
00060
00065          [[nodiscard]] Entity* getParent( ) const;
00066
00067      protected:
00072          Component( );
00073
00074      private:

```

```

00075         bool mEnabled;
00076         Entity* mParent;
00077     };
00078
00079 } // End of namespace wEngine
00080
00081 #endif

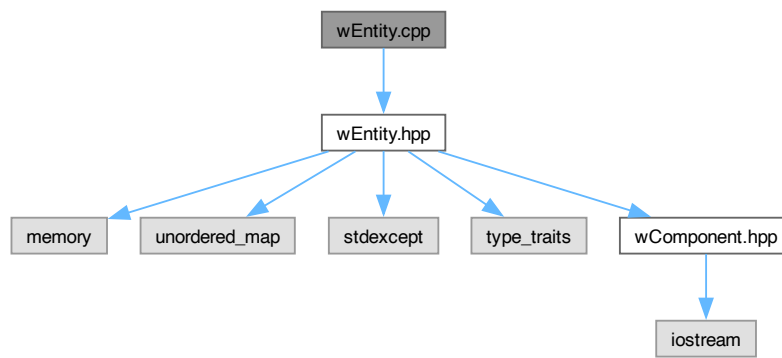
```

8.44 wEntity.cpp File Reference

Implementation of the Entity class.

```
#include "wEntity.hpp"
```

Include dependency graph for wEntity.cpp:



Namespaces

- namespace [wEngine](#)

8.44.1 Detailed Description

Implementation of the Entity class.

8.45 wEntity.hpp File Reference

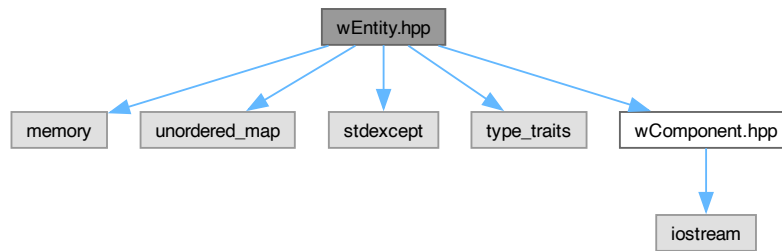
```

#include <memory>
#include <unordered_map>
#include <stdexcept>
#include <type_traits>

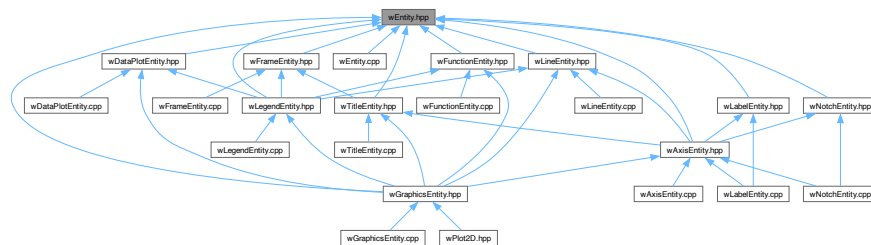
```

```
#include "wComponent.hpp"
```

Include dependency graph for wEntity.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [wEngine::Entity](#)
Represents an entity in the ECS (Entity-Component System) architecture.

Namespaces

- namespace [wEngine](#)

Functions

- `std::size_t` [wEngine::getNextComponentTypeID \(\)](#)
- template<typename ComponentType>
`std::size_t` [wEngine::getComponentTypeID \(\)](#) noexcept

8.46 wEntity.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002
00003  +-----+
00004  Created by Wilfried Koch.
00005  Copyright © 2025 Wilfried Koch. All rights reserved.
00006  +-----+
00007  */
00008  #ifndef W_ENTITY_HPP
00009  #define W_ENTITY_HPP
00010
00011  #include <memory>
00012  #include <unordered_map>
00013  #include <stdexcept>
00014  #include <type_traits>
00015
00016  #include "wComponent.hpp"
00017
00018  namespace wEngine
00019  {
00020
00021      /*
00022       * @brief Generates a new unique component type ID.
00023       * @return A unique integer for component type identification.
00024       */
00025      inline std::size_t getNextComponentTypeID( )
00026      {
00027          static std::size_t componentTypeCounter = 0;
00028          return componentTypeCounter++;
00029      }
00030
00031      /*
00032       * @brief Returns the unique type ID associated with a specific component type.
00033       * @tparam ComponentType The type of the component.
00034       * @return A unique integer identifying the component type.
00035       */
00036      template< typename ComponentType >
00037      std::size_t getComponentTypeID( ) noexcept
00038      {
00039          static std::size_t componentTypeID = getNextComponentTypeID( );
00040          return componentTypeID;
00041      }
00042
00043      class Entity
00044      {
00045      public:
00046          /*
00047           * @brief Default constructor. Generates a new entity with a unique ID.
00048           */
00049          Entity( );
00050
00051          /*
00052           * @brief Destructor. Disables all attached components before destruction.
00053           */
00054          virtual ~Entity( );
00055
00056          [[nodiscard]] unsigned int getEntityID( ) const;
00057
00058          void clearComponents( );
00059
00060          static void resetEntityIDCounter( );
00061
00062          template< typename T, typename... Args >
00063          std::shared_ptr< T > addComponent( Args&&... args )
00064          {
00065              static_assert( std::is_base_of< Component, T >::value, "T must be derived from
Component" );
00066
00067              auto typeID = getComponentTypeID< T >( );
00068              if ( mComponents.contains( typeID ) )
00069              {
00070                  std::cerr << "Warning: Component " << typeid( T ).name( )
00071                      << " already exists in entity " << mEntityID << std::endl;
00072                  throw std::runtime_error( "Component already exists in entity" );
00073              }
00074
00075              auto component = std::make_shared< T >( std::forward< Args >( args )... );
00076              component->setParent( this );
00077              mComponents[ typeID ] = component;
00078
00079              return component;
00080          }
00081      };
00082
00083  }
00084
00085  #endif

```

```

00121     }
00122
00130     template< typename T >
00131     void removeComponent( )
00132     {
00133         static_assert( std::is_base_of< Component, T >::value, "T must be derived from
Component" );
00134
00135         auto typeID = getComponentTypeID< T >( );
00136         mComponents.erase( typeID );
00137     }
00138
00145     template< typename T >
00146     [[nodiscard]] bool hasComponent( ) const noexcept
00147     {
00148         static_assert( std::is_base_of< Component, T >::value, "T must be derived from
Component" );
00149
00150         return mComponents.contains( getComponentTypeID< T >( ) );
00151     }
00152
00159     template< typename T >
00160     [[nodiscard]] std::shared_ptr< T > getComponent( ) const
00161     {
00162         static_assert( std::is_base_of< Component, T >::value, "T must be derived from
Component" );
00163
00164         auto typeID = getComponentTypeID< T >( );
00165         auto it = mComponents.find( typeID );
00166         if ( it != mComponents.end( ) )
00167         {
00168             return std::dynamic_pointer_cast< T >( it->second );
00169         }
00170
00171         return nullptr;
00172     }
00173
00186     template< typename T >
00187     [[nodiscard]] std::shared_ptr< T > requireComponent( const std::string& context = "" )
const
00188     {
00189         static_assert( std::is_base_of< Component, T >::value, "T must be derived from
Component" );
00190
00191         if ( !hasComponent< T >( ) )
00192         {
00193             std::string msg = "Missing required component: ";
00194             msg += typeid( T ).name( );
00195             if ( !context.empty( ) )
00196             {
00197                 msg += " in context: " + context;
00198             }
00199             throw std::runtime_error( msg );
00200         }
00201
00202         return getComponent< T >( );
00203     }
00204
00214     template< typename Interface >
00215     [[nodiscard]] std::shared_ptr< Interface > getInterfaceComponent( ) const
00216     {
00217         for ( const auto& [ typeID, component ] : mComponents )
00218         {
00219             auto interfaceComponent = std::dynamic_pointer_cast< Interface >( component );
00220             if ( interfaceComponent )
00221             {
00222                 return interfaceComponent;
00223             }
00224         }
00225         return nullptr;
00226     }
00227
00228     private:
00229         unsigned int mEntityID;
00230         std::unordered_map< std::size_t, std::shared_ptr< Component > > mComponents;
00231
00232         static unsigned int sEntityIDCounter;
00233         static unsigned int generateNextEntityID( );
00234 };
00235
00236 } // End of namespace wEngine
00237
00238 #endif

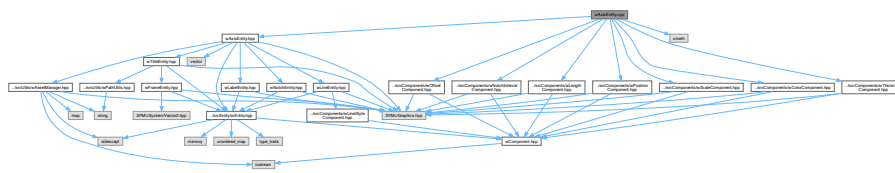
```


8.47 wAxisEntity.cpp File Reference

Implementation of the AxisEntity class.

```
#include "wAxisEntity.hpp"
#include "../srcComponents/wPositionComponent.hpp"
#include "../srcComponents/wScaleComponent.hpp"
#include "../srcComponents/wColorComponent.hpp"
#include "../srcComponents/wThicknessComponent.hpp"
#include "../srcComponents/wOffsetComponent.hpp"
#include "../srcComponents/wNotchIntervalComponent.hpp"
#include "../srcComponents/wLengthComponent.hpp"
#include <cmath>
```

Include dependency graph for wAxisEntity.cpp:



Namespaces

- namespace [wPlot2D](#)

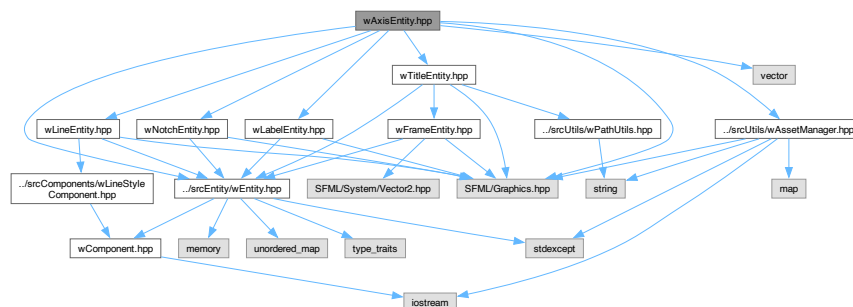
8.47.1 Detailed Description

Implementation of the AxisEntity class.

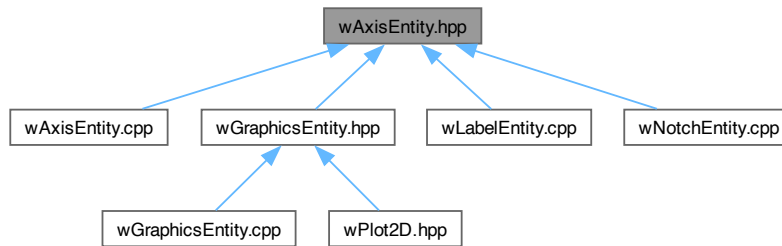
8.48 wAxisEntity.hpp File Reference

```
#include "../srcEntity/wEntity.hpp"
#include "../srcUtils/wAssetManager.hpp"
#include "wLineEntity.hpp"
#include "wNotchEntity.hpp"
#include "wTitleEntity.hpp"
#include "wLabelEntity.hpp"
#include <SFML/Graphics.hpp>
#include <vector>
```

Include dependency graph for wAxisEntity.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [wPlot2D::AxisEntity](#)
Represents a visual axis (X or Y) in a 2D plot with optional notches and title.

Namespaces

- namespace [wPlot2D](#)

Enumerations

- enum class [wPlot2D::AxisType](#) { [wPlot2D::X_AXIS](#) , [wPlot2D::Y_AXIS](#) }
Enum representing the type of axis to render.
- enum class [wPlot2D::NotchPosition](#) { [wPlot2D::Center](#) , [wPlot2D::Above](#) , [wPlot2D::Below](#) }
Enum controlling the visual placement of notches relative to the axis.

8.49 wAxisEntity.hpp

[Go to the documentation of this file.](#)

```

00001 /*
00002
00003 -----
00004 Created by Wilfried Koch.
00005 Copyright © 2025 Wilfried Koch. All rights reserved.
00006 -----
00007 */
00008 #ifndef W_AXIS_ENTITY_HPP
00009 #define W_AXIS_ENTITY_HPP
00010
00011 #include "../srcEntity/wEntity.hpp"
00012 #include "../srcUtils/wAssetManager.hpp"
00013
00014 #include "wLineEntity.hpp"
00015 #include "wNotchEntity.hpp"
00016 #include "wTitleEntity.hpp"
00017 #include "wLabelEntity.hpp"
00018
00019 #pragma GCC diagnostic push
00020 #pragma GCC diagnostic ignored "-Wfloat-equal"
00021 #pragma GCC diagnostic ignored "-Wswitch-default"
00022 #include <SFML/Graphics.hpp>

```

```

00023 #pragma GCC diagnostic pop
00024
00025 #include <vector>
00026
00027 namespace wPlot2D
00028 {
00029
00030     enum class AxisType
00031     {
00032         X_AXIS,
00033         Y_AXIS
00034     };
00035
00036     enum class NotchPosition
00037     {
00038         Center,
00039         Above,
00040         Below
00041     };
00042
00043     class AxisEntity : public wEngine::Entity
00044     {
00045     public:
00046         AxisEntity( sf::Font& font, sf::Vector2f origin, sf::Vector2f scale, sf::Vector2f offset,
00047                     AxisType type, sf::Vector2f axisRange );
00048
00049         virtual ~AxisEntity( ) = default;
00050
00051         void setColor( sf::Color color );
00052
00053         void setThickness( float thickness );
00054
00055         void setArrowSize( float arrowSize );
00056
00057         void addTitle( const std::string& title );
00058
00059         void addTitle( const std::wstring& title );
00060
00061         void setTitleFont( const sf::Font& font );
00062
00063         void setTitleCharacterSize( unsigned int size );
00064
00065         void setTitleColor( sf::Color newColor );
00066
00067         void setTitleOffset( sf::Vector2f titleOffset );
00068
00069         [[nodiscard]] sf::Vector2f getTitleOffset( ) const;
00070
00071         void addNotches( float interval, NotchPosition position, bool hasLabels = false );
00072
00073         void setNotchesColor( const sf::Color& color );
00074
00075         void setNotchesThickness( float thickness );
00076
00077         void setNotchesLength( float newLength );
00078
00079         void setLabelsFont( const sf::Font& font );
00080
00081         void setLabelsColor( const sf::Color& color );
00082
00083         [[nodiscard]] std::vector< sf::Vector2f > getLabelsOffset( ) const;
00084
00085         void setLabelsOffset( sf::Vector2f offset );
00086
00087         void addLabelsOffset( sf::Vector2f delta );
00088
00089         void setLabelsCharacterSize( unsigned int newSize );
00090
00091         void setLabelsDecimalPlaces( int places );
00092
00093         void setCustomLabels( const std::vector< std::string >& labels );
00094
00095         void render( sf::RenderWindow& window );
00096     private:
00097         sf::Font& mTitleFont;
00098         sf::Font& mLabelsFont;
00099         AxisType mAxisType;
00100         sf::Vector2f mAxisRange;
00101         std::unique_ptr< LineEntity > mAxisLine;
00102         float mArrowSize;
00103
00104         std::vector< std::unique_ptr< NotchEntity > > mNotches;
00105         NotchPosition mNotchPosition;
00106
00107         std::unique_ptr< TitleEntity > mTitle;
00108
00109         bool mHasLabels;

```

```

00241         std::vector< std::unique_ptr< LabelEntity > > mLabels;
00242
00246         void construct( );
00247
00251         template < typename T >
00252         void initTitle( const T& title );
00253
00257         void initNotches( );
00258     };
00259
00260 }//End of namespace wPlot2D
00261
00262 #endif

```

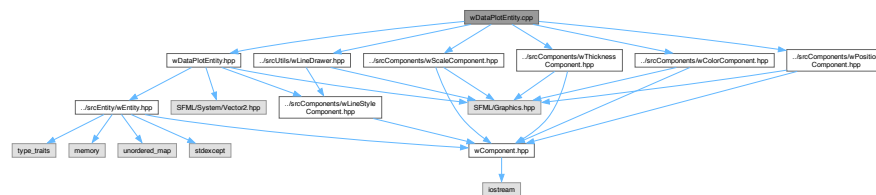
8.50 wDataPlotEntity.cpp File Reference

Implementation of the DataPlotEntity class.

```

#include "wDataPlotEntity.hpp"
#include "../srcUtils/wLineDrawer.hpp"
#include "../srcComponents/wColorComponent.hpp"
#include "../srcComponents/wPositionComponent.hpp"
#include "../srcComponents/wScaleComponent.hpp"
#include "../srcComponents/wThicknessComponent.hpp"
Include dependency graph for wDataPlotEntity.cpp:

```



Namespaces

- namespace [wPlot2D](#)

8.50.1 Detailed Description

Implementation of the DataPlotEntity class.

8.51 wDataPlotEntity.hpp File Reference

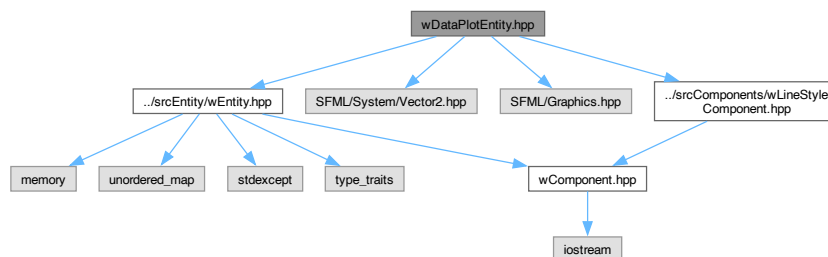
```

#include "../srcEntity/wEntity.hpp"
#include <SFML/System/Vector2.hpp>
#include <SFML/Graphics.hpp>

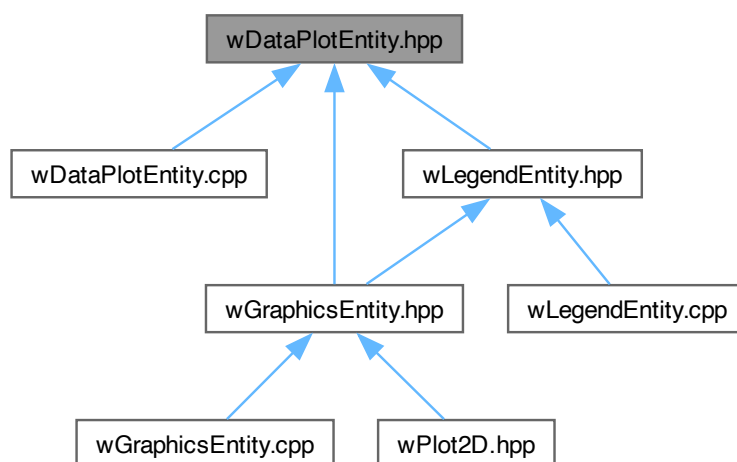
```

```
#include "../srcComponents/wLineStyleComponent.hpp"
```

Include dependency graph for wDataPlotEntity.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [wPlot2D::DataPlotEntity](#)
Entity for plotting raw data points as a connected polyline.

Namespaces

- namespace [wPlot2D](#)

8.52 wDataPlotEntity.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002
00003  +-----+
00004  Created by Wilfried Koch.
00005  Copyright © 2025 Wilfried Koch. All rights reserved.
00006  +-----+
00007  */
00008  #ifndef W_DATA_PLOT_ENTITY_HPP
00009  #define W_DATA_PLOT_ENTITY_HPP
00010
00011  #include "../srcEntity/wEntity.hpp"
00012
00013  #pragma GCC diagnostic push
00014  #pragma GCC diagnostic ignored "-Wfloat-equal"
00015  #pragma GCC diagnostic ignored "-Wswitch-default"
00016  #include <SFML/System/Vector2.hpp>
00017  #include <SFML/Graphics.hpp>
00018  #pragma GCC diagnostic pop
00019
00020  #include "../srcComponents/wLineStyleComponent.hpp"
00021
00022  namespace wPlot2D
00023  {
00024
00041      class DataPlotEntity : public wEngine::Entity
00042      {
00043      public:
00051          DataPlotEntity( const sf::Vector2f origin, const sf::Vector2f scale, const std::vector<
sf::Vector2f >& dataPoints );
00052
00056          virtual ~DataPlotEntity( ) = default;
00057
00062          [[nodiscard]] sf::Color getColor( ) const;
00063
00064
00069          [[nodiscard]] float getThickness( );
00070
00075          [[nodiscard]] wEngine::LineStyleComponent::LineStyle getLineStyle( );
00076
00077
00082          [[nodiscard]] float getDashLength( );
00083
00088          [[nodiscard]] float getGapLength( );
00089
00094          void setColor( sf::Color color );
00095
00100          void setThickness( float thickness );
00101
00106          void setLineStyle( wEngine::LineStyleComponent::LineStyle style );
00107
00112          void setDashLength( float dashLength );
00113
00118          void setGapLength( float gapLength );
00119
00128          void drawDataPlot( sf::RenderWindow &window );
00129
00130      private:
00131          std::vector< sf::Vector2f > mDataPoints;
00132      };
00133
00134  }//End of namespace wPlot2D
00135
00136  #endif

```

8.53 wFrameEntity.cpp File Reference

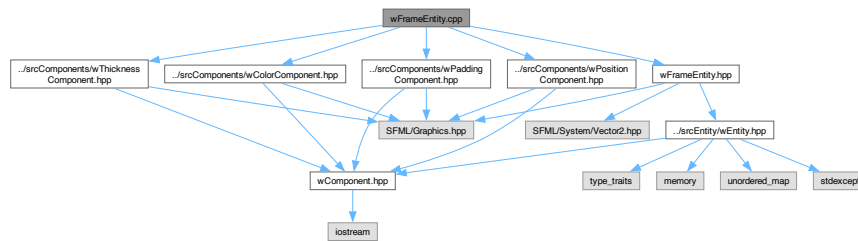
Implementation of the FrameEntity class.

```

#include "wFrameEntity.hpp"
#include "../srcComponents/wPositionComponent.hpp"
#include "../srcComponents/wThicknessComponent.hpp"

```

```
#include "../srcComponents/wColorComponent.hpp"
#include "../srcComponents/wPaddingComponent.hpp"
Include dependency graph for wFrameEntity.cpp:
```



Namespaces

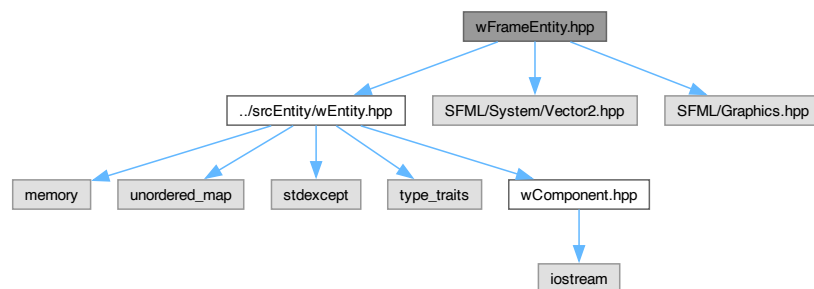
- namespace [wPlot2D](#)

8.53.1 Detailed Description

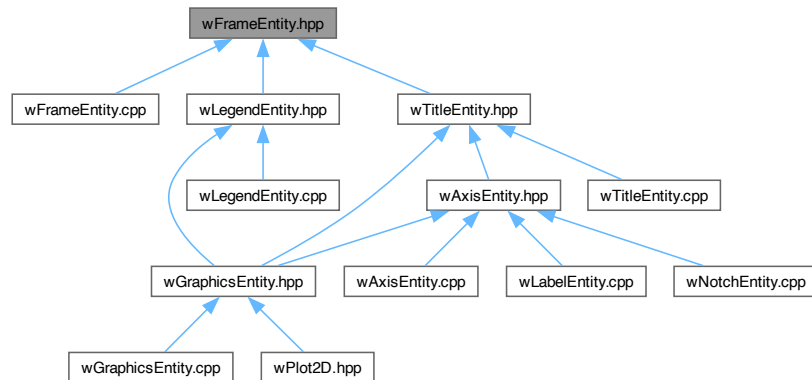
Implementation of the FrameEntity class.

8.54 wFrameEntity.hpp File Reference

```
#include "../srcEntity/wEntity.hpp"
#include <SFML/System/Vector2.hpp>
#include <SFML/Graphics.hpp>
Include dependency graph for wFrameEntity.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [wPlot2D::FrameEntity](#)
Entity representing a rectangular frame around content.

Namespaces

- namespace [wPlot2D](#)

8.55 wFrameEntity.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002  +-----+
00003  Created by Wilfried Koch.
00004  Copyright © 2025 Wilfried Koch. All rights reserved.
00005  +-----+
00006  */
00007
00008 #ifndef W_FRAME_ENTITY_HPP
00009 #define W_FRAME_ENTITY_HPP
00010
00011 #include "../srcEntity/wEntity.hpp"
00012
00013 #pragma GCC diagnostic push
00014 #pragma GCC diagnostic ignored "-Wfloat-equal"
00015 #pragma GCC diagnostic ignored "-Wswitch-default"
00016 #include <SFML/System/Vector2.hpp>
00017 #include <SFML/Graphics.hpp>
00018 #pragma GCC diagnostic pop
00019
00020 namespace wPlot2D
00021 {
00022
00023     class FrameEntity : public wEngine::Entity
00024     {
00025     public:
00026         FrameEntity( bool enabled = true );
00027
00028         virtual ~FrameEntity( ) = default;
00029
00030     };
00031
00032 }
00033
00034 #endif

```



```

00060         void setEnabled( bool enabled );
00061
00066         [[nodiscard]] bool isEnabled( ) const;
00067
00072         [[nodiscard]] sf::Color getFillColor( ) const;
00073
00078         [[nodiscard]] sf::Color getOutlineColor( ) const;
00079
00084         [[nodiscard]] float getThickness( ) const;
00085
00090         [[nodiscard]] sf::Vector2f getPadding( ) const;
00091
00096         void setFillColor( const sf::Color& color );
00097
00102         void setOutlineColor( const sf::Color& color );
00103
00108         void setThickness( float thickness );
00109
00114         void setPadding( const sf::Vector2f& padding );
00115
00121         void update( const sf::FloatRect& contentBounds, const sf::Vector2f& position );
00122
00127         void render( sf::RenderWindow& window );
00128     private:
00129         bool mEnabled;
00130         sf::RectangleShape mFrame;
00131     };
00132
00133 } //End of namespace wPlot2D
00134
00135 #endif

```

8.56 wFunctionEntity.cpp File Reference

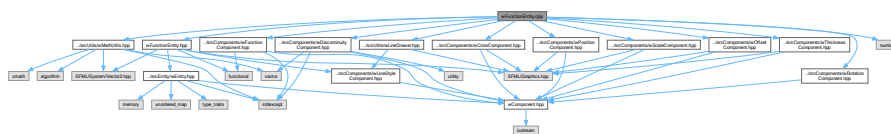
Implementation of the FunctionEntity class.

```

#include "wFunctionEntity.hpp"
#include "../srcUtils/wLineDrawer.hpp"
#include "../srcUtils/wMathUtils.hpp"
#include "../srcComponents/wColorComponent.hpp"
#include "../srcComponents/wPositionComponent.hpp"
#include "../srcComponents/wScaleComponent.hpp"
#include "../srcComponents/wOffsetComponent.hpp"
#include "../srcComponents/wThicknessComponent.hpp"
#include "../srcComponents/wFunctionComponent.hpp"
#include "../srcComponents/wDiscontinuityComponent.hpp"
#include "../srcComponents/wRotationComponent.hpp"
#include <numbers>

```

Include dependency graph for wFunctionEntity.cpp:



Namespaces

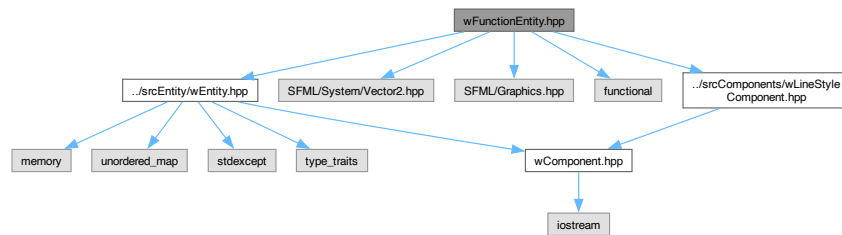
- namespace [wPlot2D](#)

8.56.1 Detailed Description

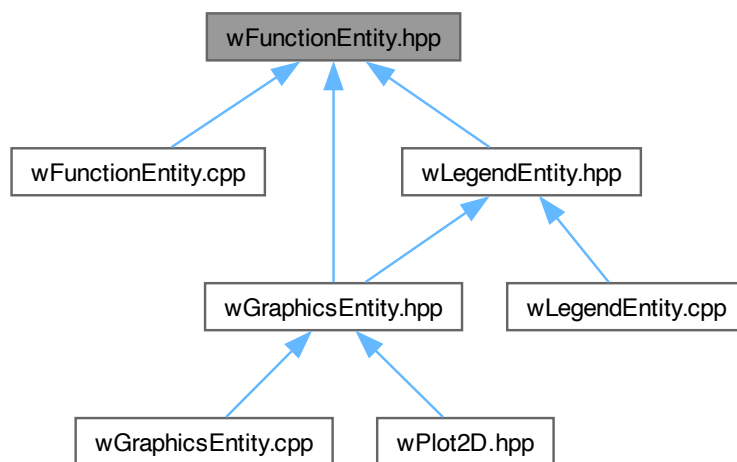
Implementation of the FunctionEntity class.

8.57 wFunctionEntity.hpp File Reference

```
#include "../srcEntity/wEntity.hpp"
#include <SFML/System/Vector2.hpp>
#include <SFML/Graphics.hpp>
#include <functional>
#include "../srcComponents/wLineStyleComponent.hpp"
Include dependency graph for wFunctionEntity.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [wPlot2D::FunctionEntity](#)

Represents a mathematical function as a drawable entity in a 2D plot.

Namespaces

- namespace [wPlot2D](#)

8.58 wFunctionEntity.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002
00003  +-----+
00004  Created by Wilfried Koch.
00005  Copyright © 2025 Wilfried Koch. All rights reserved.
00006  +-----+
00007  */
00008  #ifndef W_FUNCTION_ENTITY_HPP
00009  #define W_FUNCTION_ENTITY_HPP
00010
00011  #include "../srcEntity/wEntity.hpp"
00012
00013  #pragma GCC diagnostic push
00014  #pragma GCC diagnostic ignored "-Wfloat-equal"
00015  #pragma GCC diagnostic ignored "-Wswitch-default"
00016  #include <SFML/System/Vector2.hpp>
00017  #include <SFML/Graphics.hpp>
00018  #pragma GCC diagnostic pop
00019
00020  #include <functional>
00021
00022  #include "../srcComponents/wLineStyleComponent.hpp"
00023
00024  namespace wPlot2D
00025  {
00026
00044      class FunctionEntity : public wEngine::Entity
00045      {
00046      public:
00053          FunctionEntity( const sf::Vector2f origin, const sf::Vector2f scale, std::function<
double( double ) > func );
00054
00058          virtual ~FunctionEntity( ) = default;
00059
00064          [[nodiscard]] sf::Vector2f getPosition( ) const;
00065
00070          [[nodiscard]] sf::Color getColor( ) const;
00071
00076          [[nodiscard]] float getThickness( ) const;
00077
00082          [[nodiscard]] wEngine::LineStyleComponent::LineStyle getLineStyle( ) const;
00083
00089          [[nodiscard]] float getDashLength( ) const;
00090
00096          [[nodiscard]] float getGapLength( ) const;
00097
00102          [[nodiscard]] sf::Vector2f getOffset( ) const;
00103
00108          [[nodiscard]] float getRotation( ) const;
00109
00114          void setPosition( sf::Vector2f position );
00115
00120          void setColor( sf::Color color );
00121
00126          void setThickness( float thickness );
00127
00132          void setLineStyle( wEngine::LineStyleComponent::LineStyle style );
00133
00140          void setDashLength( float dashLength );
00141
00148          void setGapLength( float gapLength );
00149
00160          void setOffset( float offsetX, float offsetY );
00161
00171          void setRotation( float angleDegrees );
00172
00181          void setScale( sf::Vector2f scale );
00182
00191          void addExcludedInterval( double min, double max );
00192
00196          void clearExcludedIntervals( );
00197
00210          void alignToYAxis( float normalizedOffsetX = 0.0f, float normalizedOffsetY = 0.0f );
00211
00219          void drawFunction( sf::RenderWindow &window, double startX, double endX, size_t nbPoints =
1000 );
00220      private:
00221      };
00222
00223  } //End of namespace wPlot2D

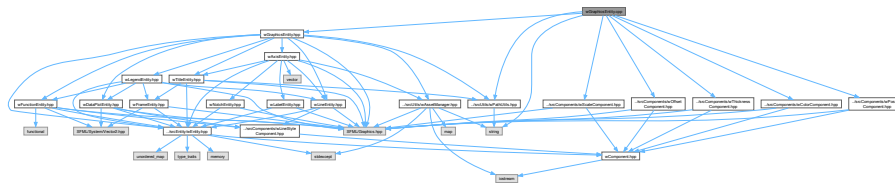
```

```
00224
00225 #endif
```

8.59 wGraphicsEntity.cpp File Reference

Implementation of the GraphicsEntity class.

```
#include "wGraphicsEntity.hpp"
#include <string>
#include "../srcUtils/wPathUtils.hpp"
#include "../srcComponents/wColorComponent.hpp"
#include "../srcComponents/wPositionComponent.hpp"
#include "../srcComponents/wScaleComponent.hpp"
#include "../srcComponents/wOffsetComponent.hpp"
#include "../srcComponents/wThicknessComponent.hpp"
Include dependency graph for wGraphicsEntity.cpp:
```



Namespaces

- namespace [wPlot2D](#)

8.59.1 Detailed Description

Implementation of the GraphicsEntity class.

8.60 wGraphicsEntity.hpp File Reference

```
#include "../srcEntity/wEntity.hpp"
#include "../srcUtils/wAssetManager.hpp"
#include "../srcUtils/wPathUtils.hpp"
#include "wAxisEntity.hpp"
#include "wTitleEntity.hpp"
#include "wFunctionEntity.hpp"
#include "wDataPlotEntity.hpp"
#include "wLegendEntity.hpp"
#include "wLineEntity.hpp"
```


8.61 wGraphicsEntity.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002
00003  +-----+
00004  Created by Wilfried Koch.
00005  Copyright © 2025 Wilfried Koch. All rights reserved.
00006  +-----+
00007  */
00008  #ifndef W_GRAPHICS_ENTITY_HPP
00009  #define W_GRAPHICS_ENTITY_HPP
00010
00011  #include "../srcEntity/wEntity.hpp"
00012
00013  #include "../srcUtils/wAssetManager.hpp"
00014  #include "../srcUtils/wPathUtils.hpp"
00015
00016  #include "wAxisEntity.hpp"
00017  #include "wTitleEntity.hpp"
00018  #include "wFunctionEntity.hpp"
00019  #include "wDataPlotEntity.hpp"
00020  #include "wLegendEntity.hpp"
00021  #include "wLineEntity.hpp"
00022
00023  #pragma GCC diagnostic push
00024  #pragma GCC diagnostic ignored "-Wfloat-equal"
00025  #pragma GCC diagnostic ignored "-Wswitch-default"
00026  #include <SFML/Graphics.hpp>
00027  #pragma GCC diagnostic pop
00028
00029  namespace wPlot2D
00030  {
00031
00032      enum class TitleAlignment
00033      {
00034          Top,
00035          Bottom
00036      };
00037
00038      class GraphicsEntity : public wEngine::Entity
00039      {
00040      public:
00041          GraphicsEntity(
00042              const std::string& windowTitle = "wPlot2D",
00043              const sf::Vector2u& windowSize = { 1600, 1600 },
00044              const sf::Vector2f& originFactor = { 0.5f, 0.5f },
00045              const sf::Vector2f& scaleFactor = { 0.1f, 0.1f } );
00046
00047          virtual ~GraphicsEntity() = default;
00048
00049          [[nodiscard]] sf::RenderWindow& getWindow();
00050
00051          [[nodiscard]] sf::Vector2u getWindowSize() const;
00052
00053          void setWindowSize( const sf::Vector2u& newSize );
00054
00055          void setWindowTitle( const std::string& title );
00056
00057          void setBackgroundColor( const sf::Color& color );
00058
00059          void addFont( const std::string& name, const std::string& fileName );
00060
00061          sf::Font& getFont( const std::string name );
00062
00063          [[nodiscard]] sf::Vector2f getOrigin() const;
00064
00065          void setOrigin( sf::Vector2f originFactor );
00066
00067          [[nodiscard]] sf::Vector2f getScale() const;
00068
00069          void setScale( sf::Vector2f scaleFactor );
00070
00071          [[nodiscard]] sf::Vector2f getOffset() const;
00072
00073          void setOffset( sf::Vector2f offset );
00074
00075          [[nodiscard]] AxisEntity* addAxis( AxisType type, sf::Vector2f axisRange );
00076
00077          [[nodiscard]] TitleEntity* addTitle( const std::string& title, TitleAlignment alignment =
00078              TitleAlignment::Bottom );
00079
00080      };
00081
00082  }
00083
00084  #endif

```

```

00195         [[nodiscard]] TitleEntity* addTitle( const std::wstring& title, TitleAlignment alignment =
TitleAlignment::Bottom );
00196
00205         [[nodiscard]] FunctionEntity* addFunction( std::function< double( double )> func,
00206             double startX, double endX, size_t nbPoints = 1000 );
00207
00213         [[nodiscard]] DataPlotEntity* addDataPlot( const std::vector< sf::Vector2f >& dataPoints
);
00214
00221         [[nodiscard]] LegendEntity* addLegend( const sf::Vector2f& position, bool hasFrame = true
);
00222
00229         [[nodiscard]] TitleEntity* addText( const std::string& text, sf::Vector2f position );
00230
00237         [[nodiscard]] TitleEntity* addText( const std::wstring& text, sf::Vector2f position );
00238
00246         [[nodiscard]] LineEntity* addLine( const sf::Vector2f& start, const sf::Vector2f& end,
bool withArrow = false );
00247
00253         void saveToFile( const std::string& filename );
00254     private:
00255         sf::RenderWindow mWindow;
00256         wEngine::AssetManager mAssets;
00257         std::unique_ptr< AxisEntity > mAxisX;
00258         std::unique_ptr< AxisEntity > mAxisY;
00259         std::unique_ptr< TitleEntity > mTitle;
00260         TitleAlignment mAlignment;
00261
00262     template < typename T >
00263     TitleEntity* addTitleImpl( const T& title, TitleAlignment alignment );
00264
00265     template < typename T >
00266     TitleEntity* initText( const T& title, sf::Vector2f position );
00267
00268     struct FunctionData
00269     {
00270         std::unique_ptr< FunctionEntity > entity;
00271         double startX;
00272         double endX;
00273         size_t nbPoints;
00274     };
00275     std::vector< FunctionData > mFunctions;
00276
00277     struct DataPlotData
00278     {
00279         std::unique_ptr< DataPlotEntity > entity;
00280     };
00281     std::vector< DataPlotData > mDataPlots;
00282
00283     std::unique_ptr< LegendEntity > mLegend;
00284
00285     struct TextData
00286     {
00287         std::unique_ptr< TitleEntity > entity;
00288     };
00289     std::vector< TextData > mTexts;
00290
00291     struct LineData
00292     {
00293         std::unique_ptr< LineEntity > entity;
00294     };
00295     std::vector< LineData > mLines;
00296
00313     void render( );
00314
00325     void validateNormalizedFactor( const sf::Vector2f& factor ) const;
00326
00338     [[nodiscard]] sf::Vector2f convertNormalizedToPixels( const sf::Vector2f& factor ) const;
00339 };
00340
00341 } //End of namespace wPlot2D
00342
00343 #endif

```

8.62 wLabelEntity.cpp File Reference

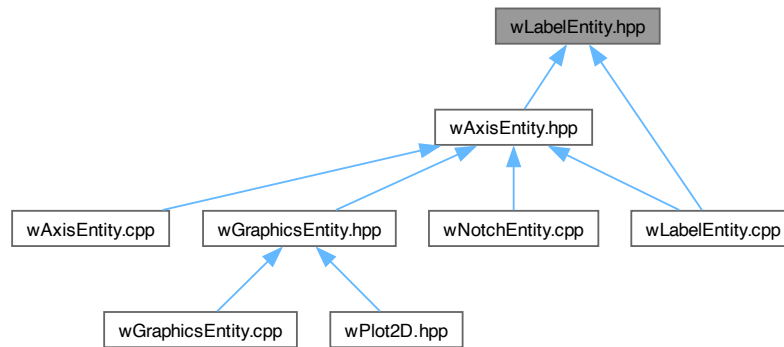
Implementation of the LabelEntity class.

```

#include "wLabelEntity.hpp"
#include "wAxisEntity.hpp"

```


This graph shows which files directly or indirectly include this file:



Classes

- class [wPlot2D::LabelEntity](#)
Represents a textual label or a collection of axis labels.

Namespaces

- namespace [wPlot2D](#)

8.64 wLabelEntity.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002  +-----+
00003  Created by Wilfried Koch.
00004  Copyright © 2025 Wilfried Koch. All rights reserved.
00005  +-----+
00006  */
00007
00008 #ifndef W_LABEL_ENTITY_HPP
00009 #define W_LABEL_ENTITY_HPP
00010
00011 #include "../srcEntity/wEntity.hpp"
00012
00013 #pragma GCC diagnostic push
00014 #pragma GCC diagnostic ignored "-Wfloat-equal"
00015 #pragma GCC diagnostic ignored "-Wswitch-default"
00016 #include <SFML/Graphics.hpp>
00017 #pragma GCC diagnostic pop
00018
00019 namespace wPlot2D
00020 {
00021     enum class AxisType;
00022
00023     class LabelEntity : public wEngine::Entity
00024     {
00025     public:
00026         LabelEntity( const sf::Font& font, AxisType type, sf::Vector2f initialPosition );
00027
00028         virtual ~LabelEntity( ) = default;
00029     };
00030 }

```

```

00063         [[nodiscard]] float  getValue( ) const;
00064
00069         [[nodiscard]] unsigned int  getCharacterSize( ) const;
00070
00075         [[nodiscard]] int  getDecimalPlaces( ) const;
00076
00081     void  setFont( const sf::Font& font );
00082
00091     void  setLabelText( std::string text );
00092
00097     void  setCharacterSize( unsigned int newSize );
00098
00103     void  setDecimalPlaces( int places );
00104
00113     void  setCustomLabels( const std::string& labels );
00114
00119     [[nodiscard]] bool  usesCustomLabels( ) const;
00120
00130     std::string  formatLabel( float value );
00131
00136     void  render( sf::RenderWindow& window );
00137 private:
00138     AxisType  mAlignment;
00139     unsigned int  mCharacterSize;
00140     float  mValue;
00141     int  mDecimalPlaces;
00142     sf::Vector2f  mOffset;
00143
00144     std::string  mCustomLabels;
00145     bool  mUseCustomLabels;
00146
00147     sf::Text  mLabel;
00148 };
00149
00150 } // namespace wPlot2D
00151
00152 #endif

```

8.65 wLegendEntity.cpp File Reference

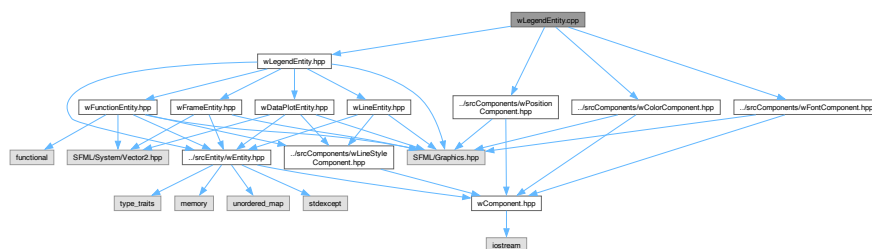
Implementation of the LegendEntity class.

```

#include "wLegendEntity.hpp"
#include "../srcComponents/wPositionComponent.hpp"
#include "../srcComponents/wColorComponent.hpp"
#include "../srcComponents/wFontComponent.hpp"

```

Include dependency graph for wLegendEntity.cpp:



Namespaces

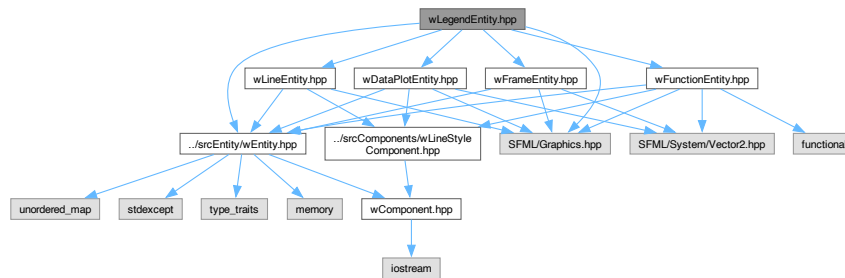
- namespace [wPlot2D](#)

8.65.1 Detailed Description

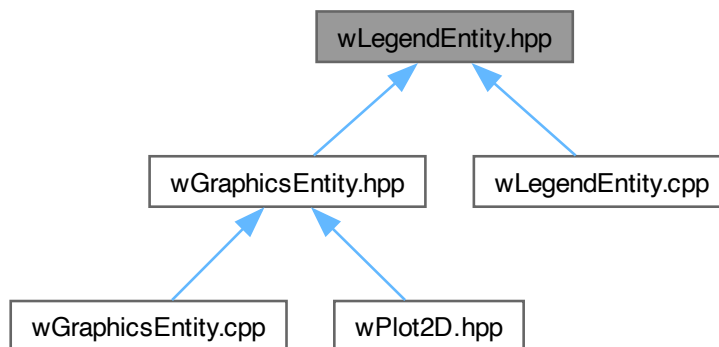
Implementation of the LegendEntity class.

8.66 wLegendEntity.hpp File Reference

```
#include "../srcEntity/wEntity.hpp"
#include "wLineEntity.hpp"
#include "wFrameEntity.hpp"
#include "wFunctionEntity.hpp"
#include "wDataPlotEntity.hpp"
#include <SFML/Graphics.hpp>
Include dependency graph for wLegendEntity.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [wPlot2D::LegendEntity](#)
Represents a legend box that describes functions and data plots.

Namespaces

- namespace [wPlot2D](#)

8.67 wLegendEntity.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002
00003  +-----+
00004  Created by Wilfried Koch.
00005  Copyright @ 2025 Wilfried Koch. All rights reserved.
00006  +-----+
00007  */
00008  #ifndef W_LEGEND_ENTITY_HPP
00009  #define W_LEGEND_ENTITY_HPP
00010
00011  #include "../srcEntity/wEntity.hpp"
00012  #include "wLineEntity.hpp"
00013  #include "wFrameEntity.hpp"
00014
00015  #include "wFunctionEntity.hpp"
00016  #include "wDataPlotEntity.hpp"
00017
00018  #pragma GCC diagnostic push
00019  #pragma GCC diagnostic ignored "-Wfloat-equal"
00020  #pragma GCC diagnostic ignored "-Wswitch-default"
00021  #include <SFML/Graphics.hpp>
00022  #pragma GCC diagnostic pop
00023
00024  namespace wPlot2D
00025  {
00026
00027
00055      class LegendEntity : public wEngine::Entity
00056      {
00057      public:
00067          LegendEntity( const sf::Font& font, const sf::Vector2f& position, bool hasFrame = true );
00068
00072          virtual ~LegendEntity( ) = default;
00073
00079          void addItem( const std::string& label, FunctionEntity* function );
00080
00086          void addItem( const std::wstring& label, FunctionEntity* function );
00087
00093          void addItem( const std::string& label, DataPlotEntity* plot );
00094
00100          void addItem( const std::wstring& label, DataPlotEntity* plot );
00101
00106          void setFrameEnabled( bool enabled );
00107
00112          void setFrameFillColor( const sf::Color& color );
00113
00118          void setFrameOutlineColor( const sf::Color& color );
00119
00124          void setFrameThickness( float thickness );
00125
00135          void setPadding( const sf::Vector2f& padding );
00136
00143          void setFont( const sf::Font& font );
00144
00149          void setCharacterSize( unsigned int size );
00150
00155          void setTextColor( const sf::Color& color );
00156
00165          void render( sf::RenderWindow& window );
00166      private:
00171          struct LegendItem
00172          {
00173              std::string mLabel;
00174              std::unique_ptr< LineEntity > mLine;
00175              sf::Text mLabelText;
00176
00177              LegendItem( std::unique_ptr< LineEntity > line, sf::Text&& txt )
00178              : mLine{ std::move( line ) },
00179                mLabelText{ std::move( txt ) }
00180              {
00181
00182              }
00182          };
00183
00184
00197          template < typename LabelT, typename SourceT >
00198          void addItemGeneric( const LabelT& label, SourceT* source );
00199
00211          template< typename T >
00212          void createItem( const T& label, std::unique_ptr< LineEntity > line );
00213

```

```

00214         std::vector< LegendItem > mItems;
00215         const sf::Font& mFont;
00216         unsigned int mCharacterSize = 30;
00217         FrameEntity mFrame;
00218     };
00219
00220 } // namespace wPlot2D
00221
00222 #endif

```

8.68 wLineEntity.cpp File Reference

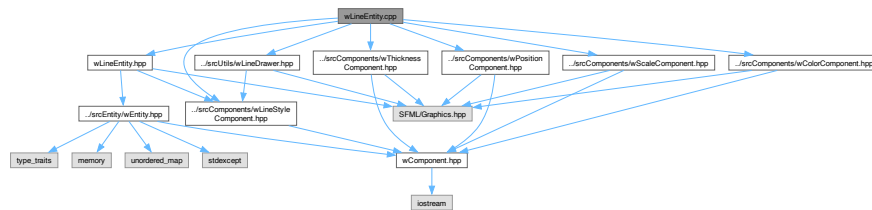
Implementation of the LineEntity class.

```

#include "wLineEntity.hpp"
#include "../srcComponents/wPositionComponent.hpp"
#include "../srcComponents/wScaleComponent.hpp"
#include "../srcComponents/wColorComponent.hpp"
#include "../srcComponents/wThicknessComponent.hpp"
#include "../srcComponents/wLineStyleComponent.hpp"
#include "../srcUtils/wLineDrawer.hpp"

```

Include dependency graph for wLineEntity.cpp:



Namespaces

- namespace [wPlot2D](#)

8.68.1 Detailed Description

Implementation of the LineEntity class.

8.69 wLineEntity.hpp File Reference

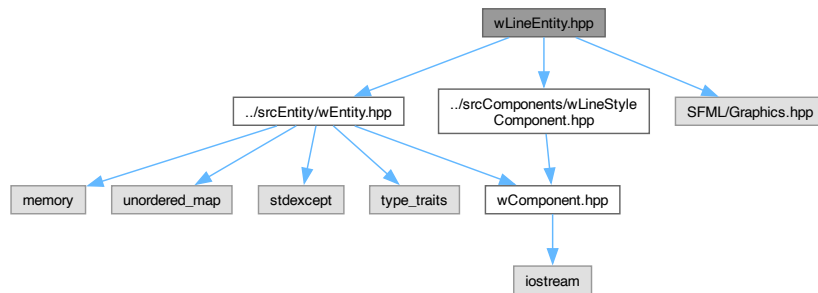
```

#include "../srcEntity/wEntity.hpp"
#include "../srcComponents/wLineStyleComponent.hpp"

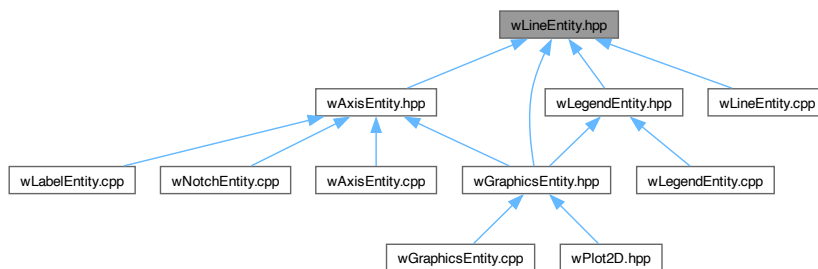
```

```
#include <SFML/Graphics.hpp>
```

Include dependency graph for `wLineEntity.hpp`:



This graph shows which files directly or indirectly include this file:



Classes

- class `wPlot2D::LineEntity`
Entity representing a straight line segment with optional arrowhead.

Namespaces

- namespace `wPlot2D`

8.70 wLineEntity.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002
00003  +-----+
00004  Created by Wilfried Koch.
00005  Copyright © 2025 Wilfried Koch. All rights reserved.
00006  +-----+
00007  */
00008  #ifndef W_LINE_ENTITY_HPP

```

```

00009 #define W_LINE_ENTITY_HPP
00010
00011 #include "../srcEntity/wEntity.hpp"
00012 #include "../srcComponents/wLineStyleComponent.hpp"
00013
00014 #pragma GCC diagnostic push
00015 #pragma GCC diagnostic ignored "-Wfloat-equal"
00016 #pragma GCC diagnostic ignored "-Wswitch-default"
00017 #include <SFML/Graphics.hpp>
00018 #pragma GCC diagnostic pop
00019
00020 namespace wPlot2D
00021 {
00022
00042     class LineEntity : public wEngine::Entity
00043     {
00044     public:
00054         LineEntity( const sf::Vector2f& origin, const sf::Vector2f& scale, const sf::Vector2f&
start, const sf::Vector2f& end,
00055                     bool withArrow = false );
00056
00060         virtual ~LineEntity( ) = default;
00061
00066         void setColor( sf::Color color );
00067
00072         void setThickness( float thickness );
00073
00078         [[nodiscard]] float getThickness( ) const;
00079
00084         void setLineStyle( wEngine::LineStyleComponent::LineStyle style );
00085
00090         void setDashLength( float dashLength );
00091
00096         void setGapLength( float gapLength );
00097
00102         [[nodiscard]] sf::Vector2f getStartPoint( ) const;
00103
00108         [[nodiscard]] sf::Vector2f getEndPoint( ) const;
00109
00114         [[nodiscard]] bool hasArrow( ) const;
00115
00120         [[nodiscard]] float getArrowSize( ) const;
00121
00126         void setArrowSize( float arrowSize );
00127
00132         void render( sf::RenderWindow& window );
00133
00134     private:
00135         sf::Vector2f mStart;
00136         sf::Vector2f mEnd;
00137         bool mWithArrow;
00138         sf::ConvexShape mArrowHead;
00139         float mArrowSize;
00140
00152         void initArrowHead( const sf::Vector2f& lineEnd, const sf::Vector2f& dir, float arrowSize,
sf::Color color );
00153     };
00154
00155 } //End of namespace wPlot2D
00156
00157 #endif

```

8.71 wNotchEntity.cpp File Reference

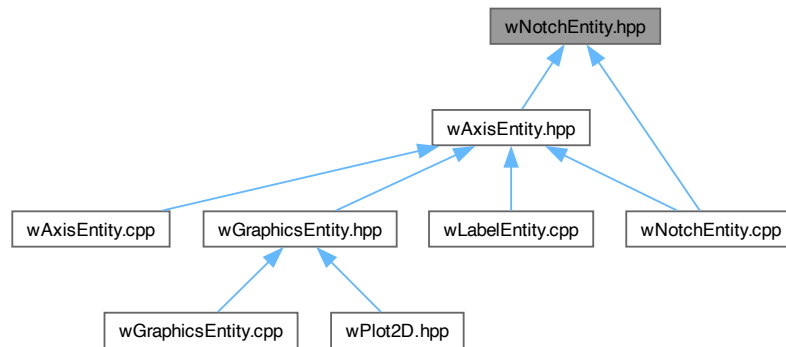
Implementation of the NotchEntity class.

```

#include "wNotchEntity.hpp"
#include "wAxisEntity.hpp"
#include "../srcComponents/wPositionComponent.hpp"
#include "../srcComponents/wColorComponent.hpp"
#include "../srcComponents/wThicknessComponent.hpp"
#include "../srcComponents/wLengthComponent.hpp"

```


This graph shows which files directly or indirectly include this file:



Classes

- class [wPlot2D::NotchEntity](#)
Represents a single tick mark ("notch") on a 2D axis.

Namespaces

- namespace [wPlot2D](#)

8.73 wNotchEntity.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002  +-----+
00003  Created by Wilfried Koch.
00004  Copyright © 2025 Wilfried Koch. All rights reserved.
00005  +-----+
00006  */
00007
00008 #ifndef W_NOTCH_ENTITY_HPP
00009 #define W_NOTCH_ENTITY_HPP
00010
00011 #include "../srcEntity/wEntity.hpp"
00012
00013 #pragma GCC diagnostic push
00014 #pragma GCC diagnostic ignored "-Wfloat-equal"
00015 #pragma GCC diagnostic ignored "-Wswitch-default"
00016 #include <SFML/Graphics.hpp>
00017 #pragma GCC diagnostic pop
00018
00019 namespace wPlot2D
00020 {
00021     enum class AxisType;
00022
00023     class NotchEntity : public wEngine::Entity
00024     {
00025     public:
00026         NotchEntity( AxisType type );
00027
00028         virtual ~NotchEntity( ) = default;
00029     };
00030 }

```

```

00074         void render( sf::RenderWindow& window );
00075
00076     private:
00077         AxisType mAlignment;
00078     };
00079
00080 } // namespace wPlot2D
00081
00082 #endif

```

8.74 wTitleEntity.cpp File Reference

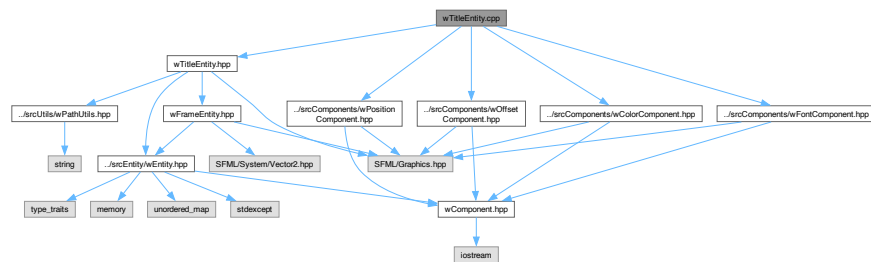
Implementation of the TitleEntity class.

```

#include "wTitleEntity.hpp"
#include "../srcComponents/wPositionComponent.hpp"
#include "../srcComponents/wOffsetComponent.hpp"
#include "../srcComponents/wColorComponent.hpp"
#include "../srcComponents/wFontComponent.hpp"

```

Include dependency graph for wTitleEntity.cpp:



Namespaces

- namespace [wPlot2D](#)

8.74.1 Detailed Description

Implementation of the TitleEntity class.

8.75 wTitleEntity.hpp File Reference

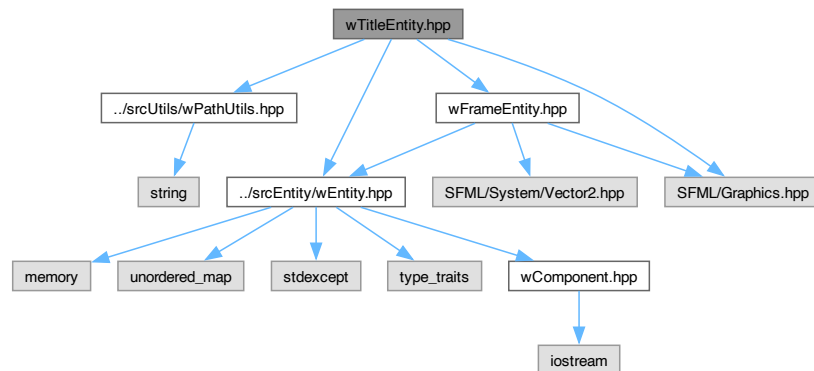
```

#include "../srcEntity/wEntity.hpp"
#include "../srcUtils/wPathUtils.hpp"
#include "wFrameEntity.hpp"

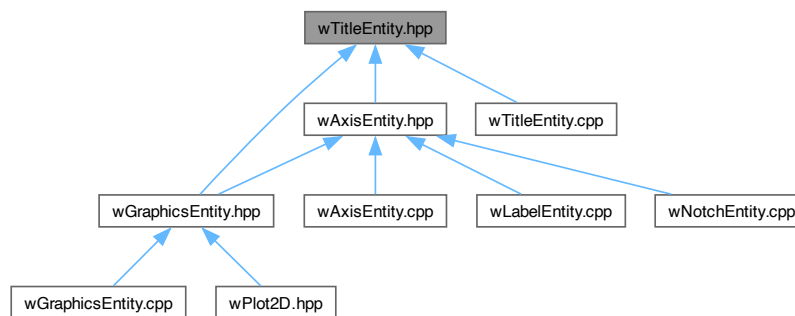
```

```
#include <SFML/Graphics.hpp>
```

Include dependency graph for wTitleEntity.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [wPlot2D::TitleEntity](#)

Represents a textual label (typically an axis title or main plot title) in a 2D plot.

Namespaces

- namespace [wPlot2D](#)

8.76 wTitleEntity.hpp

[Go to the documentation of this file.](#)

```
00001 /*
00002
```

```

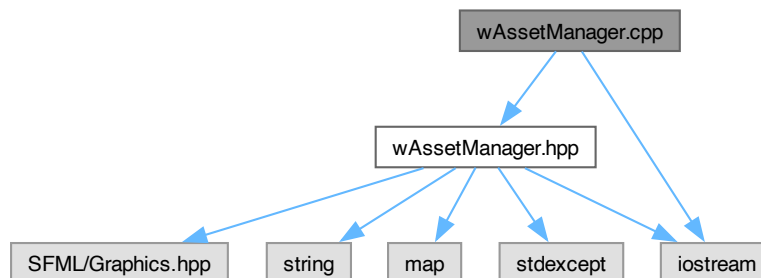
00003 Created by Wilfried Koch.
00004 Copyright @ 2025 Wilfried Koch. All rights reserved.
00005
00006 +-----+
00006 */
00007
00008 #ifndef W_TITLE_ENTITY_HPP
00009 #define W_TITLE_ENTITY_HPP
00010
00011 #include "../srcEntity/wEntity.hpp"
00012 #include "../srcUtils/wPathUtils.hpp"
00013 #include "wFrameEntity.hpp"
00014
00015 #pragma GCC diagnostic push
00016 #pragma GCC diagnostic ignored "-Wfloat-equal"
00017 #pragma GCC diagnostic ignored "-Wswitch-default"
00018 #include <SFML/Graphics.hpp>
00019 #pragma GCC diagnostic pop
00020
00021 namespace wPlot2D
00022 {
00023
00024     class TitleEntity : public wEngine::Entity
00025     {
00026     public:
00027         TitleEntity( const sf::Font& font, const std::string& title, bool hasFrame = false );
00028         TitleEntity( const sf::Font& font, const std::wstring& title, bool hasFrame = false );
00029         virtual ~TitleEntity( ) = default;
00030
00031         [[nodiscard]] unsigned int getCharacterSize( ) const;
00032         [[nodiscard]] sf::FloatRect getTextSize( ) const;
00033         void setTextColor( sf::Color textColor );
00034         void setOffset( sf::Vector2f offset );
00035         void setCharacterSize( unsigned int size );
00036         void setFont( const sf::Font& font );
00037         [[nodiscard]] sf::Color getFrameOutlineColor( ) const;
00038         [[nodiscard]] sf::Color getFrameFillColor( ) const;
00039         [[nodiscard]] float getFrameThickness( ) const;
00040         [[nodiscard]] sf::Vector2f getPadding( ) const;
00041         [[nodiscard]] bool isFrameEnabled( ) const;
00042         void setFrameEnabled( bool enabled );
00043         void setFrameOutlineColor( const sf::Color& color );
00044         void setFrameFillColor( const sf::Color& color );
00045         void setFrameThickness( float thickness );
00046         void setPadding( sf::Vector2f padding );
00047         void render( sf::RenderWindow& window );
00048     private:
00049         sf::Text mTitleText;
00050         FrameEntity mFrame;
00051         template < typename T >
00052         void init( const T& title );
00053     };
00054 } // namespace wPlot2D
00055 #endif

```

8.77 wAssetManager.cpp File Reference

Implementation of the AssetManager class.

```
#include "wAssetManager.hpp"
#include <iostream>
Include dependency graph for wAssetManager.cpp:
```



Namespaces

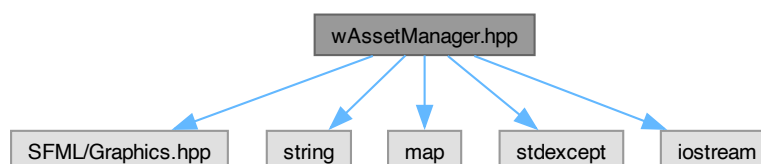
- namespace `wEngine`

8.77.1 Detailed Description

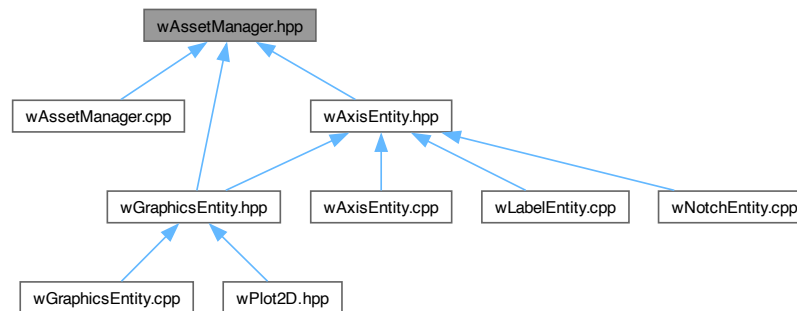
Implementation of the `AssetManager` class.

8.78 wAssetManager.hpp File Reference

```
#include <SFML/Graphics.hpp>
#include <string>
#include <map>
#include <stdexcept>
#include <iostream>
Include dependency graph for wAssetManager.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [wEngine::AssetManager](#)
Manages graphical assets such as fonts for reuse across the application.

Namespaces

- namespace [wEngine](#)

8.79 wAssetManager.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002
00003  +-----+
00003  Created by Wilfried Koch.
00004  Copyright © 2025 Wilfried Koch. All rights reserved.
00005
00006  +-----+
00006  */
00007
00008  #ifndef W_ASSET_MANAGER_HPP
00009  #define W_ASSET_MANAGER_HPP
00010
00011  #pragma GCC diagnostic push
00012  #pragma GCC diagnostic ignored "-Wfloat-equal"
00013  #pragma GCC diagnostic ignored "-Wswitch-default"
00014  #include <SFML/Graphics.hpp>
00015  #pragma GCC diagnostic pop
00016
00017  #include <string>
00018  #include <map>
00019  #include <stdexcept>
00020  #include <iostream>
00021
00022  namespace wEngine
00023  {
00024
00049      class AssetManager
00050      {
00051      public:
00052          AssetManager( ) = default;
00053          AssetManager( const AssetManager& ) = delete;
00054          AssetManager& operator=( const AssetManager& ) = delete;
00055          ~AssetManager( ) = default;
00056
00068          void LoadFont( const std::string& name, const std::string& fileName );
  
```

```

00069
00089         void LoadFont( const std::string& name, const std::vector< std::string >& possiblePaths );
00090
00097         sf::Font& getFont( const std::string& name );
00098
00104         void RemoveFont( const std::string& name );
00105
00109         void debugPrintFonts( ) const;
00110     private:
00111         std::map< std::string, sf::Font > mFont;
00112
00126         template < typename Map >
00127         static void EnsureExists( const Map& map, const std::string& name, const std::string& type
00128     );
00129 };
00130 } //End of namespace wEngine
00131
00132 #endif

```

8.80 wLineDrawer.cpp File Reference

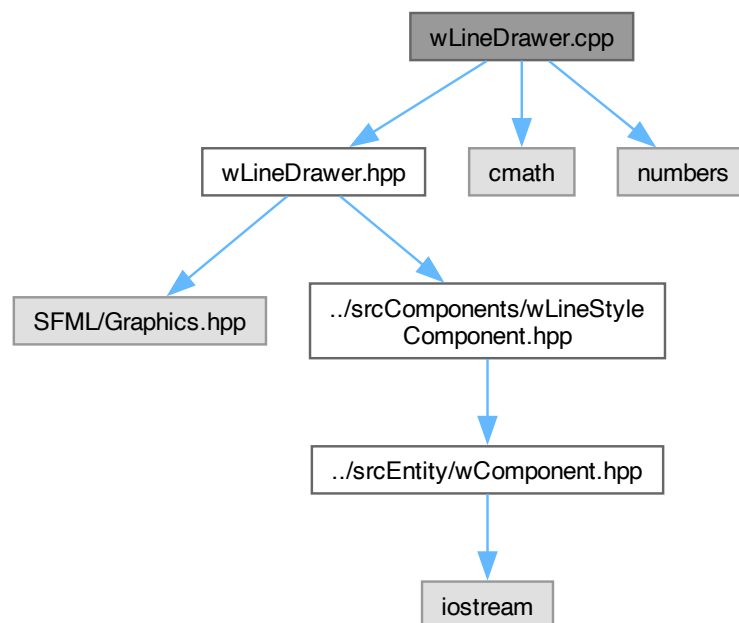
Implementation of the LineDrawer class.

```

#include "wLineDrawer.hpp"
#include <cmath>
#include <numbers>

```

Include dependency graph for wLineDrawer.cpp:



Namespaces

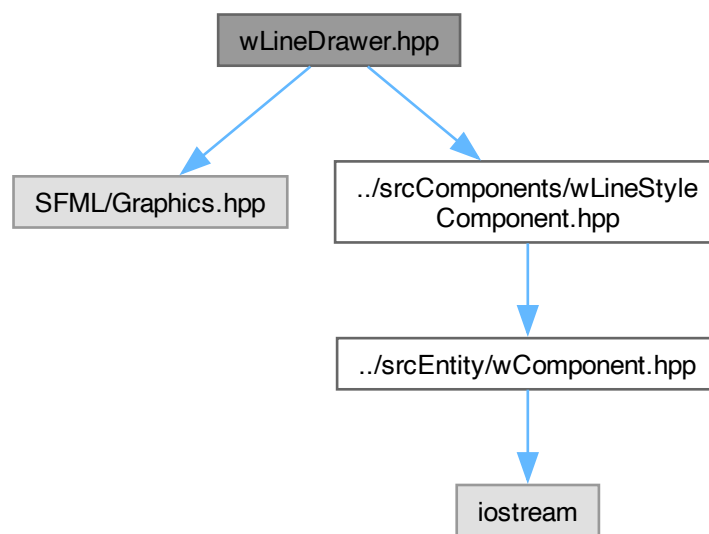
- namespace [wEngine](#)

8.80.1 Detailed Description

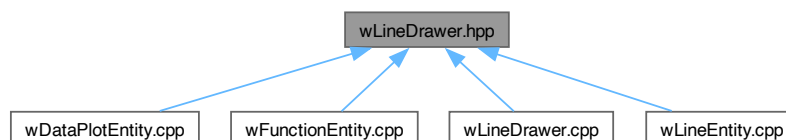
Implementation of the LineDrawer class.

8.81 wLineDrawer.hpp File Reference

```
#include <SFML/Graphics.hpp>
#include "../srcComponents/wLineStyleComponent.hpp"
Include dependency graph for wLineDrawer.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [wEngine::LineDrawer](#)

Utility class for rendering thick lines and polylines with style support.

Namespaces

- namespace [wEngine](#)

8.82 wLineDrawer.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002  +-----+
00003  Created by Wilfried Koch.
00004  Copyright © 2025 Wilfried Koch. All rights reserved.
00005  +-----+
00006  */
00007
00008 #ifndef W_LINE_DRAWER_HPP
00009 #define W_LINE_DRAWER_HPP
00010
00011 #pragma GCC diagnostic push
00012 #pragma GCC diagnostic ignored "-Wfloat-equal"
00013 #pragma GCC diagnostic ignored "-Wswitch-default"
00014 #include <SFML/Graphics.hpp>
00015 #pragma GCC diagnostic pop
00016
00017 #include "../srcComponents/wLineStyleComponent.hpp"
00018
00019 namespace wEngine
00020 {
00021
00051     class LineDrawer
00052     {
00053     public:
00054
00081         static float drawLine( sf::RenderWindow& window, const sf::Vector2f& point1, const
sf::Vector2f& point2,
00082             const sf::Color& color, float thickness, LineStyleComponent::LineStyle style =
LineStyleComponent::LineStyle::Solid,
00083             float dashLength = 20.0f, float gapLength = 5.0f, float patternOffset = 0.0f );
00084
00102         static void drawPolylineRound( sf::RenderWindow& window, const std::vector<sf::Vector2f>&
points, const sf::Color& color,
00103             float thickness, LineStyleComponent::LineStyle style =
LineStyleComponent::LineStyle::Solid, float dashLength = 20.0f,
00104             float gapLength = 5.0f, unsigned int arcResolution = 12 );
00105     };
00106
00107 } //End of namespace wEngine
00108
00109 #endif

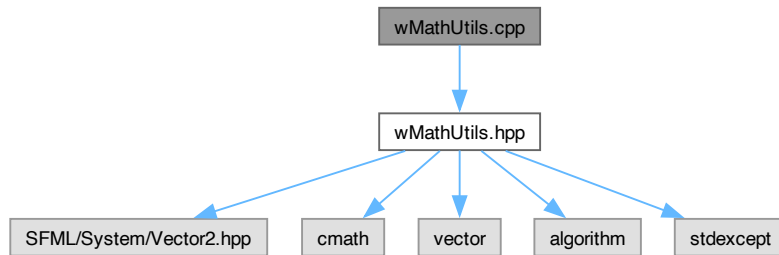
```

8.83 wMathUtils.cpp File Reference

Implementation of the MathUtils class.

```
#include "wMathUtils.hpp"
```

Include dependency graph for wMathUtils.cpp:



Namespaces

- namespace [wEngine](#)

8.83.1 Detailed Description

Implementation of the MathUtils class.

8.84 wMathUtils.hpp File Reference

```
#include <SFML/System/Vector2.hpp>
```

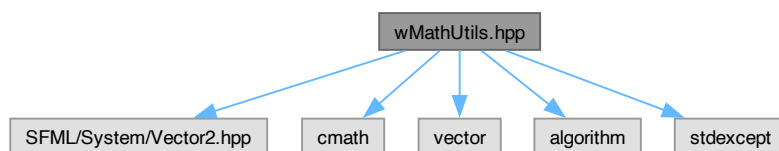
```
#include <cmath>
```

```
#include <vector>
```

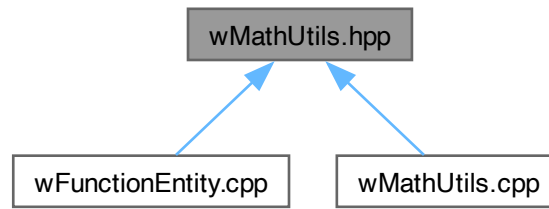
```
#include <algorithm>
```

```
#include <stdexcept>
```

Include dependency graph for wMathUtils.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [wEngine::MathUtils](#)
Provides common mathematical helper functions for plotting and geometry.

Namespaces

- namespace [wEngine](#)

8.85 wMathUtils.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002  +-----+
00003  Created by Wilfried Koch.
00004  Copyright © 2025 Wilfried Koch. All rights reserved.
00005  +-----+
00006  */
00007
00008 #ifndef W_MATH_UTILS_HPP
00009 #define W_MATH_UTILS_HPP
00010
00011 #pragma GCC diagnostic push
00012 #pragma GCC diagnostic ignored "-Wfloat-equal"
00013 #include <SFML/System/Vector2.hpp>
00014 #pragma GCC diagnostic pop
00015
00016 #include <cmath>
00017 #include <vector>
00018 #include <algorithm>
00019 #include <stdexcept>
00020
00021 namespace wEngine
00022 {
00023
00037     class MathUtils
00038     {
00039     public:
00057         [[nodiscard]] static std::vector< double > linspace( double start, double end, size_t
nbPoints );
00058     };
00059
00060 } //End of namespace wEngine
00061
00062 #endif

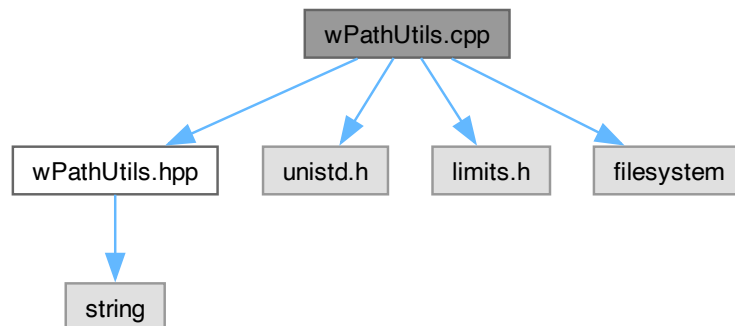
```

8.86 wPathUtils.cpp File Reference

Implementation of the PathUtils class.

```
#include "wPathUtils.hpp"
#include <unistd.h>
#include <limits.h>
#include <filesystem>
```

Include dependency graph for wPathUtils.cpp:



Namespaces

- namespace [wEngine](#)

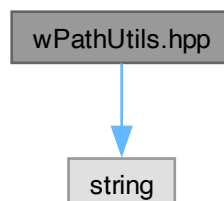
8.86.1 Detailed Description

Implementation of the PathUtils class.

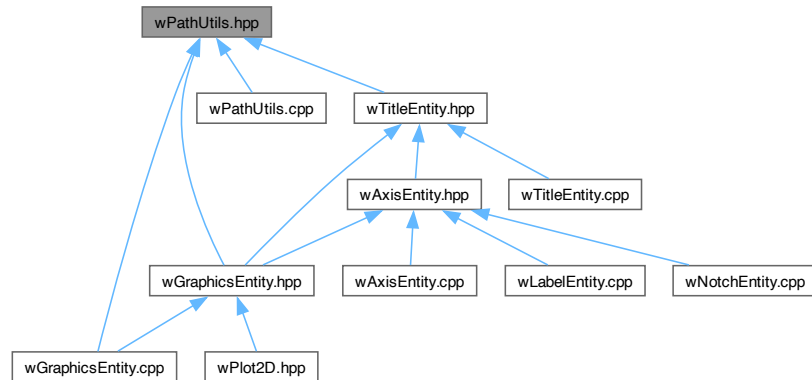
8.87 wPathUtils.hpp File Reference

```
#include <string>
```

Include dependency graph for wPathUtils.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [wEngine::PathUtils](#)

Utility class providing static functions for managing executable and resource paths across platforms.

Namespaces

- namespace [wEngine](#)

8.88 wPathUtils.hpp

[Go to the documentation of this file.](#)

```

00001  /*
00002
00003  +-----+
00004  Created by Wilfried Koch.
00005  Copyright @ 2025 Wilfried Koch. All rights reserved.
00006  +-----+
00007  */
00008  #ifndef W_PATH_UTILS_HPP
00009  #define W_PATH_UTILS_HPP
00010
00011  #include <string>
00012
00013  namespace wEngine
00014  {
00015
00044      class PathUtils
00045      {
00046      public:
00059          [[nodiscard]] static std::string getExecutablePath( );
00060
00070          [[nodiscard]] static std::string getExecutableDir( );
00071      };
00072
00073  }//End of namespace wEngine
00074
00075  #endif

```


Index

- ~AssetManager
 - wEngine::AssetManager, [16](#)
- ~AxisEntity
 - wPlot2D::AxisEntity, [22](#)
- ~ColorComponent
 - wEngine::ColorComponent, [30](#)
- ~Component
 - wEngine::Component, [31](#)
- ~DataPlotEntity
 - wPlot2D::DataPlotEntity, [35](#)
- ~DiscontinuityComponent
 - wEngine::DiscontinuityComponent, [40](#)
- ~Entity
 - wEngine::Entity, [43](#)
- ~FontComponent
 - wEngine::FontComponent, [49](#)
- ~FrameEntity
 - wPlot2D::FrameEntity, [53](#)
- ~FunctionComponent
 - wEngine::FunctionComponent, [58](#)
- ~FunctionEntity
 - wPlot2D::FunctionEntity, [62](#)
- ~GraphicsEntity
 - wPlot2D::GraphicsEntity, [72](#)
- ~LabelEntity
 - wPlot2D::LabelEntity, [82](#)
- ~LegendEntity
 - wPlot2D::LegendEntity, [89](#)
- ~LengthComponent
 - wEngine::LengthComponent, [95](#)
- ~LineEntity
 - wPlot2D::LineEntity, [102](#)
- ~LineStyleComponent
 - wEngine::LineStyleComponent, [107](#)
- ~NotchEntity
 - wPlot2D::NotchEntity, [113](#)
- ~NotchIntervalComponent
 - wEngine::NotchIntervalComponent, [116](#)
- ~OffsetComponent
 - wEngine::OffsetComponent, [119](#)
- ~PaddingComponent
 - wEngine::PaddingComponent, [122](#)
- ~PositionComponent
 - wEngine::PositionComponent, [127](#)
- ~RotationComponent
 - wEngine::RotationComponent, [130](#)
- ~ScaleComponent
 - wEngine::ScaleComponent, [133](#)
- ~ThicknessComponent
 - wEngine::ThicknessComponent, [136](#)
- ~TitleEntity
 - wPlot2D::TitleEntity, [140](#)
- Above
 - wPlot2D, [13](#)
- addAxis
 - wPlot2D::GraphicsEntity, [76](#)
- addComponent
 - wEngine::Entity, [44](#)
- addDataPlot
 - wPlot2D::GraphicsEntity, [77](#)
- addExcludedInterval
 - wEngine::DiscontinuityComponent, [41](#)
 - wPlot2D::FunctionEntity, [67](#)
- addFont
 - wPlot2D::GraphicsEntity, [73](#)
- addFunction
 - wPlot2D::GraphicsEntity, [77](#)
- addItem
 - wPlot2D::LegendEntity, [89](#), [90](#)
- addLabelsOffset
 - wPlot2D::AxisEntity, [26](#)
- addLegend
 - wPlot2D::GraphicsEntity, [77](#)
- addLine
 - wPlot2D::GraphicsEntity, [78](#)
- addNotches
 - wPlot2D::AxisEntity, [24](#)
- addOffset
 - wEngine::OffsetComponent, [119](#)
- addText
 - wPlot2D::GraphicsEntity, [78](#)
- addTitle
 - wPlot2D::AxisEntity, [23](#)
 - wPlot2D::GraphicsEntity, [76](#)
- alignToYAxis
 - wPlot2D::FunctionEntity, [67](#)
- AssetManager
 - wEngine::AssetManager, [16](#)
- AxisEntity
 - wPlot2D::AxisEntity, [22](#)
- AxisType
 - wPlot2D, [13](#)
- Below
 - wPlot2D, [13](#)
- Bottom
 - wPlot2D, [13](#)

- calculate
 - wEngine::FunctionComponent, 58
- Center
 - wPlot2D, 13
- clearComponents
 - wEngine::Entity, 43
- clearExcludedIntervals
 - wEngine::DiscontinuityComponent, 41
 - wPlot2D::FunctionEntity, 67
- ColorComponent
 - wEngine::ColorComponent, 29
- Component
 - wEngine::Component, 31
- Dashed
 - wEngine::LineStyleComponent, 107
- DataPlotEntity
 - wPlot2D::DataPlotEntity, 35
- debugPrint
 - wEngine::ColorComponent, 30
 - wEngine::DiscontinuityComponent, 41
 - wEngine::FontComponent, 49
 - wEngine::FunctionComponent, 58
 - wEngine::LengthComponent, 95
 - wEngine::LineStyleComponent, 109
 - wEngine::NotchIntervalComponent, 116
 - wEngine::OffsetComponent, 119
 - wEngine::PaddingComponent, 122
 - wEngine::PositionComponent, 128
 - wEngine::RotationComponent, 130
 - wEngine::ScaleComponent, 133
 - wEngine::ThicknessComponent, 136
- debugPrintFonts
 - wEngine::AssetManager, 18
- disable
 - wEngine::Component, 32
- DiscontinuityComponent
 - wEngine::DiscontinuityComponent, 40
- Dotted
 - wEngine::LineStyleComponent, 107
- drawDataPlot
 - wPlot2D::DataPlotEntity, 38
- drawFunction
 - wPlot2D::FunctionEntity, 67
- drawLine
 - wEngine::LineDrawer, 97
- drawPolylineRound
 - wEngine::LineDrawer, 97
- enable
 - wEngine::Component, 32
- Entity
 - wEngine::Entity, 43
- FontComponent
 - wEngine::FontComponent, 48
- formatLabel
 - wPlot2D::LabelEntity, 85
- FrameEntity
 - wPlot2D::FrameEntity, 52
- FunctionComponent
 - wEngine::FunctionComponent, 57
- FunctionEntity
 - wPlot2D::FunctionEntity, 62
- getAngle
 - wEngine::RotationComponent, 130
- getArrowSize
 - wPlot2D::LineEntity, 104
- getCharacterSize
 - wPlot2D::LabelEntity, 83
 - wPlot2D::TitleEntity, 140
- getColor
 - wEngine::ColorComponent, 30
 - wPlot2D::DataPlotEntity, 36
 - wPlot2D::FunctionEntity, 62
- getComponent
 - wEngine::Entity, 45
- getComponentTypeID
 - wEngine, 12
- getDashLength
 - wEngine::LineStyleComponent, 107
 - wPlot2D::DataPlotEntity, 36
 - wPlot2D::FunctionEntity, 63
- getDecimalPlaces
 - wPlot2D::LabelEntity, 83
- getEndPoint
 - wPlot2D::LineEntity, 103
- getEntityID
 - wEngine::Entity, 43
- getExcludedIntervals
 - wEngine::DiscontinuityComponent, 41
- getExecutableDir
 - wEngine::PathUtils, 124
- getExecutablePath
 - wEngine::PathUtils, 124
- getFillColor
 - wPlot2D::FrameEntity, 53
- getFont
 - wEngine::AssetManager, 17
 - wEngine::FontComponent, 49
 - wPlot2D::GraphicsEntity, 74
- getFrameFillColor
 - wPlot2D::TitleEntity, 142
- getFrameOutlineColor
 - wPlot2D::TitleEntity, 142
- getFrameThickness
 - wPlot2D::TitleEntity, 142
- getGapLength
 - wEngine::LineStyleComponent, 108
 - wPlot2D::DataPlotEntity, 36
 - wPlot2D::FunctionEntity, 63
- getInterfaceComponent
 - wEngine::Entity, 46
- getInterval
 - wEngine::NotchIntervalComponent, 116
- getLabelsOffset
 - wPlot2D::AxisEntity, 26

- getLastPosition
 - wEngine::PositionComponent, 127
- getLength
 - wEngine::LengthComponent, 95
- getLineStyle
 - wPlot2D::DataPlotEntity, 36
 - wPlot2D::FunctionEntity, 63
- getNextComponentTypeID
 - wEngine, 12
- getOffset
 - wEngine::OffsetComponent, 119
 - wPlot2D::FunctionEntity, 63
 - wPlot2D::GraphicsEntity, 75
- getOrigin
 - wPlot2D::GraphicsEntity, 74
- getOutlineColor
 - wPlot2D::FrameEntity, 53
- getPadding
 - wEngine::PaddingComponent, 122
 - wPlot2D::FrameEntity, 54
 - wPlot2D::TitleEntity, 142
- getParent
 - wEngine::Component, 32
- getPosition
 - wEngine::PositionComponent, 127
 - wPlot2D::FunctionEntity, 62
- getRotation
 - wPlot2D::FunctionEntity, 64
- getScale
 - wEngine::ScaleComponent, 133
 - wPlot2D::GraphicsEntity, 75
- getStartPoint
 - wPlot2D::LineEntity, 103
- getStyle
 - wEngine::LineStyleComponent, 107
- getTextSize
 - wPlot2D::TitleEntity, 140
- getThickness
 - wEngine::ThicknessComponent, 136
 - wPlot2D::DataPlotEntity, 36
 - wPlot2D::FrameEntity, 54
 - wPlot2D::FunctionEntity, 62
 - wPlot2D::LineEntity, 102
- getTitleOffset
 - wPlot2D::AxisEntity, 24
- getValue
 - wPlot2D::LabelEntity, 83
- getWindow
 - wPlot2D::GraphicsEntity, 72
- getWindowSize
 - wPlot2D::GraphicsEntity, 72
- GraphicsEntity
 - wPlot2D::GraphicsEntity, 72
- hasArrow
 - wPlot2D::LineEntity, 103
- hasComponent
 - wEngine::Entity, 45
- isEnabled
 - wEngine::Component, 32
 - wPlot2D::FrameEntity, 53
- isFrameEnabled
 - wPlot2D::TitleEntity, 142
- isInExcludedInterval
 - wEngine::DiscontinuityComponent, 41
- LabelEntity
 - wPlot2D::LabelEntity, 82
- LegendEntity
 - wPlot2D::LegendEntity, 89
- LengthComponent
 - wEngine::LengthComponent, 94
- LineEntity
 - wPlot2D::LineEntity, 101
- LineStyle
 - wEngine::LineStyleComponent, 106
- LineStyleComponent
 - wEngine::LineStyleComponent, 107
- linspace
 - wEngine::MathUtils, 109
- LoadFont
 - wEngine::AssetManager, 16, 17
- main
 - main.cpp, 145
- main.cpp, 145
 - main, 145
- move
 - wEngine::PositionComponent, 127
- NotchEntity
 - wPlot2D::NotchEntity, 113
- NotchIntervalComponent
 - wEngine::NotchIntervalComponent, 115
- NotchPosition
 - wPlot2D, 13
- OffsetComponent
 - wEngine::OffsetComponent, 118
- operator=
 - wEngine::AssetManager, 16
- PaddingComponent
 - wEngine::PaddingComponent, 122
- PositionComponent
 - wEngine::PositionComponent, 126
- removeComponent
 - wEngine::Entity, 44
- RemoveFont
 - wEngine::AssetManager, 18
- render
 - wPlot2D::AxisEntity, 27
 - wPlot2D::FrameEntity, 55
 - wPlot2D::LabelEntity, 85
 - wPlot2D::LegendEntity, 92
 - wPlot2D::LineEntity, 104
 - wPlot2D::NotchEntity, 113

- wPlot2D::TitleEntity, 144
- requireComponent
 - wEngine::Entity, 45
- resetEntityIDCounter
 - wEngine::Entity, 44
- RotationComponent
 - wEngine::RotationComponent, 129
- saveToFile
 - wPlot2D::GraphicsEntity, 79
- ScaleComponent
 - wEngine::ScaleComponent, 132
- setAngle
 - wEngine::RotationComponent, 130
- setArrowSize
 - wPlot2D::AxisEntity, 23
 - wPlot2D::LineEntity, 104
- setBackgroundColor
 - wPlot2D::GraphicsEntity, 73
- setCharacterSize
 - wPlot2D::LabelEntity, 84
 - wPlot2D::LegendEntity, 92
 - wPlot2D::TitleEntity, 141
- setColor
 - wEngine::ColorComponent, 30
 - wPlot2D::AxisEntity, 22
 - wPlot2D::DataPlotEntity, 37
 - wPlot2D::FunctionEntity, 64
 - wPlot2D::LineEntity, 102
- setCustomLabels
 - wPlot2D::AxisEntity, 27
 - wPlot2D::LabelEntity, 84
- setDashLength
 - wEngine::LineStyleComponent, 108
 - wPlot2D::DataPlotEntity, 37
 - wPlot2D::FunctionEntity, 65
 - wPlot2D::LineEntity, 103
- setDecimalPlaces
 - wPlot2D::LabelEntity, 84
- setEnabled
 - wPlot2D::FrameEntity, 53
- setFillColor
 - wPlot2D::FrameEntity, 54
- setFont
 - wEngine::FontComponent, 49
 - wPlot2D::LabelEntity, 83
 - wPlot2D::LegendEntity, 91
 - wPlot2D::TitleEntity, 141
- setFrameEnabled
 - wPlot2D::LegendEntity, 90
 - wPlot2D::TitleEntity, 143
- setFrameFillColor
 - wPlot2D::LegendEntity, 90
 - wPlot2D::TitleEntity, 143
- setFrameOutlineColor
 - wPlot2D::LegendEntity, 91
 - wPlot2D::TitleEntity, 143
- setFrameThickness
 - wPlot2D::LegendEntity, 91
- wPlot2D::TitleEntity, 143
- setGapLength
 - wEngine::LineStyleComponent, 108
 - wPlot2D::DataPlotEntity, 38
 - wPlot2D::FunctionEntity, 65
 - wPlot2D::LineEntity, 103
- setInterval
 - wEngine::NotchIntervalComponent, 116
- setLabelsCharacterSize
 - wPlot2D::AxisEntity, 26
- setLabelsColor
 - wPlot2D::AxisEntity, 26
- setLabelsDecimalPlaces
 - wPlot2D::AxisEntity, 27
- setLabelsFont
 - wPlot2D::AxisEntity, 25
- setLabelsOffset
 - wPlot2D::AxisEntity, 26
- setLabelText
 - wPlot2D::LabelEntity, 84
- setLength
 - wEngine::LengthComponent, 95
- setLineStyle
 - wPlot2D::DataPlotEntity, 37
 - wPlot2D::FunctionEntity, 65
 - wPlot2D::LineEntity, 102
- setNotchesColor
 - wPlot2D::AxisEntity, 25
- setNotchesLength
 - wPlot2D::AxisEntity, 25
- setNotchesThickness
 - wPlot2D::AxisEntity, 25
- setOffset
 - wEngine::OffsetComponent, 119
 - wPlot2D::FunctionEntity, 66
 - wPlot2D::GraphicsEntity, 76
 - wPlot2D::TitleEntity, 141
- setOrigin
 - wPlot2D::GraphicsEntity, 74
- setOutlineColor
 - wPlot2D::FrameEntity, 54
- setPadding
 - wEngine::PaddingComponent, 122
 - wPlot2D::FrameEntity, 55
 - wPlot2D::LegendEntity, 91
 - wPlot2D::TitleEntity, 144
- setParent
 - wEngine::Component, 32
- setPosition
 - wEngine::PositionComponent, 127
 - wPlot2D::FunctionEntity, 64
- setRotation
 - wPlot2D::FunctionEntity, 66
- setScale
 - wEngine::ScaleComponent, 133
 - wPlot2D::FunctionEntity, 66
 - wPlot2D::GraphicsEntity, 75
- setStyle

- wEngine::LineStyleComponent, 107
- setTextColor
 - wPlot2D::LegendEntity, 92
 - wPlot2D::TitleEntity, 141
- setThickness
 - wEngine::ThicknessComponent, 136
 - wPlot2D::AxisEntity, 22
 - wPlot2D::DataPlotEntity, 37
 - wPlot2D::FrameEntity, 55
 - wPlot2D::FunctionEntity, 64
 - wPlot2D::LineEntity, 102
- setTitleCharacterSize
 - wPlot2D::AxisEntity, 24
- setTitleColor
 - wPlot2D::AxisEntity, 24
- setTitleFont
 - wPlot2D::AxisEntity, 23
- setTitleOffset
 - wPlot2D::AxisEntity, 24
- setWindowSize
 - wPlot2D::GraphicsEntity, 73
- setWindowTitle
 - wPlot2D::GraphicsEntity, 73
- Solid
 - wEngine::LineStyleComponent, 107
- ThicknessComponent
 - wEngine::ThicknessComponent, 135
- TitleAlignment
 - wPlot2D, 13
- TitleEntity
 - wPlot2D::TitleEntity, 140
- Top
 - wPlot2D, 13
- update
 - wPlot2D::FrameEntity, 55
- usesCustomLabels
 - wPlot2D::LabelEntity, 85
- wAssetManager.cpp, 212
- wAssetManager.hpp, 213, 214
- wAxisEntity.cpp, 185
- wAxisEntity.hpp, 185, 186
- wColorComponent.cpp, 146
- wColorComponent.hpp, 146, 147
- wComponent.cpp, 178
- wComponent.hpp, 179, 180
- wDataPlotEntity.cpp, 188
- wDataPlotEntity.hpp, 188, 190
- wDiscontinuityComponent.cpp, 148
- wDiscontinuityComponent.hpp, 149, 150
- wEngine, 11
 - getComponentTypeID, 12
 - getNextComponentTypeID, 12
- wEngine::AssetManager, 15
 - ~AssetManager, 16
 - AssetManager, 16
 - debugPrintFonts, 18
 - getFont, 17
 - LoadFont, 16, 17
 - operator=, 16
 - RemoveFont, 18
- wEngine::ColorComponent, 28
 - ~ColorComponent, 30
 - ColorComponent, 29
 - debugPrint, 30
 - getColor, 30
 - setColor, 30
- wEngine::Component, 31
 - ~Component, 31
 - Component, 31
 - disable, 32
 - enable, 32
 - getParent, 32
 - isEnabled, 32
 - setParent, 32
- wEngine::DiscontinuityComponent, 38
 - ~DiscontinuityComponent, 40
 - addExcludedInterval, 41
 - clearExcludedIntervals, 41
 - debugPrint, 41
 - DiscontinuityComponent, 40
 - getExcludedIntervals, 41
 - isInExcludedInterval, 41
- wEngine::Entity, 42
 - ~Entity, 43
 - addComponent, 44
 - clearComponents, 43
 - Entity, 43
 - getComponent, 45
 - getEntityID, 43
 - getInterfaceComponent, 46
 - hasComponent, 45
 - removeComponent, 44
 - requireComponent, 45
 - resetEntityIDCounter, 44
- wEngine::FontComponent, 47
 - ~FontComponent, 49
 - debugPrint, 49
 - FontComponent, 48
 - getFont, 49
 - setFont, 49
- wEngine::FunctionComponent, 56
 - ~FunctionComponent, 58
 - calculate, 58
 - debugPrint, 58
 - FunctionComponent, 57
- wEngine::LengthComponent, 93
 - ~LengthComponent, 95
 - debugPrint, 95
 - getLength, 95
 - LengthComponent, 94
 - setLength, 95
- wEngine::LineDrawer, 96
 - drawLine, 97
 - drawPolylineRound, 97

- wEngine::LineStyleComponent, 105
 - ~LineStyleComponent, 107
 - Dashed, 107
 - debugPrint, 109
 - Dotted, 107
 - getDashLength, 107
 - getGapLength, 108
 - getStyle, 107
 - LineStyle, 106
 - LineStyleComponent, 107
 - setDashLength, 108
 - setGapLength, 108
 - setStyle, 107
 - Solid, 107
- wEngine::MathUtils, 109
 - linspace, 109
- wEngine::NotchIntervalComponent, 114
 - ~NotchIntervalComponent, 116
 - debugPrint, 116
 - getInterval, 116
 - NotchIntervalComponent, 115
 - setInterval, 116
- wEngine::OffsetComponent, 117
 - ~OffsetComponent, 119
 - addOffset, 119
 - debugPrint, 119
 - getOffset, 119
 - OffsetComponent, 118
 - setOffset, 119
- wEngine::PaddingComponent, 120
 - ~PaddingComponent, 122
 - debugPrint, 122
 - getPadding, 122
 - PaddingComponent, 122
 - setPadding, 122
- wEngine::PathUtils, 123
 - getExecutableDir, 124
 - getExecutablePath, 124
- wEngine::PositionComponent, 125
 - ~PositionComponent, 127
 - debugPrint, 128
 - getLastPosition, 127
 - getPosition, 127
 - move, 127
 - PositionComponent, 126
 - setPosition, 127
- wEngine::RotationComponent, 128
 - ~RotationComponent, 130
 - debugPrint, 130
 - getAngle, 130
 - RotationComponent, 129
 - setAngle, 130
- wEngine::ScaleComponent, 131
 - ~ScaleComponent, 133
 - debugPrint, 133
 - getScale, 133
 - ScaleComponent, 132
 - setScale, 133
- wEngine::ThicknessComponent, 134
 - ~ThicknessComponent, 136
 - debugPrint, 136
 - getThickness, 136
 - setThickness, 136
 - ThicknessComponent, 135
- wEntity.cpp, 181
- wEntity.hpp, 181, 183
- wFontComponent.cpp, 150
- wFontComponent.hpp, 151, 152
- wFrameEntity.cpp, 190
- wFrameEntity.hpp, 191, 192
- wFunctionComponent.cpp, 153
- wFunctionComponent.hpp, 154, 155
- wFunctionEntity.cpp, 193
- wFunctionEntity.hpp, 194, 195
- wGraphicsEntity.cpp, 196
- wGraphicsEntity.hpp, 196, 198
- wLabelEntity.cpp, 199
- wLabelEntity.hpp, 200, 201
- wLegendEntity.cpp, 202
- wLegendEntity.hpp, 203, 204
- wLengthComponent.cpp, 155
- wLengthComponent.hpp, 156, 157
- wLineDrawer.cpp, 215
- wLineDrawer.hpp, 216, 217
- wLineEntity.cpp, 205
- wLineEntity.hpp, 205, 206
- wLineStyleComponent.cpp, 158
- wLineStyleComponent.hpp, 159, 160
- wMathUtils.cpp, 217
- wMathUtils.hpp, 218, 219
- wNotchEntity.cpp, 207
- wNotchEntity.hpp, 208, 209
- wNotchIntervalComponent.cpp, 160
- wNotchIntervalComponent.hpp, 161, 162
- wOffsetComponent.cpp, 163
- wOffsetComponent.hpp, 164
- wPaddingComponent.cpp, 165
- wPaddingComponent.hpp, 166, 167
- wPathUtils.cpp, 220
- wPathUtils.hpp, 220, 221
- wPlot2D, 12
 - Above, 13
 - AxisType, 13
 - Below, 13
 - Bottom, 13
 - Center, 13
 - NotchPosition, 13
 - TitleAlignment, 13
 - Top, 13
 - X_AXIS, 13
 - Y_AXIS, 13
- wPlot2D - ECS-Based 2D Plotting Engine, 1
- wPlot2D.hpp, 222
- wPlot2D::AxisEntity, 19
 - ~AxisEntity, 22
 - addLabelsOffset, 26

- addNotches, 24
- addTitle, 23
- AxisEntity, 22
- getLabelsOffset, 26
- getTitleOffset, 24
- render, 27
- setArrowSize, 23
- setColor, 22
- setCustomLabels, 27
- setLabelsCharacterSize, 26
- setLabelsColor, 26
- setLabelsDecimalPlaces, 27
- setLabelsFont, 25
- setLabelsOffset, 26
- setNotchesColor, 25
- setNotchesLength, 25
- setNotchesThickness, 25
- setThickness, 22
- setTitleCharacterSize, 24
- setTitleColor, 24
- setTitleFont, 23
- setTitleOffset, 24
- wPlot2D::DataPlotEntity, 33
 - ~DataPlotEntity, 35
 - DataPlotEntity, 35
 - drawDataPlot, 38
 - getColor, 36
 - getDashLength, 36
 - getGapLength, 36
 - getLineStyle, 36
 - getThickness, 36
 - setColor, 37
 - setDashLength, 37
 - setGapLength, 38
 - setLineStyle, 37
 - setThickness, 37
- wPlot2D::FrameEntity, 50
 - ~FrameEntity, 53
 - FrameEntity, 52
 - getFillColor, 53
 - getOutlineColor, 53
 - getPadding, 54
 - getThickness, 54
 - isEnabled, 53
 - render, 55
 - setEnabled, 53
 - setFillColor, 54
 - setOutlineColor, 54
 - setPadding, 55
 - setThickness, 55
 - update, 55
- wPlot2D::FunctionEntity, 59
 - ~FunctionEntity, 62
 - addExcludedInterval, 67
 - alignToYAxis, 67
 - clearExcludedIntervals, 67
 - drawFunction, 67
 - FunctionEntity, 62
 - getColor, 62
 - getDashLength, 63
 - getGapLength, 63
 - getLineStyle, 63
 - getOffset, 63
 - getPosition, 62
 - getRotation, 64
 - getThickness, 62
 - setColor, 64
 - setDashLength, 65
 - setGapLength, 65
 - setLineStyle, 65
 - setOffset, 66
 - setPosition, 64
 - setRotation, 66
 - setScale, 66
 - setThickness, 64
- wPlot2D::GraphicsEntity, 68
 - ~GraphicsEntity, 72
 - addAxis, 76
 - addDataPlot, 77
 - addFont, 73
 - addFunction, 77
 - addLegend, 77
 - addLine, 78
 - addText, 78
 - addTitle, 76
 - getFont, 74
 - getOffset, 75
 - getOrigin, 74
 - getScale, 75
 - getWindow, 72
 - getWindowSize, 72
 - GraphicsEntity, 72
 - saveToFile, 79
 - setBackgroundColor, 73
 - setOffset, 76
 - setOrigin, 74
 - setScale, 75
 - setWindowSize, 73
 - setWindowTitle, 73
- wPlot2D::LabelEntity, 79
 - ~LabelEntity, 82
 - formatLabel, 85
 - getCharacterSize, 83
 - getDecimalPlaces, 83
 - getValue, 83
 - LabelEntity, 82
 - render, 85
 - setCharacterSize, 84
 - setCustomLabels, 84
 - setDecimalPlaces, 84
 - setFont, 83
 - setLabelText, 84
 - usesCustomLabels, 85
- wPlot2D::LegendEntity, 86
 - ~LegendEntity, 89
 - addItem, 89, 90

- LegendEntity, 89
- render, 92
- setCharacterSize, 92
- setFont, 91
- setFrameEnabled, 90
- setFrameFillColor, 90
- setFrameOutlineColor, 91
- setFrameThickness, 91
- setPadding, 91
- setTextColor, 92
- wPlot2D::LineEntity, 98
 - ~LineEntity, 102
 - getArrowSize, 104
 - getEndPoint, 103
 - getStartPoint, 103
 - getThickness, 102
 - hasArrow, 103
 - LineEntity, 101
 - render, 104
 - setArrowSize, 104
 - setColor, 102
 - setDashLength, 103
 - setGapLength, 103
 - setLineStyle, 102
 - setThickness, 102
- wPlot2D::NotchEntity, 110
 - ~NotchEntity, 113
 - NotchEntity, 113
 - render, 113
- wPlot2D::TitleEntity, 137
 - ~TitleEntity, 140
 - getCharacterSize, 140
 - getFrameFillColor, 142
 - getFrameOutlineColor, 142
 - getFrameThickness, 142
 - getPadding, 142
 - getTextSize, 140
 - isFrameEnabled, 142
 - render, 144
 - setCharacterSize, 141
 - setFont, 141
 - setFrameEnabled, 143
 - setFrameFillColor, 143
 - setFrameOutlineColor, 143
 - setFrameThickness, 143
 - setOffset, 141
 - setPadding, 144
 - setTextColor, 141
 - TitleEntity, 140
- wPositionComponent.cpp, 168
- wPositionComponent.hpp, 169, 170
- wRotationComponent.cpp, 170
- wRotationComponent.hpp, 172, 173
- wScaleComponent.cpp, 173
- wScaleComponent.hpp, 174, 175
- wThicknessComponent.cpp, 176
- wThicknessComponent.hpp, 177, 178
- wTitleEntity.cpp, 210
- wTitleEntity.hpp, 210, 211
- X_AXIS
 - wPlot2D, 13
- Y_AXIS
 - wPlot2D, 13